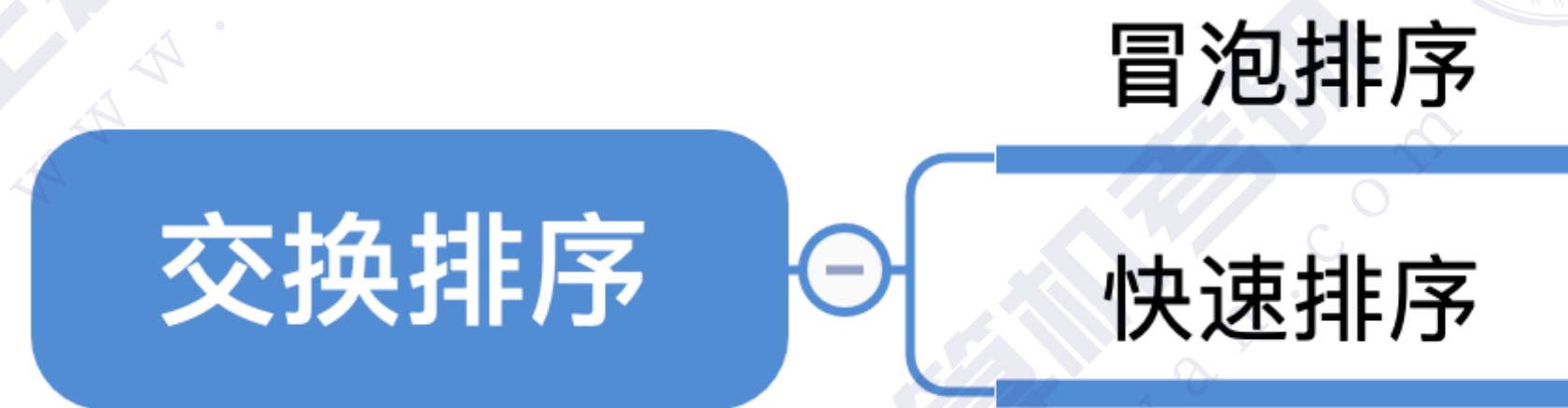


本节内容

冒泡排序

知识总览



基于“交换”的排序：根据序列中两个元素关键字的比较结果来对换这两个记录在序列中的位置

冒泡?



冒泡排序



从后往前（或从前往后）两两比较相邻元素的值，若为逆序（即 $A[i-1] > A[i]$ ），则交换它们，直到序列比较完。称这样过程为“一趟”冒泡排序。

49	38	65	97	76	13	27	<u>49</u>
0	1	2	3	4	5	6	7

冒泡排序



从后往前（或从前往后）两两比较相邻元素的值，若为逆序（即 $A[i-1] > A[i]$ ），则交换它们，直到序列比较完。称这样过程为“一趟”冒泡排序。

第1趟:

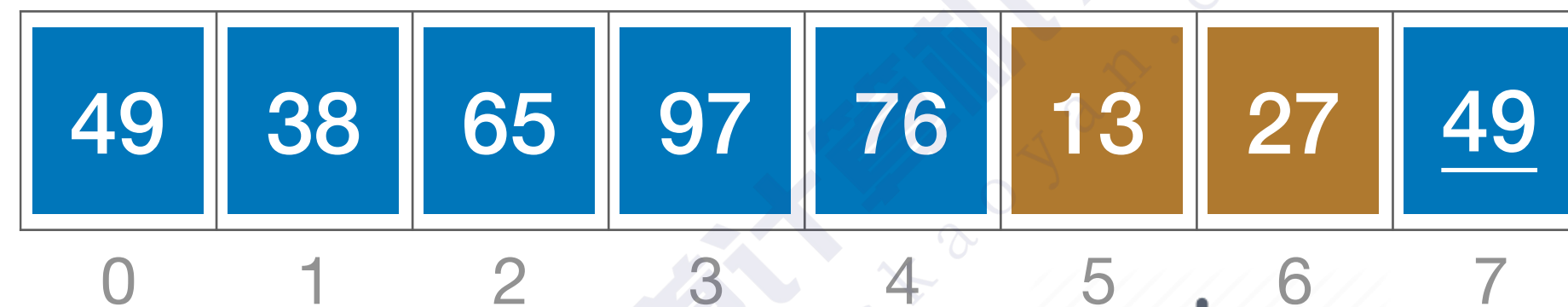
49	38	65	97	76	13	27	49
0	1	2	3	4	5	6	7



冒泡排序



第1趟:



冒泡排序



第1趟:

49	38	65	97	76	13	27	<u>49</u>
0	1	2	3	4	5	6	7



冒泡排序



第1趟:

49	38	65	97	13	76	27	<u>49</u>
0	1	2	3	4	5	6	7



冒泡排序



第1趟:

49	38	65	97	13	76	27	<u>49</u>
0	1	2	3	4	5	6	7



冒泡排序



第1趟:

49	38	65	13	97	76	27	<u>49</u>
0	1	2	3	4	5	6	7



冒泡排序



第1趟:

49	38	65	13	97	76	27	<u>49</u>
0	1	2	3	4	5	6	7



冒泡排序



第1趟:

49	38	13	65	97	76	27	<u>49</u>
0	1	2	3	4	5	6	7



冒泡排序



第1趟:

49	38	13	65	97	76	27	<u>49</u>
----	----	----	----	----	----	----	-----------

0 1 2 3 4 5 6 7



冒泡排序



第1趟:

49	13	38	65	97	76	27	<u>49</u>
----	----	----	----	----	----	----	-----------

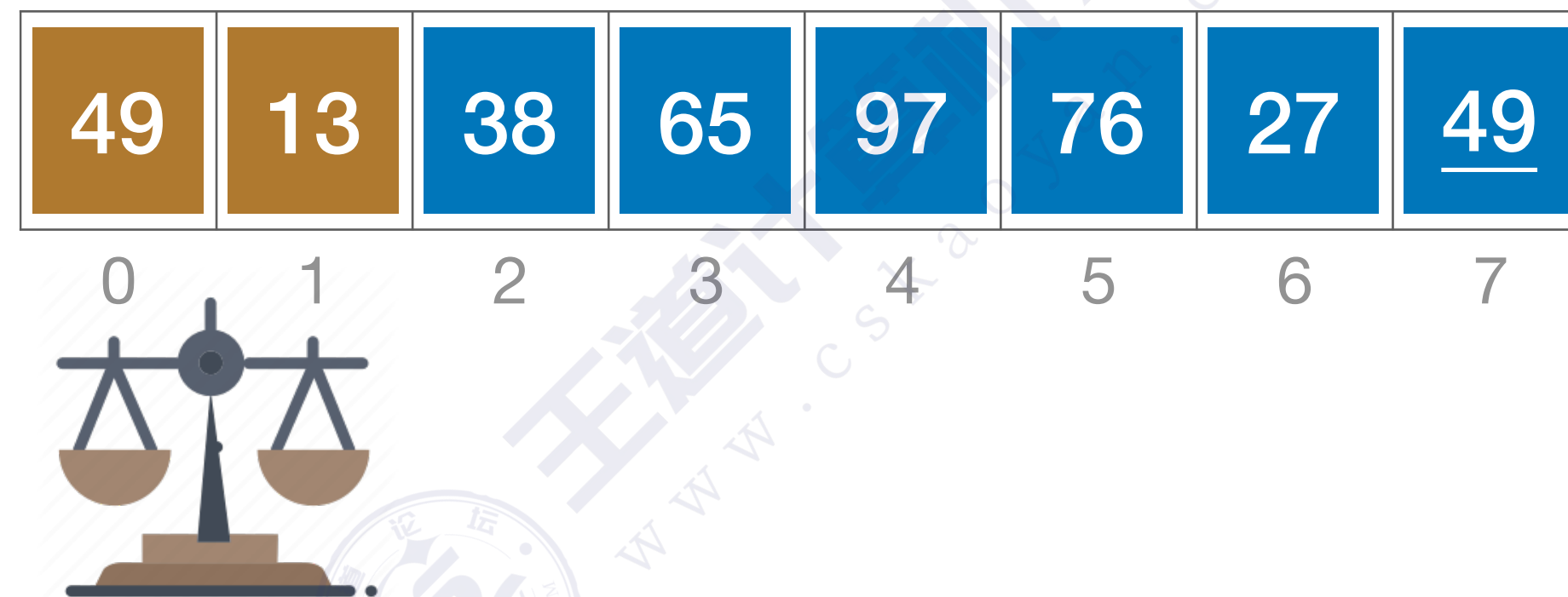
0 1 2 3 4 5 6 7



冒泡排序



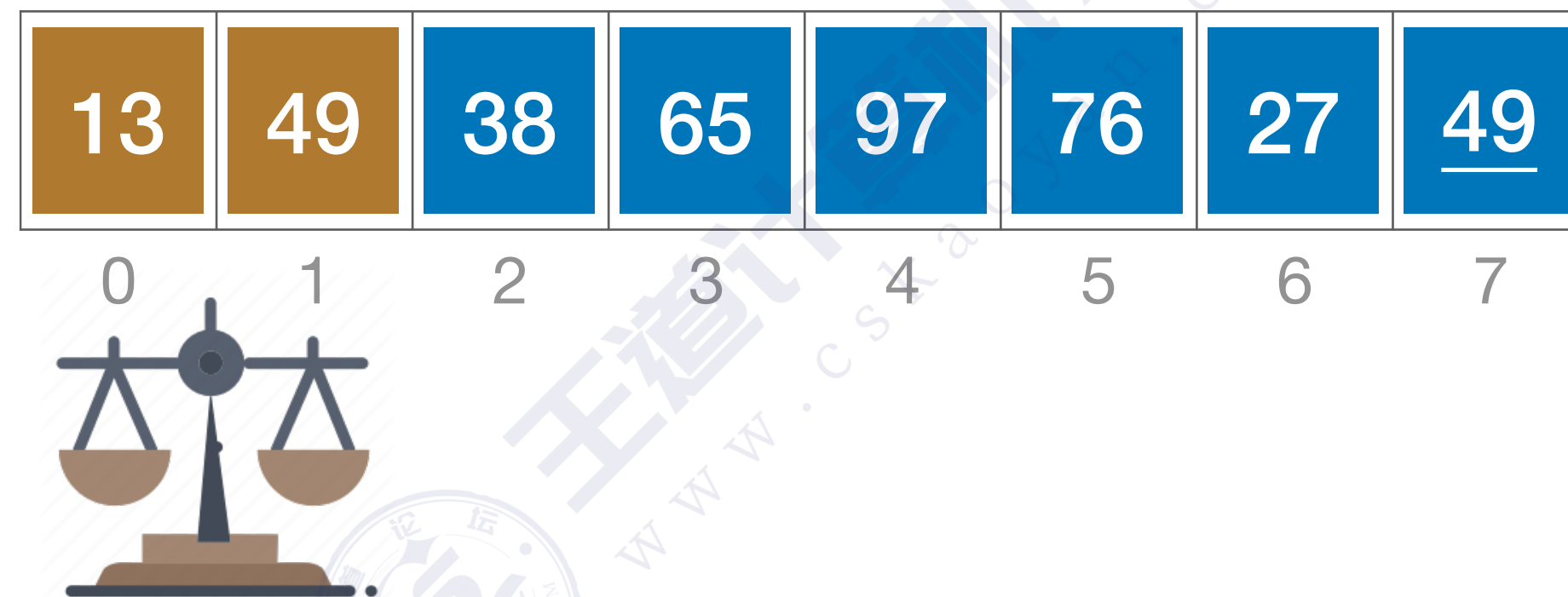
第1趟:



冒泡排序



第1趟:



冒泡排序



第1趟结束:

13	49	38	65	97	76	27	49
0	1	2	3	4	5	6	7

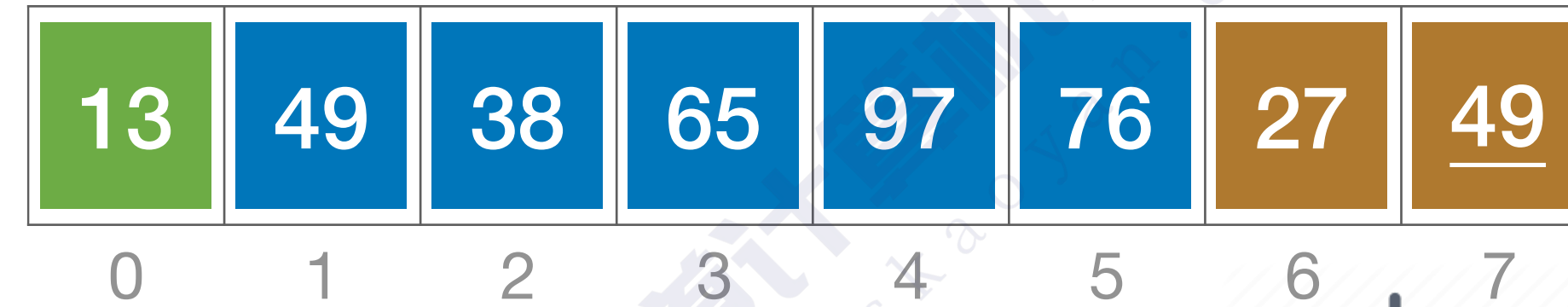
第一趟排序使关键字值最小的一个元素“冒”到最前面

冒泡排序



从后往前（或从前往后）两两比较相邻元素的值，若为逆序（即 $A[i-1] > A[i]$ ），则交换它们，直到序列比较完。称这样过程为“一趟”冒泡排序。

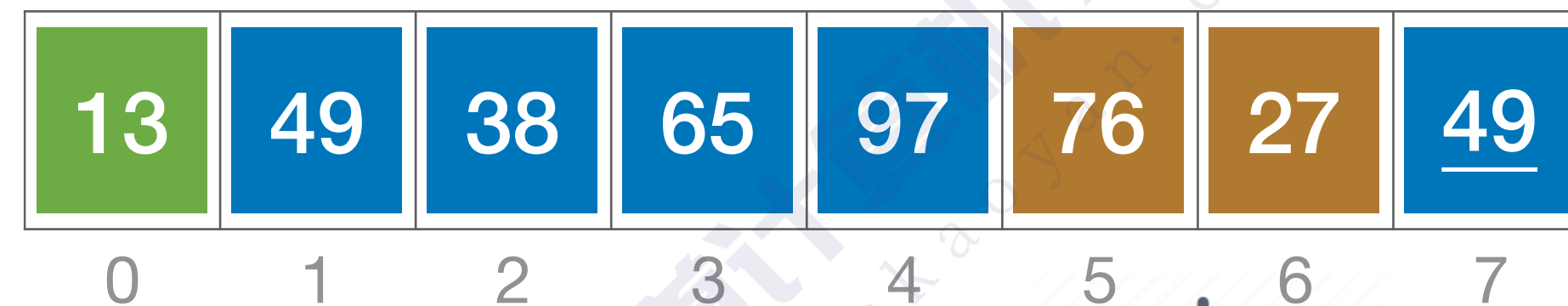
第2趟:



冒泡排序



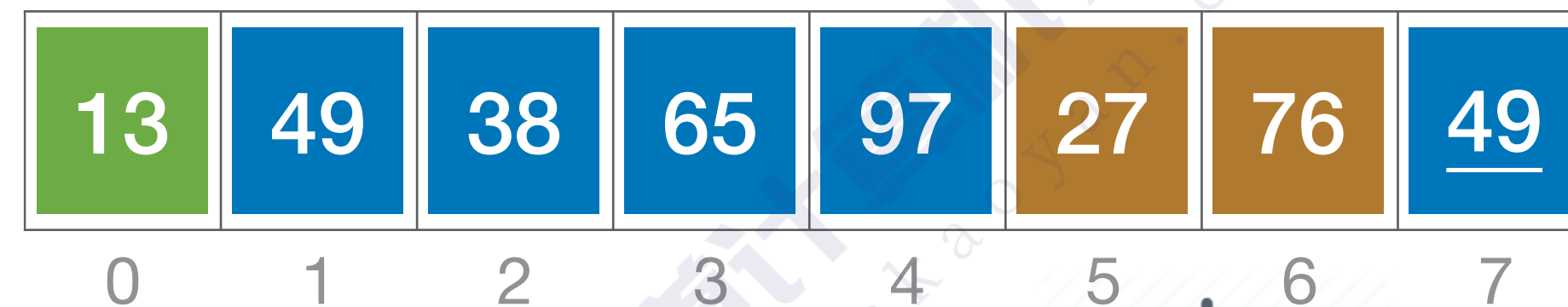
第2趟:



冒泡排序



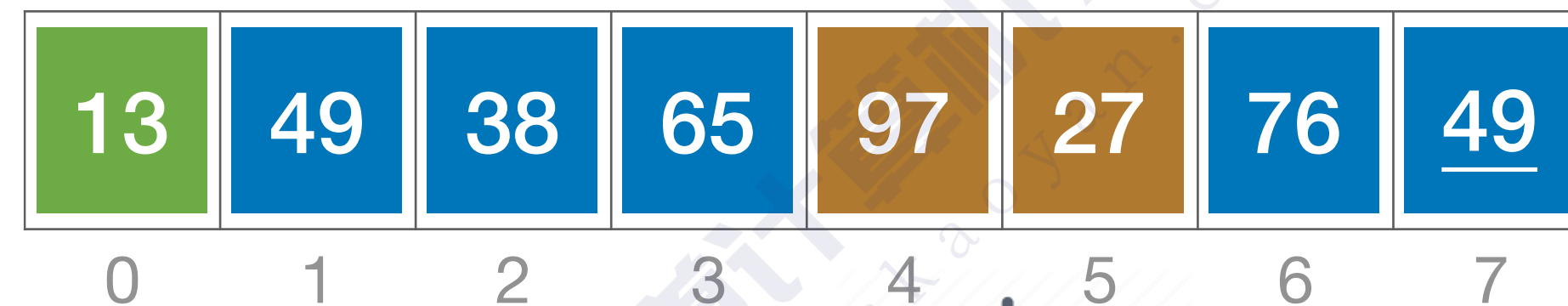
第2趟:



冒泡排序



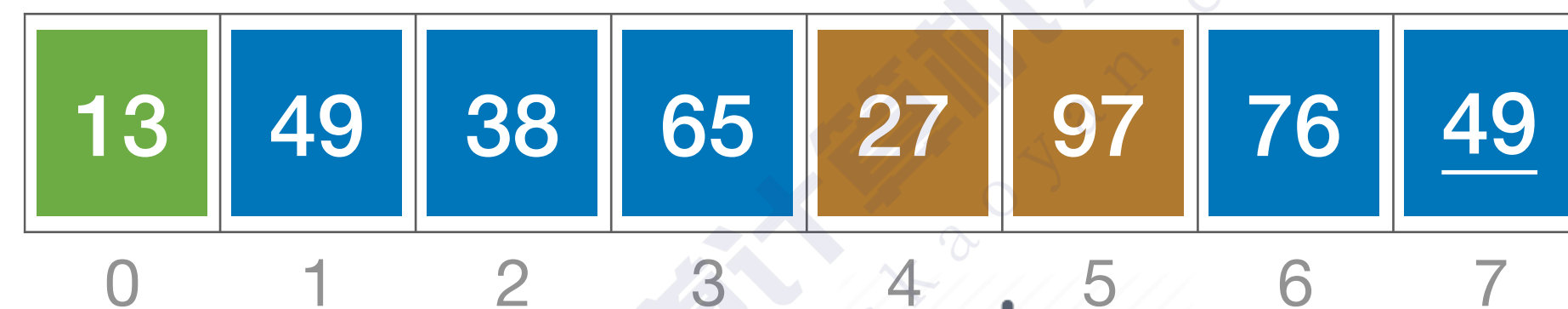
第2趟:



冒泡排序



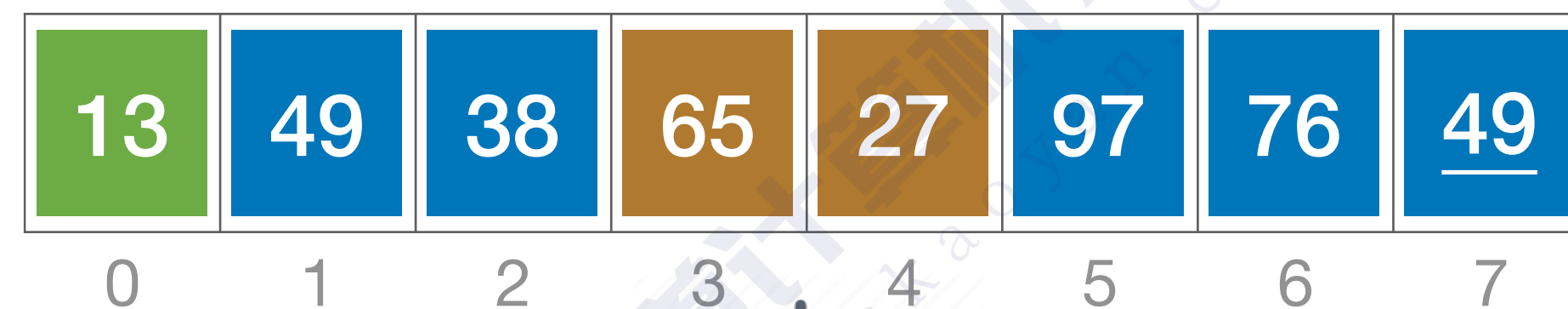
第2趟:



冒泡排序



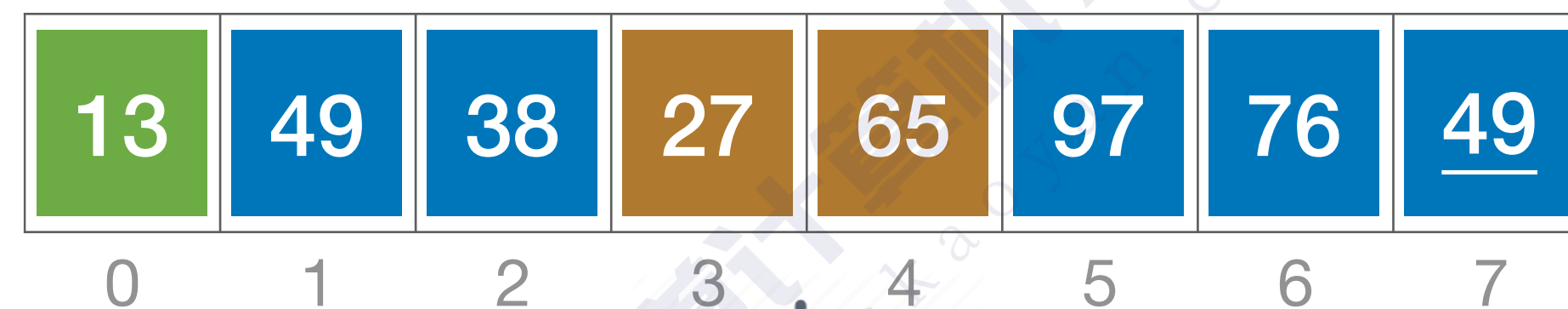
第2趟:



冒泡排序



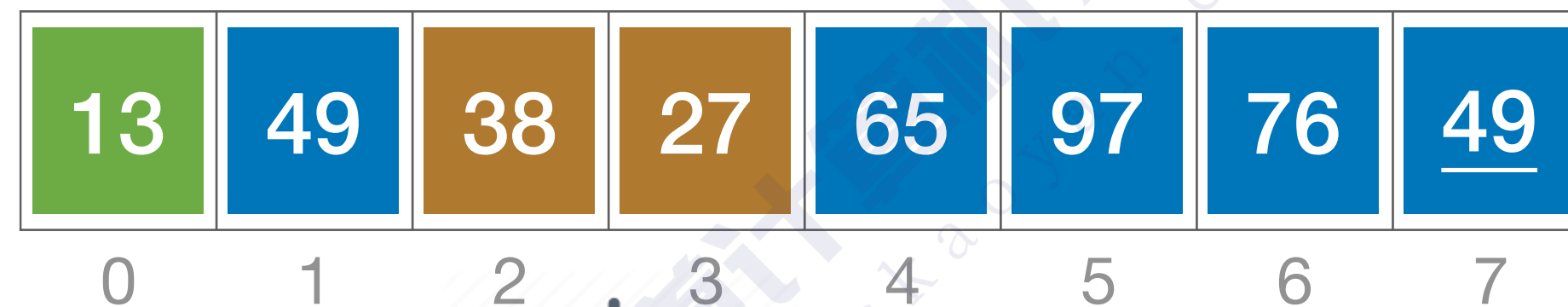
第2趟:



冒泡排序



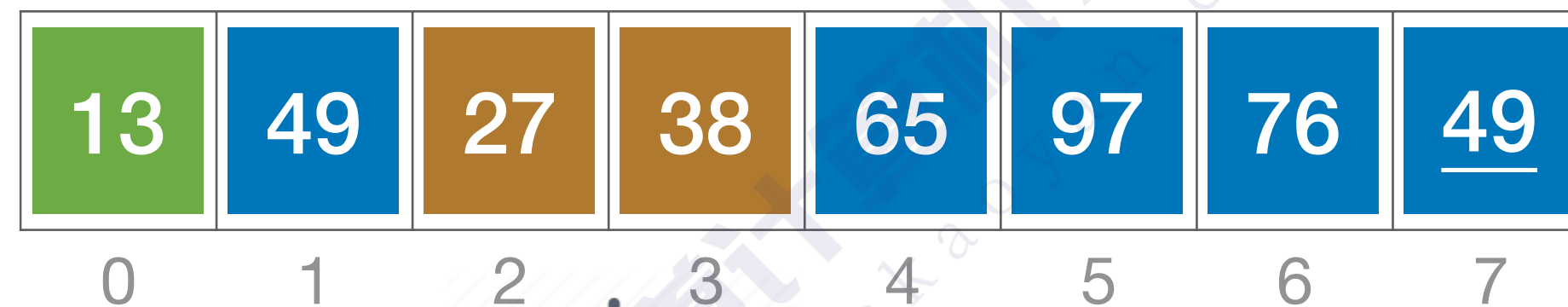
第2趟:



冒泡排序



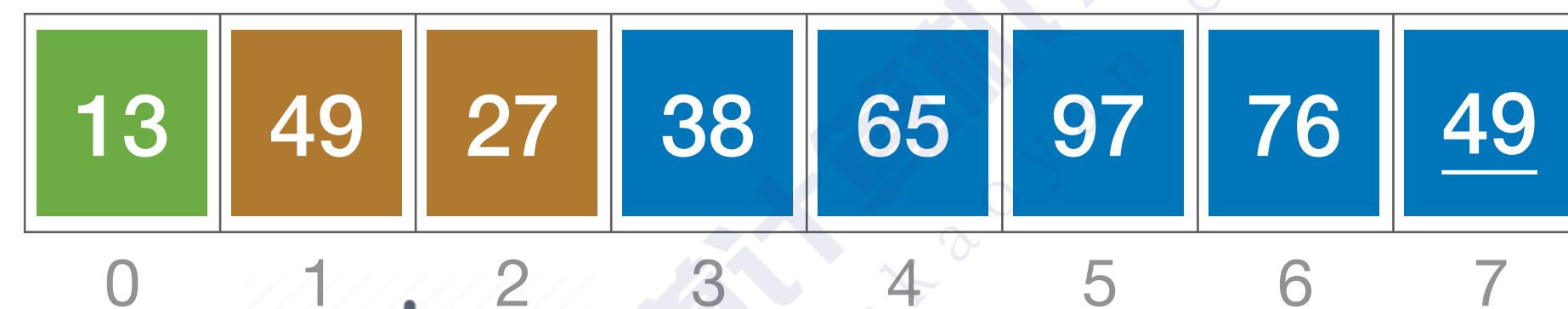
第2趟:



冒泡排序



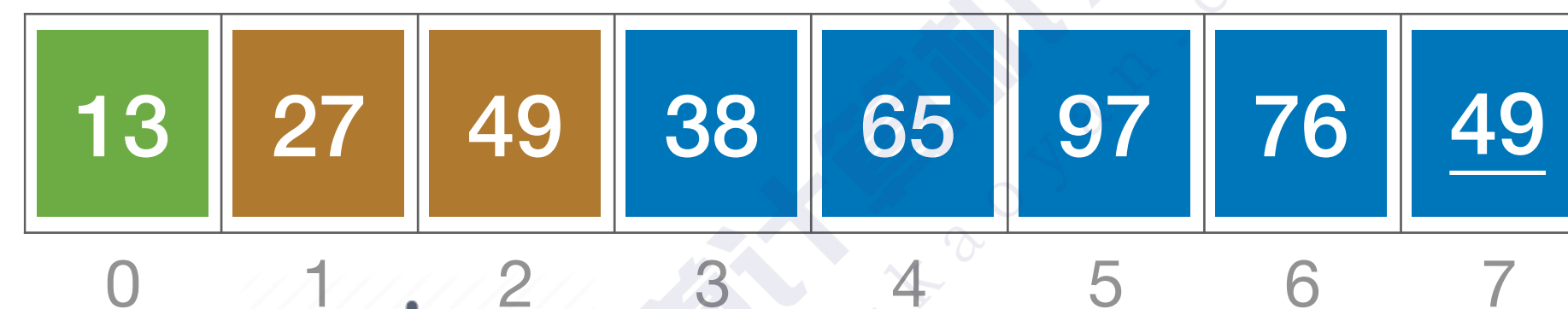
第2趟:



冒泡排序



第2趟:



前边已经确定最终位置
的元素不用再对比

冒泡排序



第2趟结束:

13	27	49	38	65	97	76	<u>49</u>
0	1	2	3	4	5	6	7

第2趟结束后，最小的两个元素会“冒”到最前边

冒泡排序



从后往前（或从前往后）两两比较相邻元素的值，若为逆序（即 $A[i-1] > A[i]$ ），则交换它们，直到序列比较完。称这样过程为“一趟”冒泡排序。

第3趟：

13	27	49	38	65	97	76	<u>49</u>
0	1	2	3	4	5	6	7



冒泡排序



第3趟:

13	27	49	38	65	97	<u>49</u>	76
0	1	2	3	4	5	6	7



冒泡排序



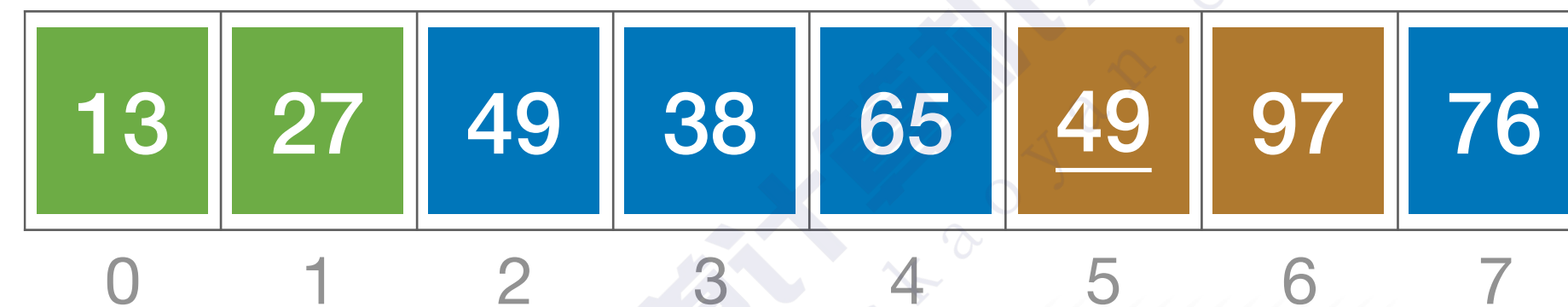
第3趟:



冒泡排序



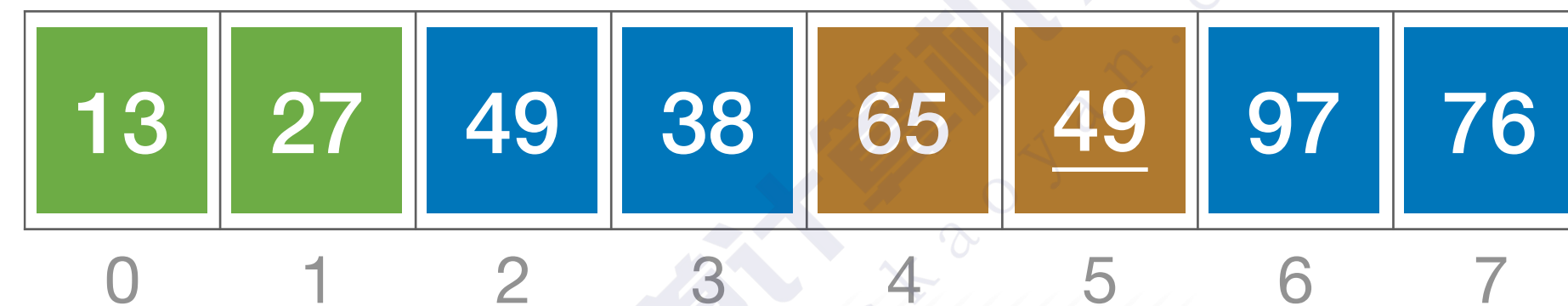
第3趟:



冒泡排序



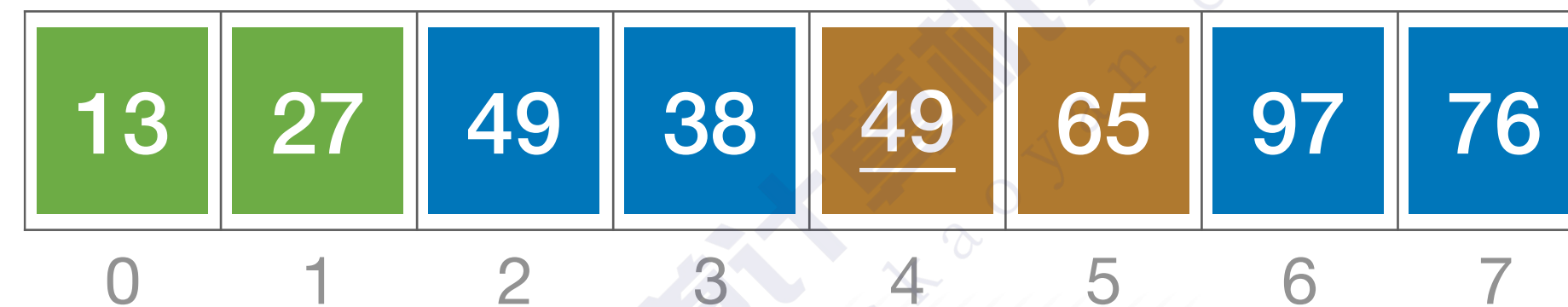
第3趟:



冒泡排序



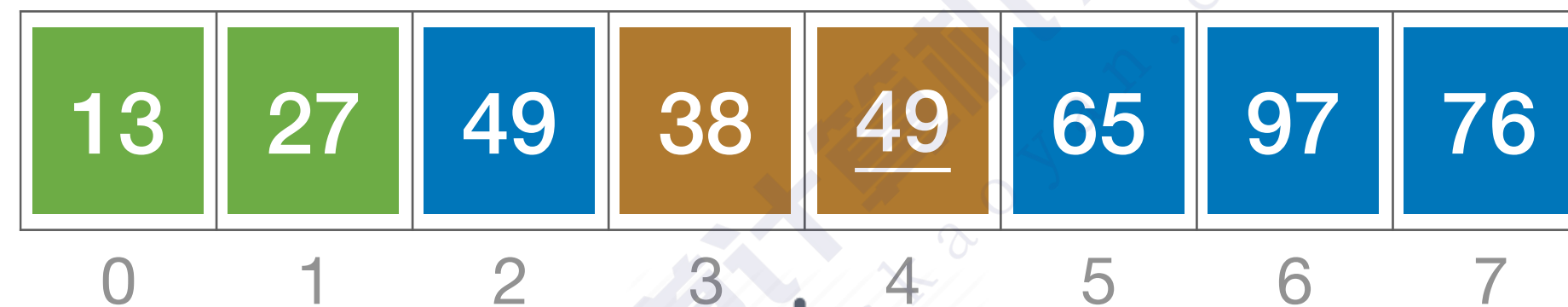
第3趟:



冒泡排序



第3趟:



冒泡排序



第3趟:

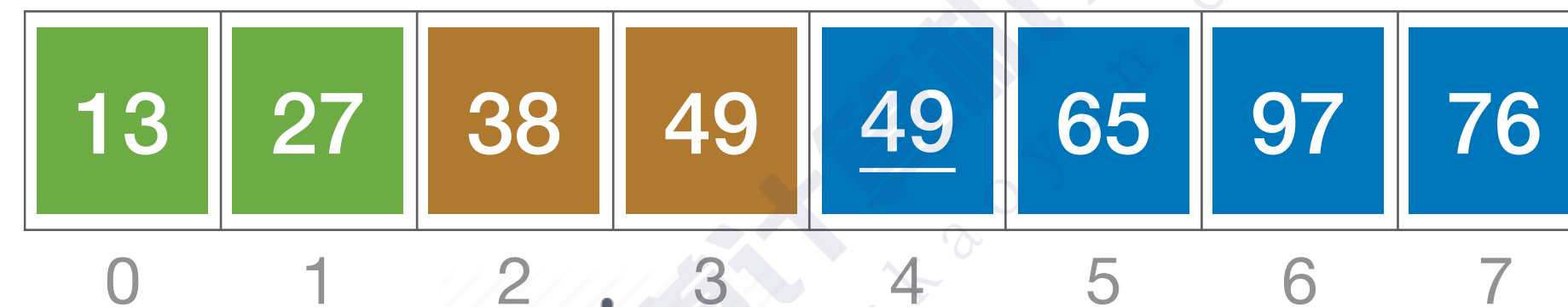
13	27	49	38	<u>49</u>	65	97	76
0	1	2	3	4	5	6	7



冒泡排序



第3趟:



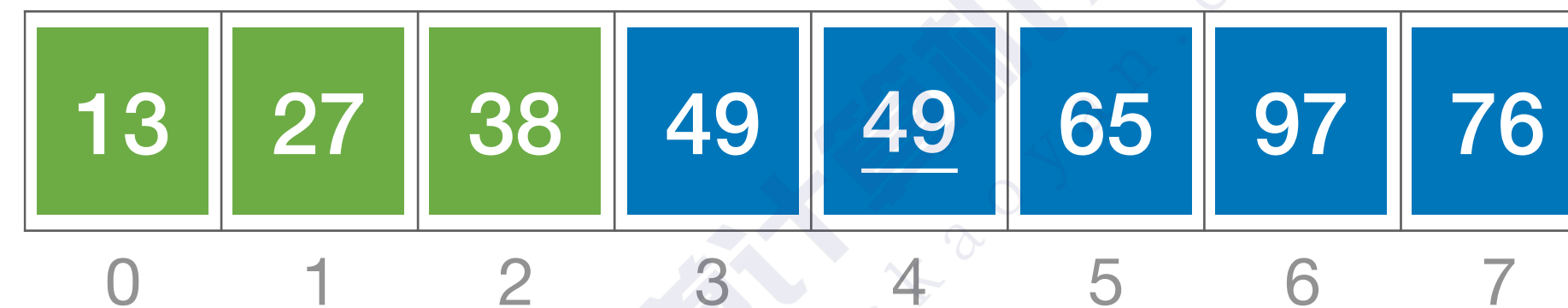
前边已经确定最终位置
的元素不用再对比



冒泡排序



第3趟结束:



第3趟结束后，最小的3个元素会“冒”到最前边

冒泡排序



从后往前（或从前往后）两两比较相邻元素的值，若为逆序（即 $A[i-1] > A[i]$ ），则交换它们，直到序列比较完。称这样过程为“一趟”冒泡排序。

第4趟:

13	27	38	49	<u>49</u>	65	97	76
0	1	2	3	4	5	6	7



冒泡排序



第4趟:

13	27	38	49	<u>49</u>	65	76	97
0	1	2	3	4	5	6	7



冒泡排序



第4趟:

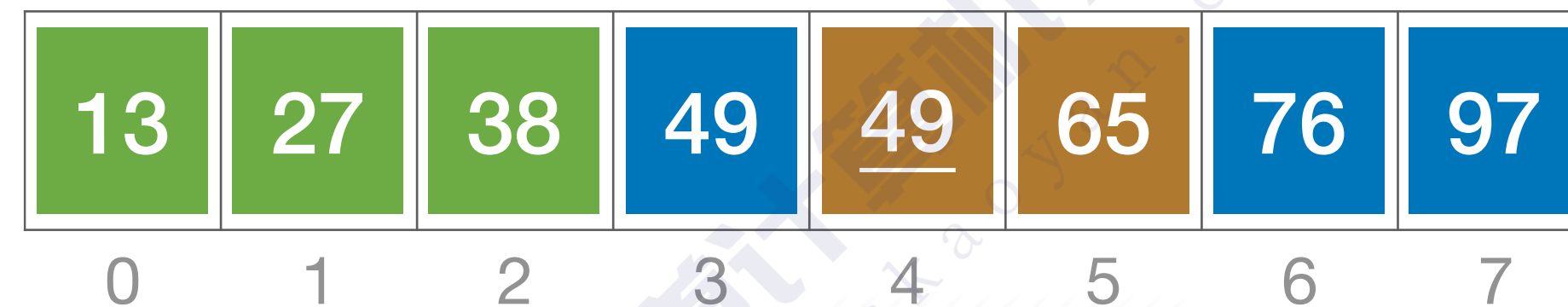
13	27	38	49	<u>49</u>	65	76	97
0	1	2	3	4	5	6	7



冒泡排序



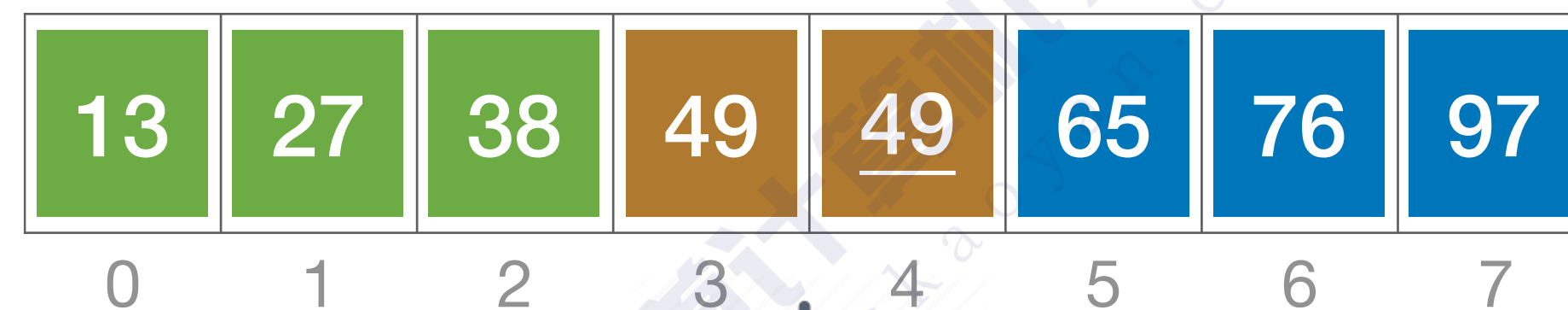
第4趟:



冒泡排序



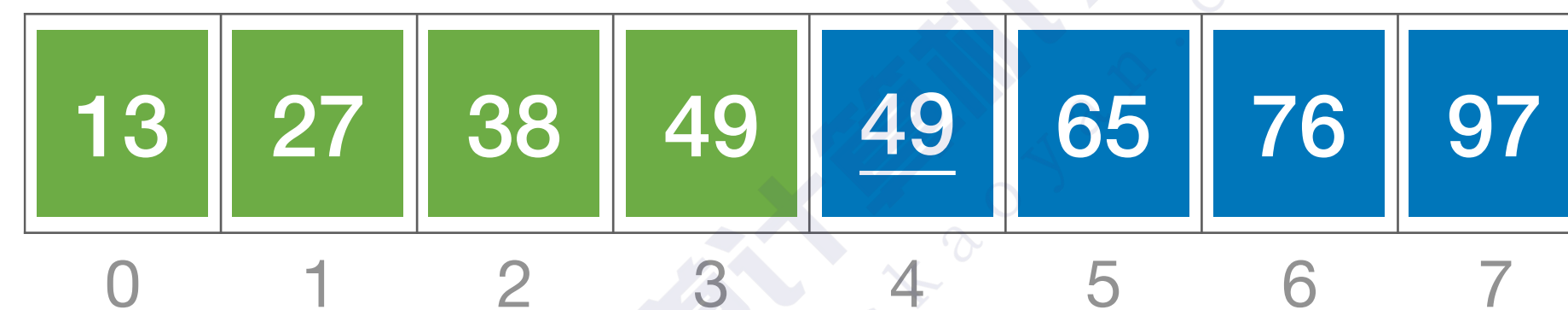
第4趟:



冒泡排序



第4趟结束:

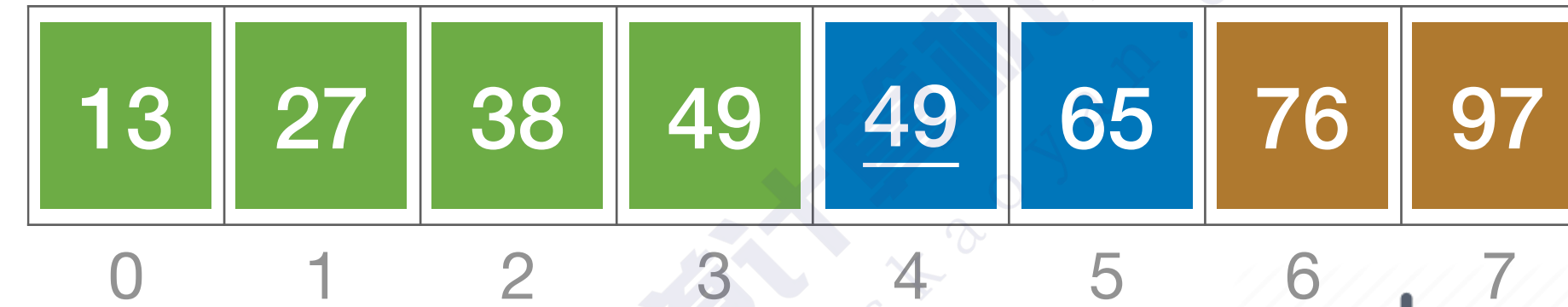


冒泡排序



从后往前（或从前往后）两两比较相邻元素的值，若为逆序（即 $A[i-1] > A[i]$ ），则交换它们，直到序列比较完。称这样过程为“一趟”冒泡排序。

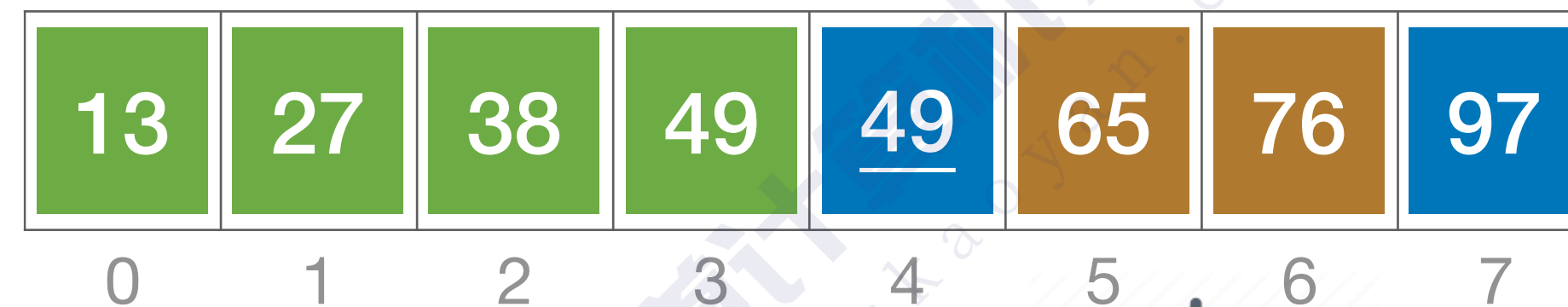
第5趟:



冒泡排序



第5趟:

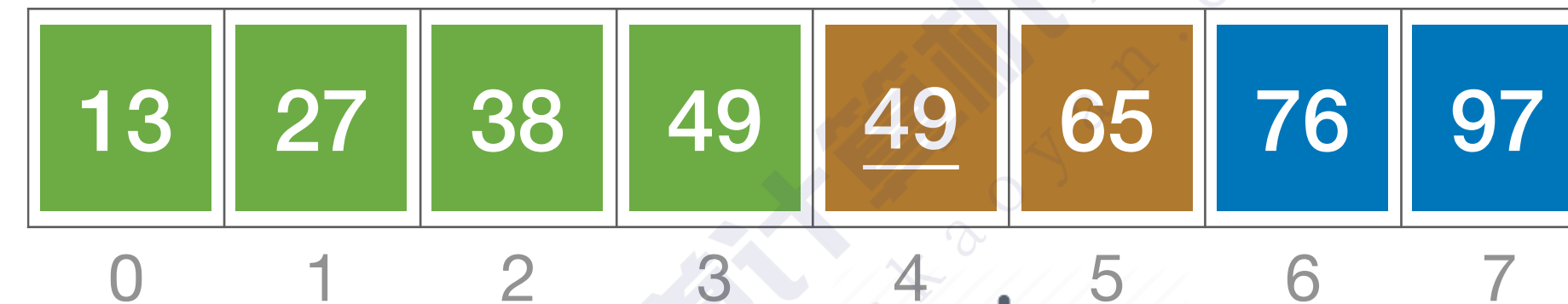


冒泡排序



从后往前（或从前往后）两两比较相邻元素的值，若为逆序（即 $A[i-1] > A[i]$ ），则交换它们，直到序列比较完。称这样过程为“一趟”冒泡排序。总共需进行 $n-1$ 趟冒泡。

第5趟:



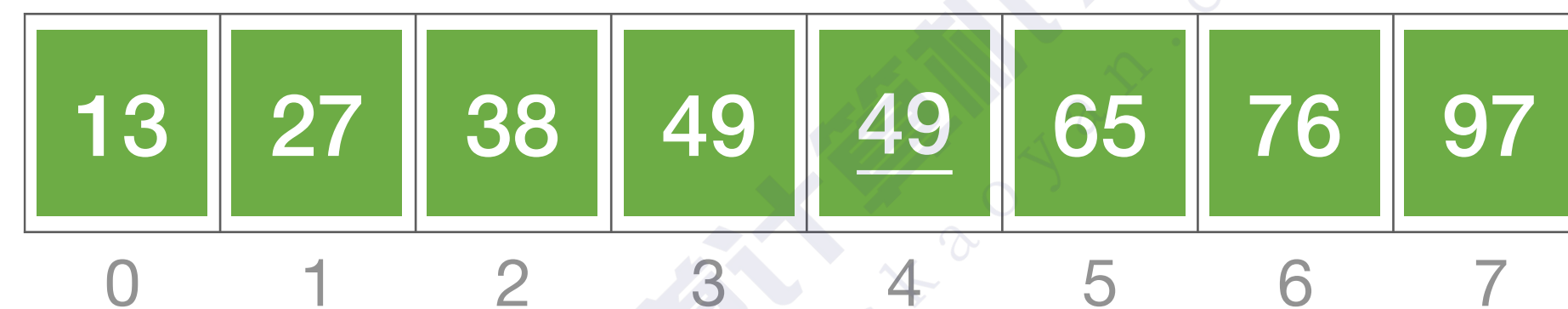
若某一趟排序没有发生“交换”，
说明此时已经整体有序。



冒泡排序



第5趟结束:



算法实现

//交换

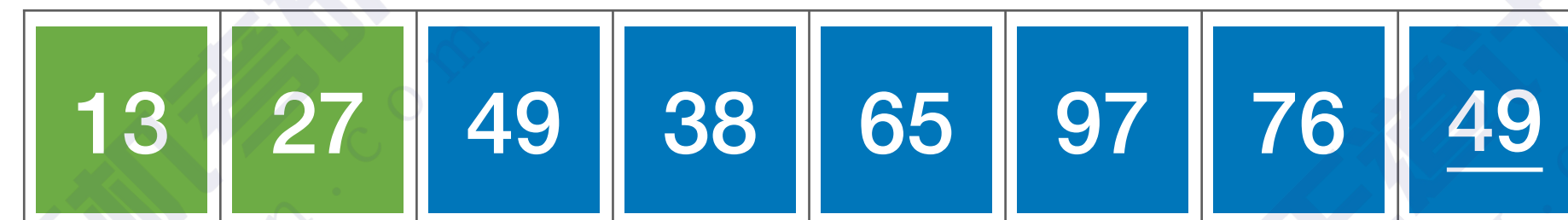
```
void swap(int &a, int &b){  
    int temp = a;  
    a = b;  
    b = temp;  
}
```

//冒泡排序

```
void BubbleSort(int A[], int n){  
    for(int i=0; i<n-1; i++){  
        bool flag=false;  
        for(int j=n-1; j>i; j--){  
            if(A[j-1]>A[j]){  
                swap(A[j-1], A[j]);  
                flag=true;  
            }  
        }  
        if(flag==false)  
            return;  
    }  
}
```

//表示本趟冒泡是否发生交换的标志
//一趟冒泡过程
//若为逆序
//交换

只有 $A[j-1] > A[j]$ 时才交换,
因此算法是稳定的



0 1 2 3 4 5 6 7

i 所指位置之前的元素
都已“有序”

i

$j-1$

j

算法性能分析

空间复杂度: $O(1)$

最好情况 (有序):



比较次数= $n-1$; 交换次数=0

最好时间复杂度= $O(n)$

最坏情况 (逆序):



比较次数= $(n-1)+(n-2)+\dots+1 = \frac{n(n-1)}{2}$ = 交换次数

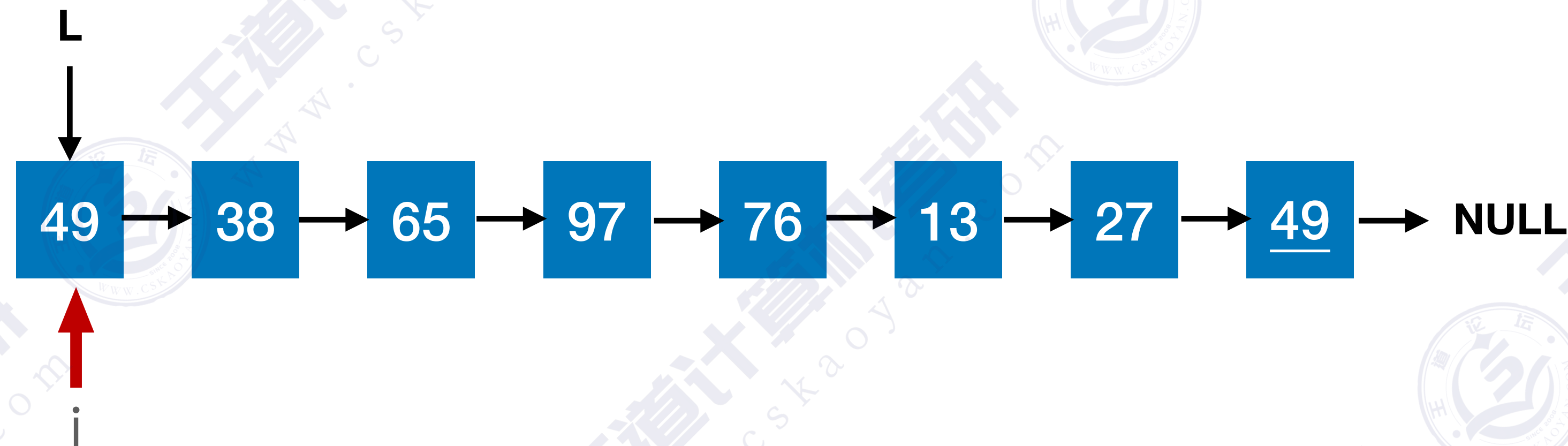
最坏时间复杂度= $O(n^2)$

平均时间复杂度= $O(n^2)$

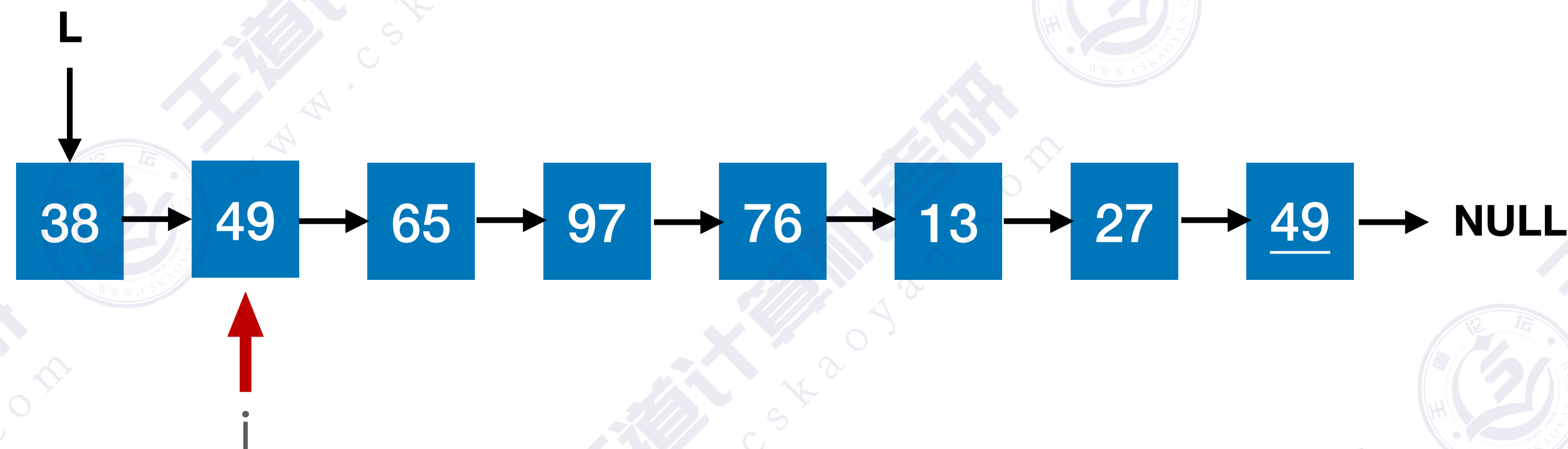
每次交换都需要
移动元素3次

```
//交换  
void swap(int &a, int &b){  
    int temp = a;  
    a = b;  
    b = temp;  
}
```

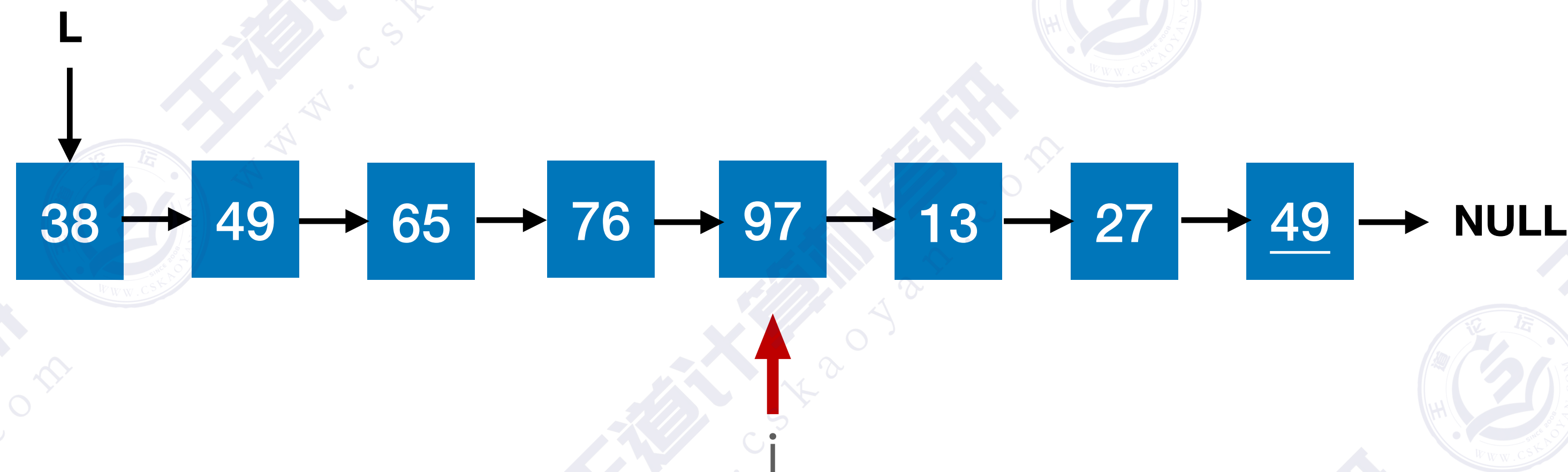
冒泡排序是否适用于链表?



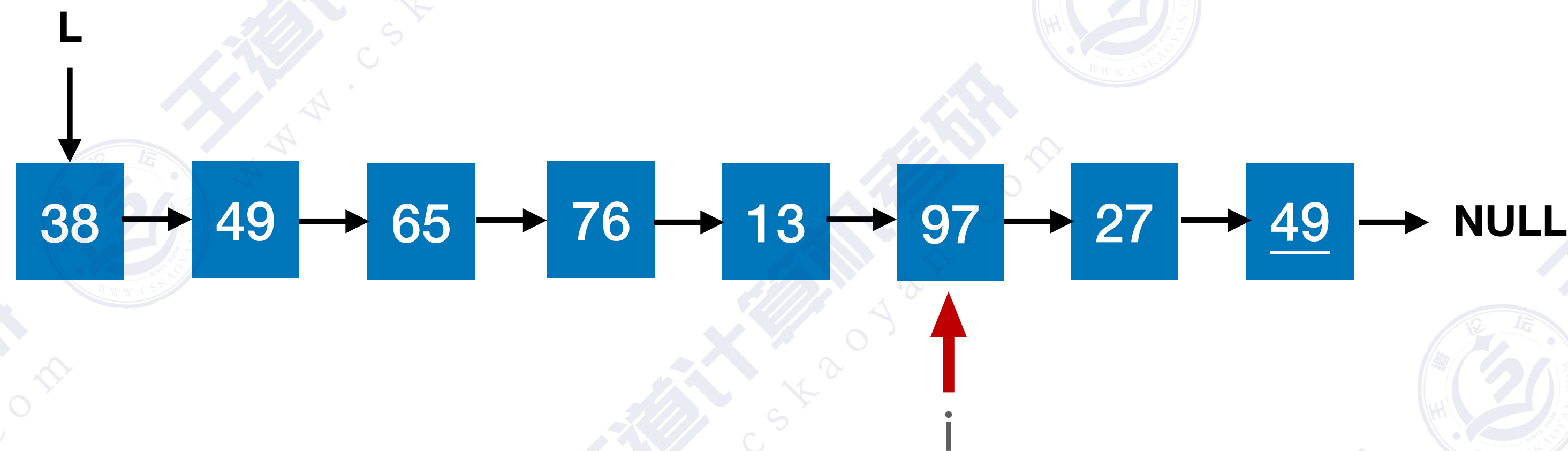
冒泡排序是否适用于链表?



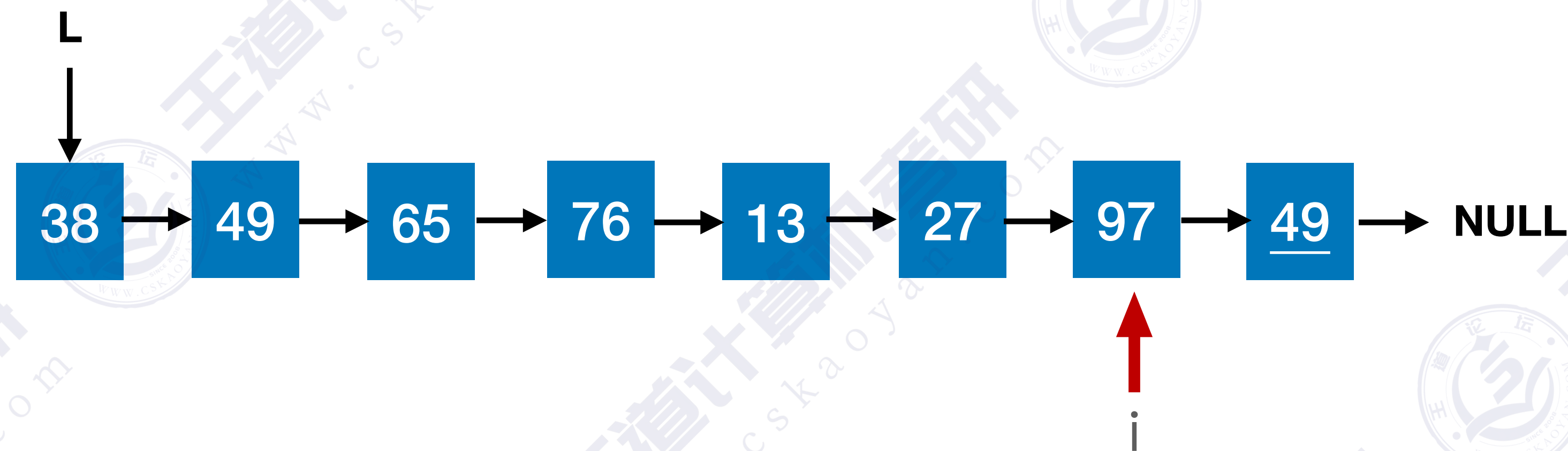
冒泡排序是否适用于链表?



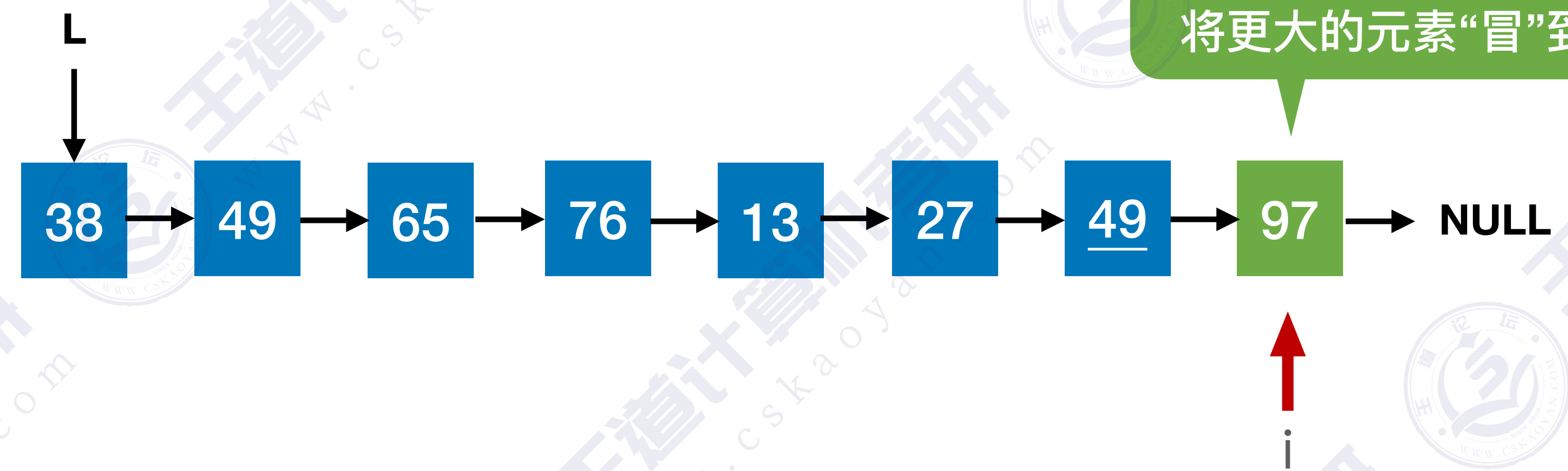
冒泡排序是否适用于链表?



冒泡排序是否适用于链表?



冒泡排序是否适用于链表?



知识回顾与重要考点

冒泡排序

算法原理

从后往前（或从前往后）两两比较相邻元素的值，若为逆序，则交换它们，直到序列比较完。称这样过程为“一趟”冒泡排序。最多只需 $n-1$ 趟排序

每一趟排序都可以使一个元素的移动到最终位置，已经确定最终位置的元素在之后的处理中无需再对比

如果某一趟排序过程中未发生“交换”，则算法可提前结束

空间复杂度

$O(1)$

性能

时间复杂度

最好 $O(n)$

有序

最差 $O(n^2)$

逆序

平均 $O(n^2)$

稳定性

稳定

适用性

顺序表、链表都可以

欢迎大家对本节视频进行评价~



学员评分：8.3.1 冒泡排序

扫一扫二维码打开或分享给好友



— 腾讯文档 —

可多人实时在线编辑，权限安全可控



公众号：王道在线



b站：王道计算机教育



抖音：王道计算机考研

本节内容

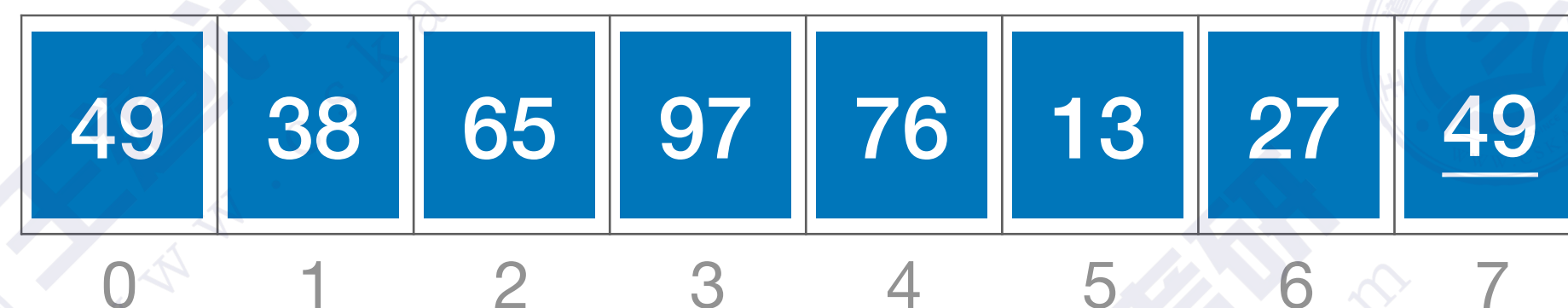
快速排序

知识总览



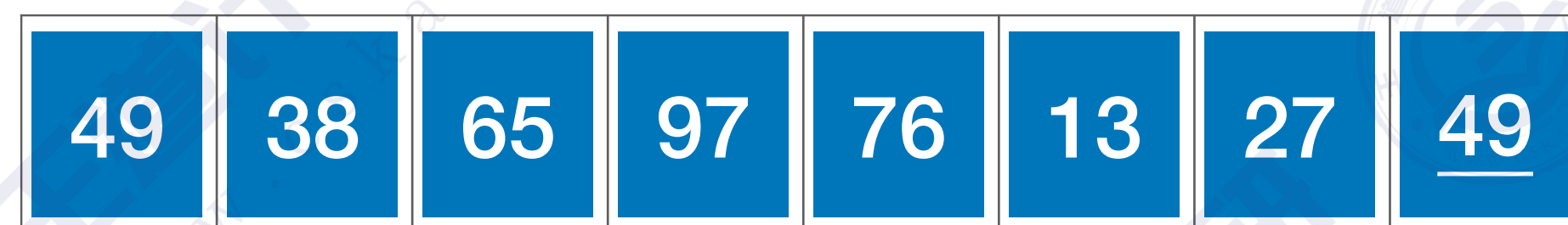
基于“交换”的排序：根据序列中两个元素关键字的比较结果来对换这两个记录在序列中的位置

快速排序



算法思想：在待排序表 $L[1...n]$ 中任取一个元素pivot作为枢轴（或基准，通常取首元素），通过一趟排序将待排序表划分为独立的两部分 $L[1...k-1]$ 和 $L[k+1...n]$ ，使得 $L[1...k-1]$ 中的所有元素小于pivot， $L[k+1...n]$ 中的所有元素大于等于pivot，则pivot放在了其最终位置 $L(k)$ 上，这个过程称为一次“划分”。然后分别递归地对两个子表重复上述过程，直至每部分内只有一个元素或空为止，即所有元素放在了其最终位置上。

快速排序

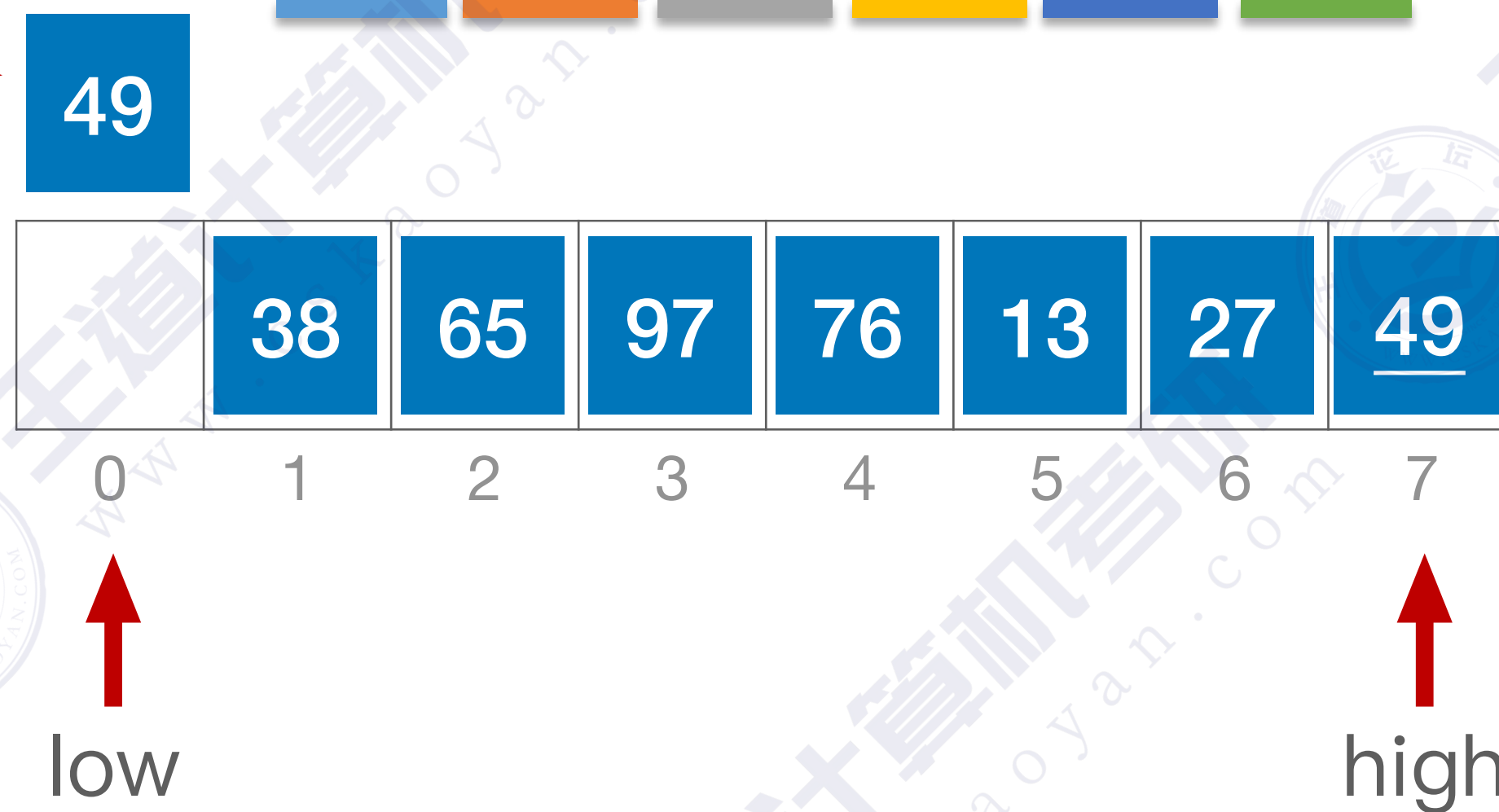


0
↑
low

7
↑
high

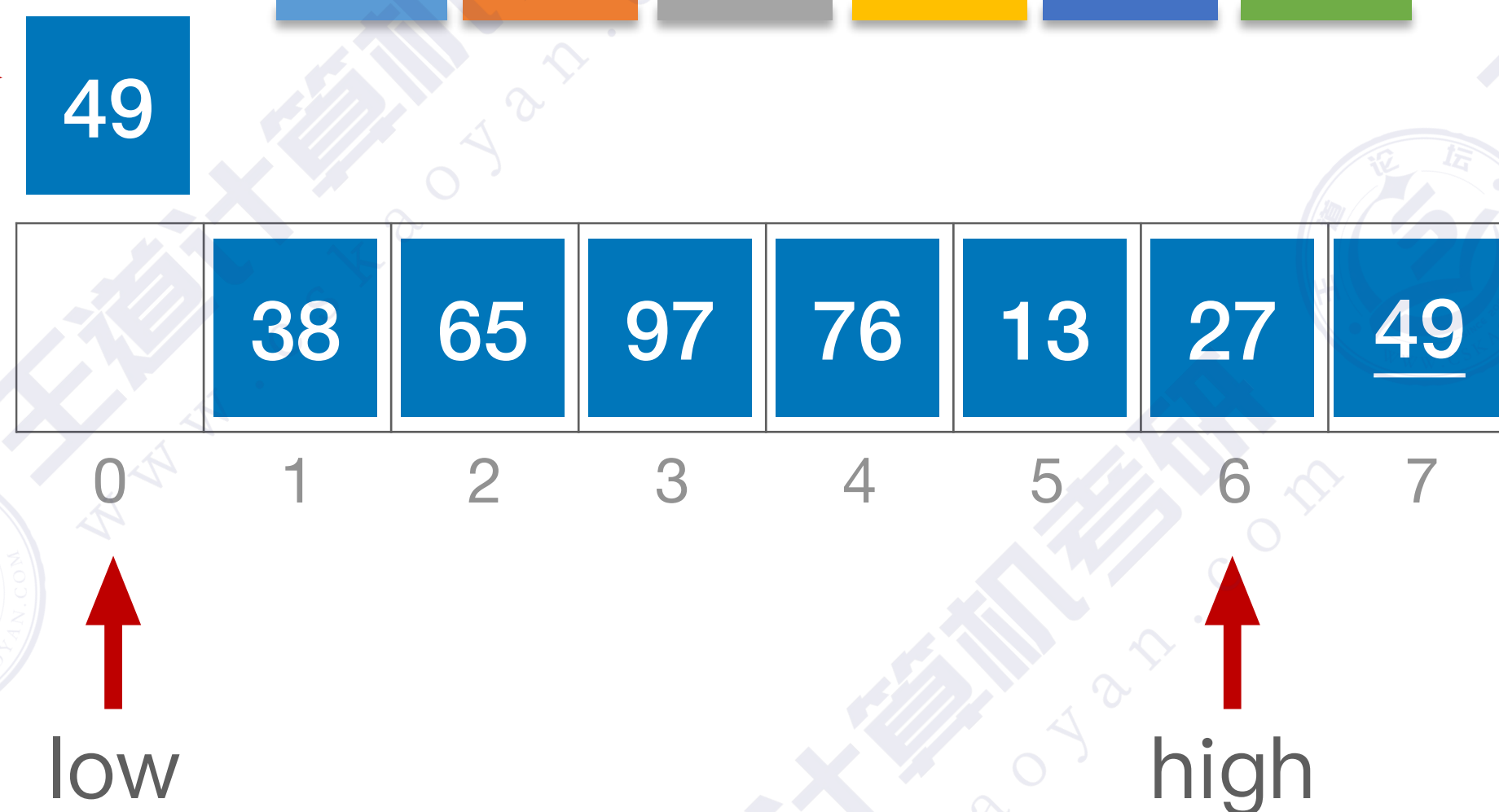
快速排序

更小的元素都交换到左边
更大的元素都交换到右边



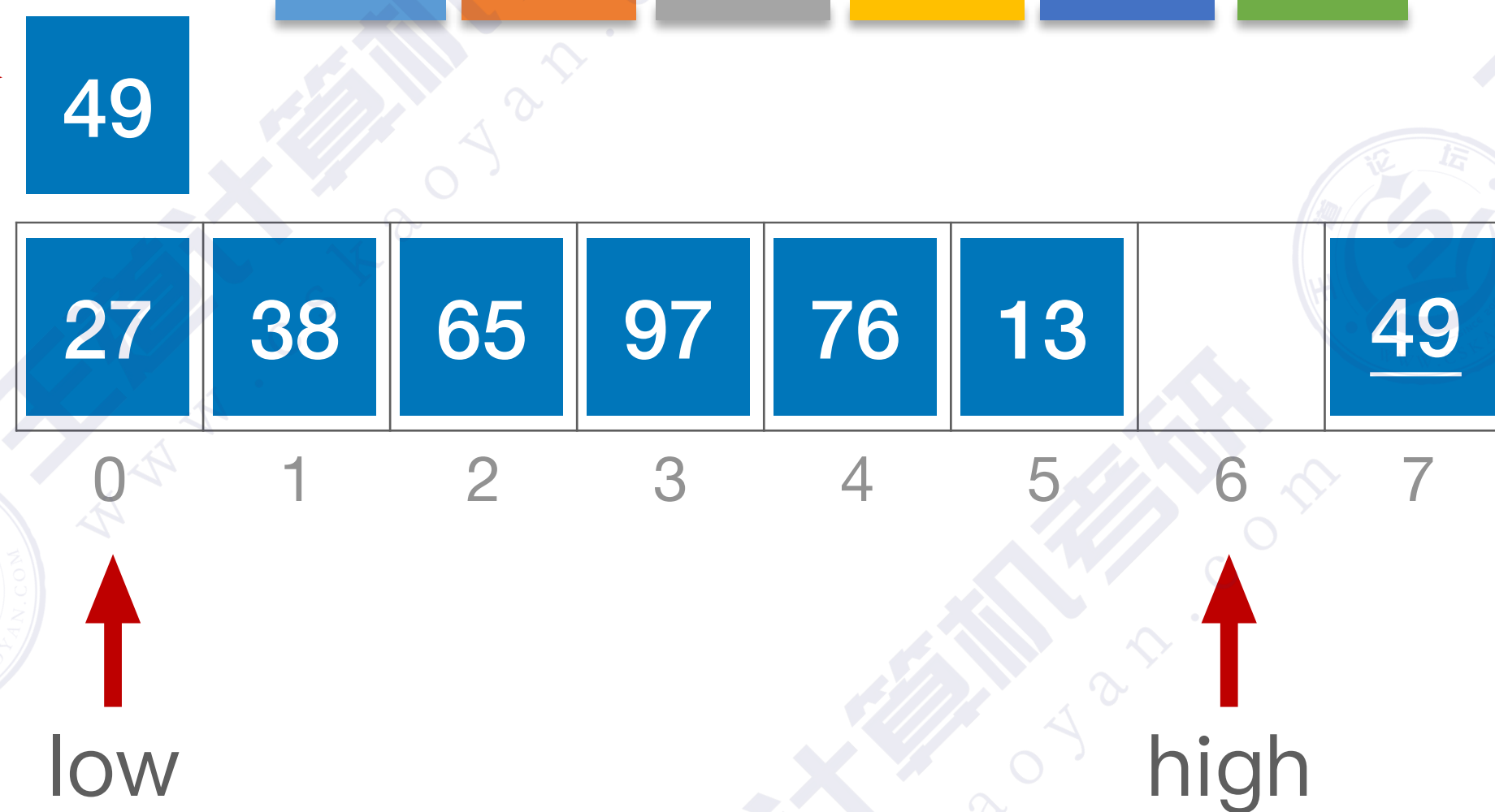
快速排序

更小的元素都交换到左边
更大的元素都交换到右边



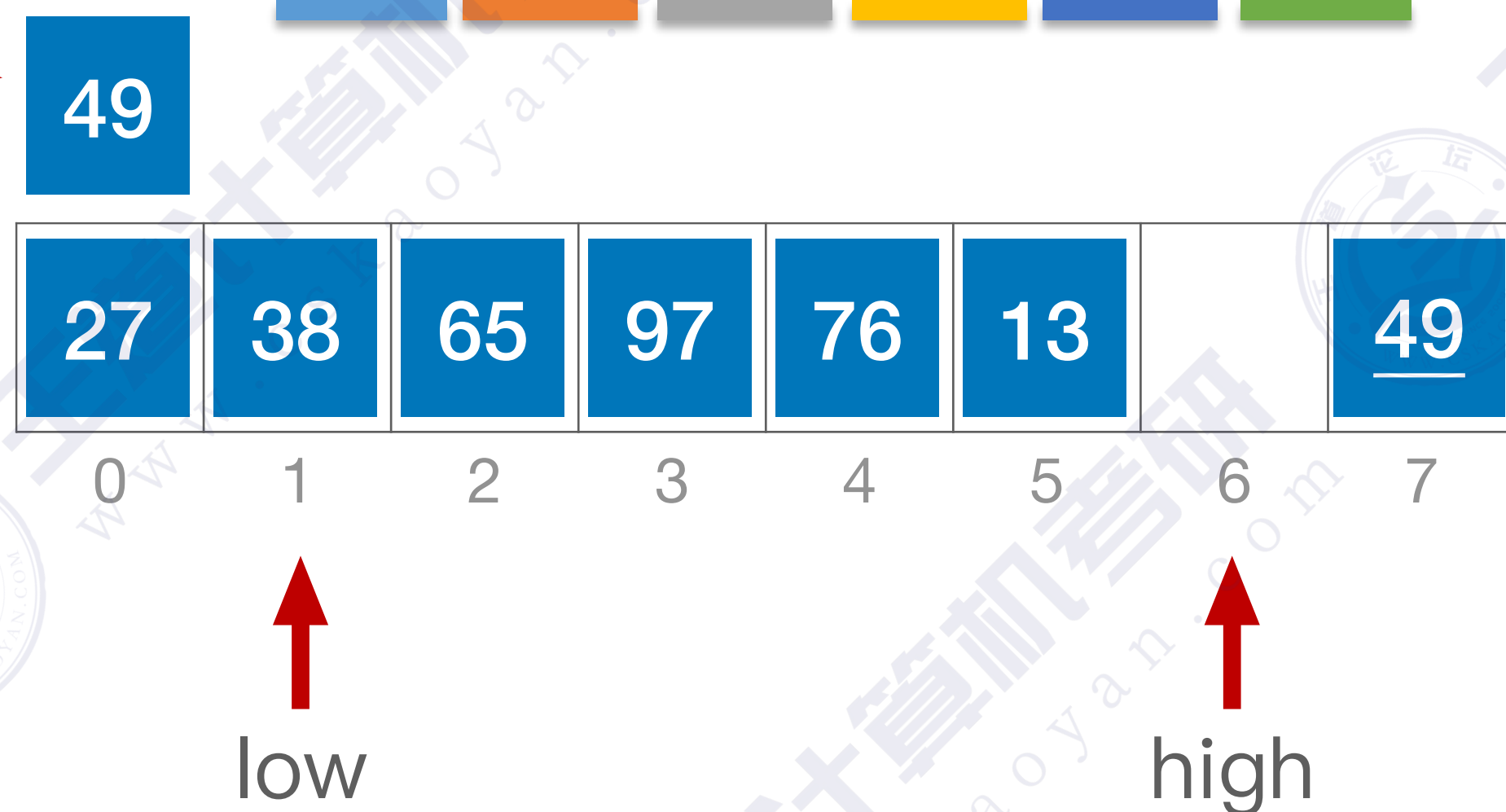
快速排序

更小的元素都交换到左边
更大的元素都交换到右边



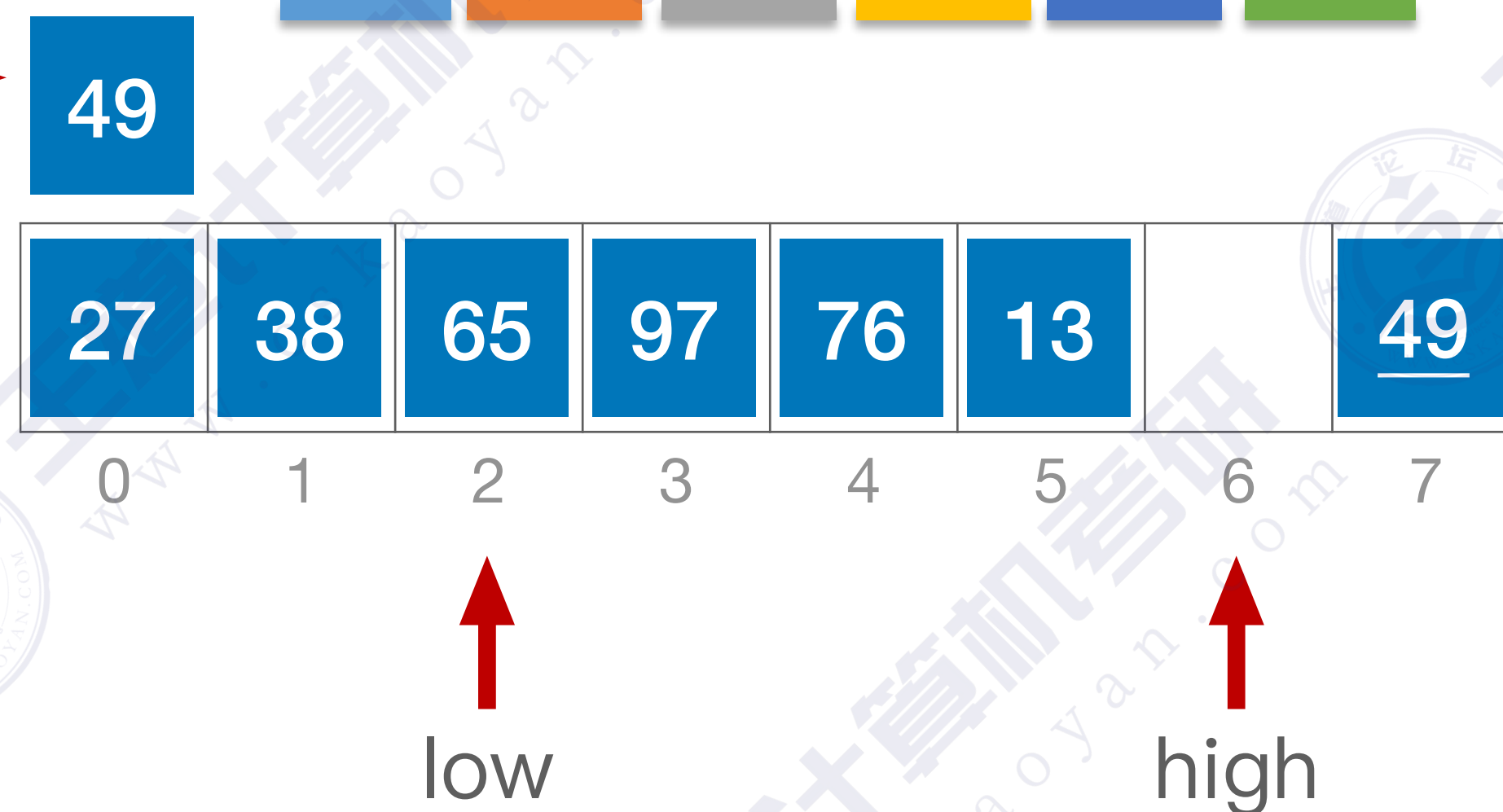
快速排序

更小的元素都交换到左边
更大的元素都交换到右边



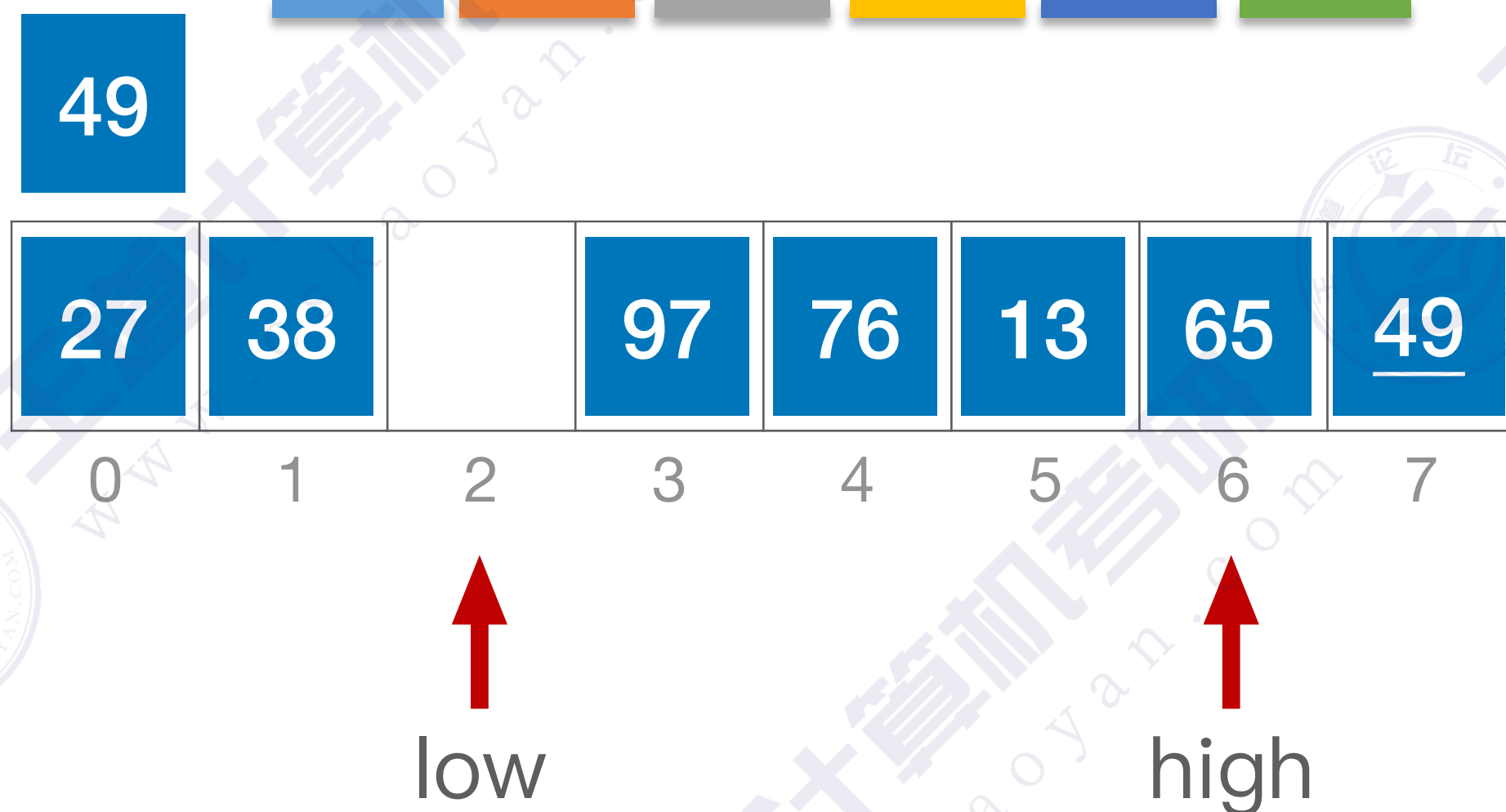
快速排序

更小的元素都交换到左边
更大的元素都交换到右边



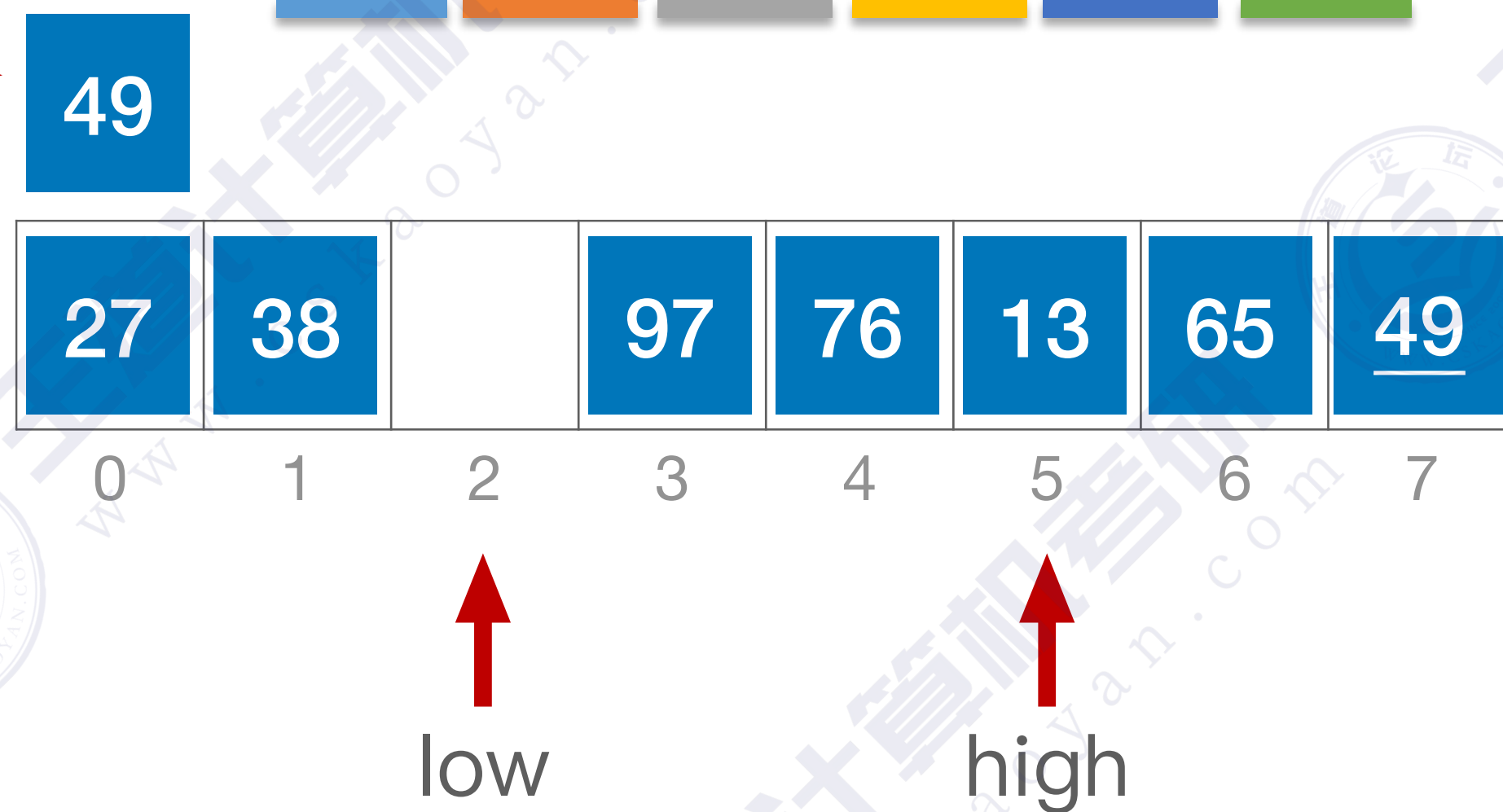
快速排序

更小的元素都交换到左边
更大的元素都交换到右边



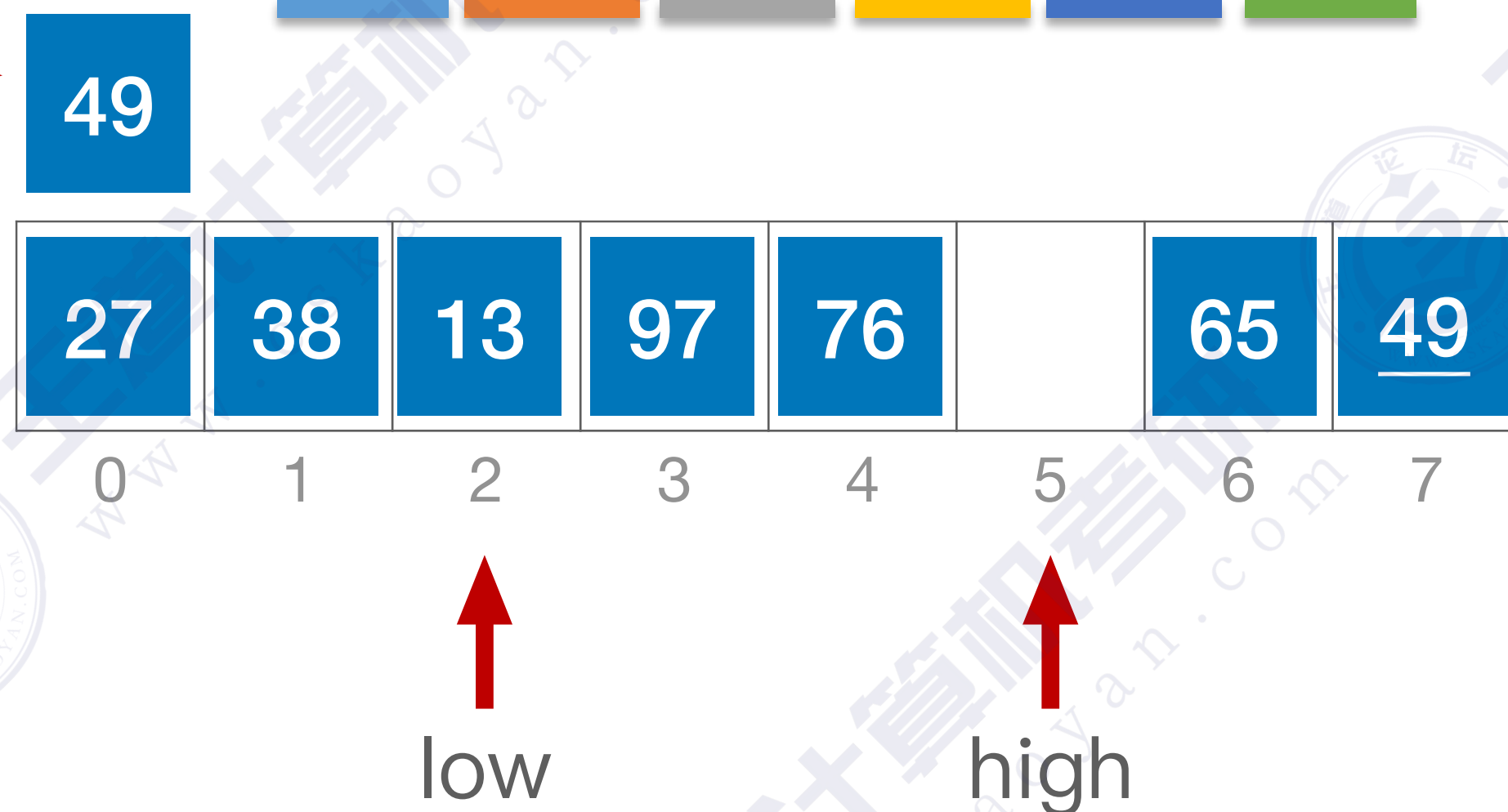
快速排序

更小的元素都交换到左边
更大的元素都交换到右边



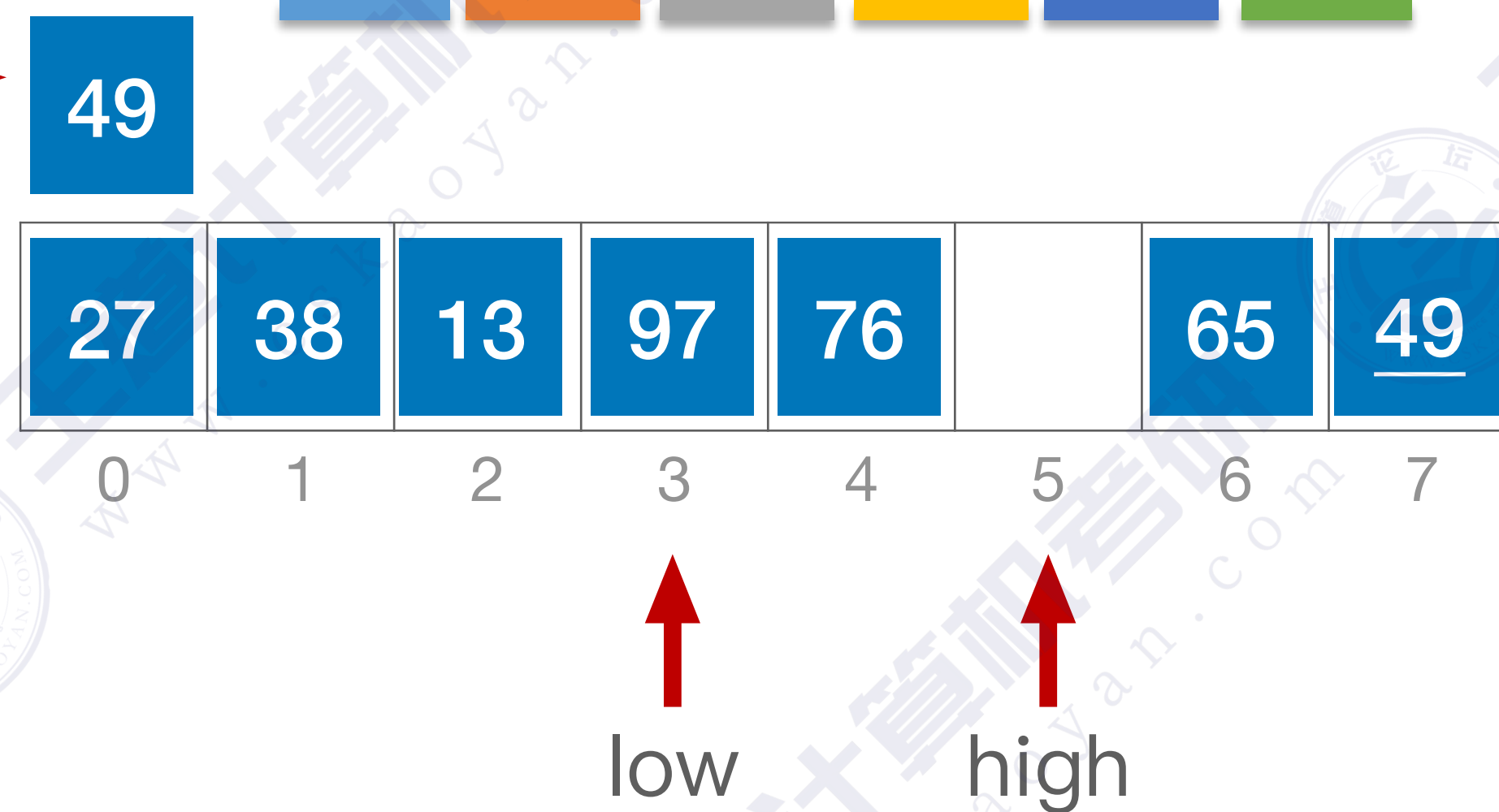
快速排序

更小的元素都交换到左边
更大的元素都交换到右边



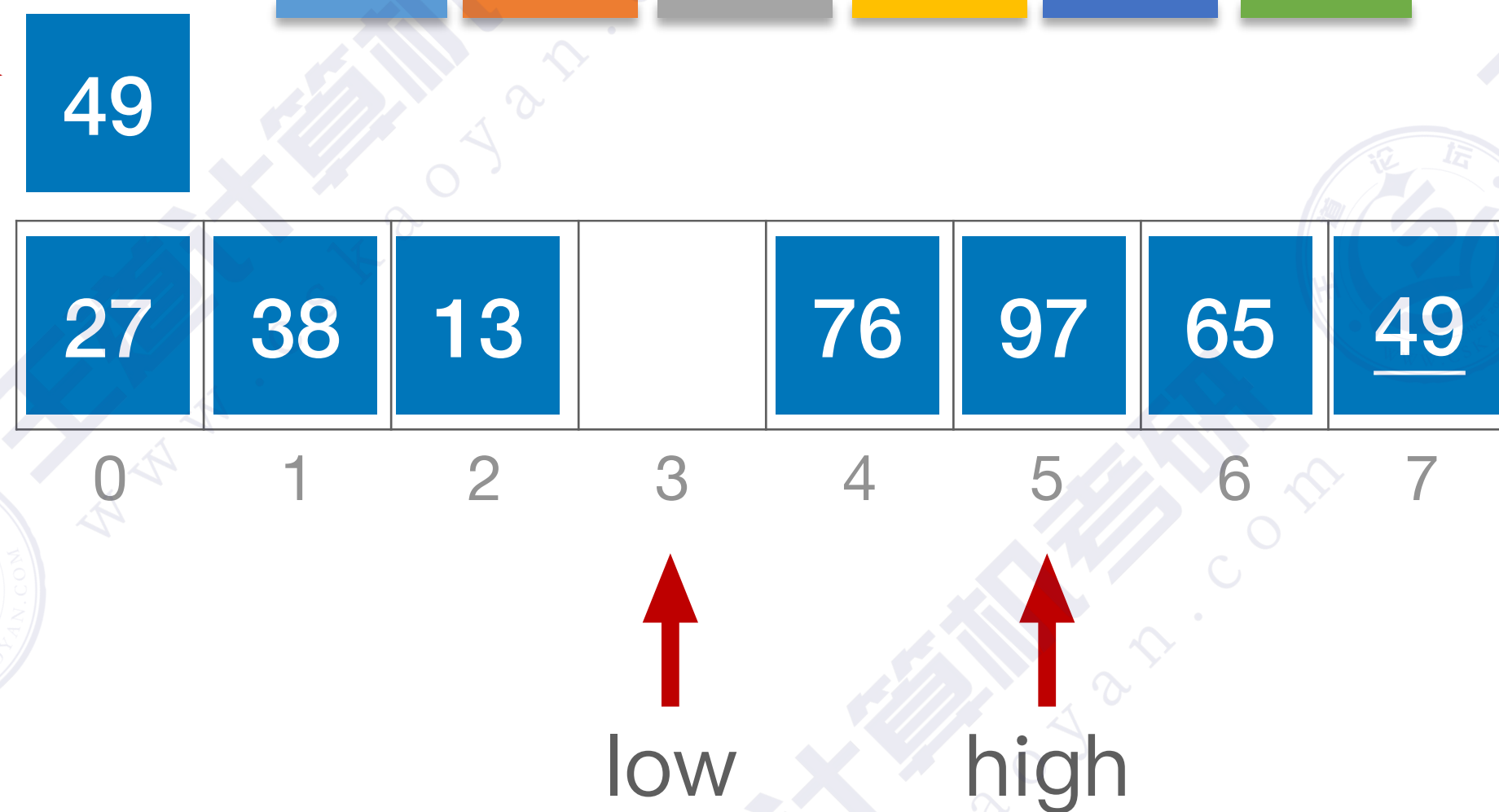
快速排序

更小的元素都交换到左边
更大的元素都交换到右边



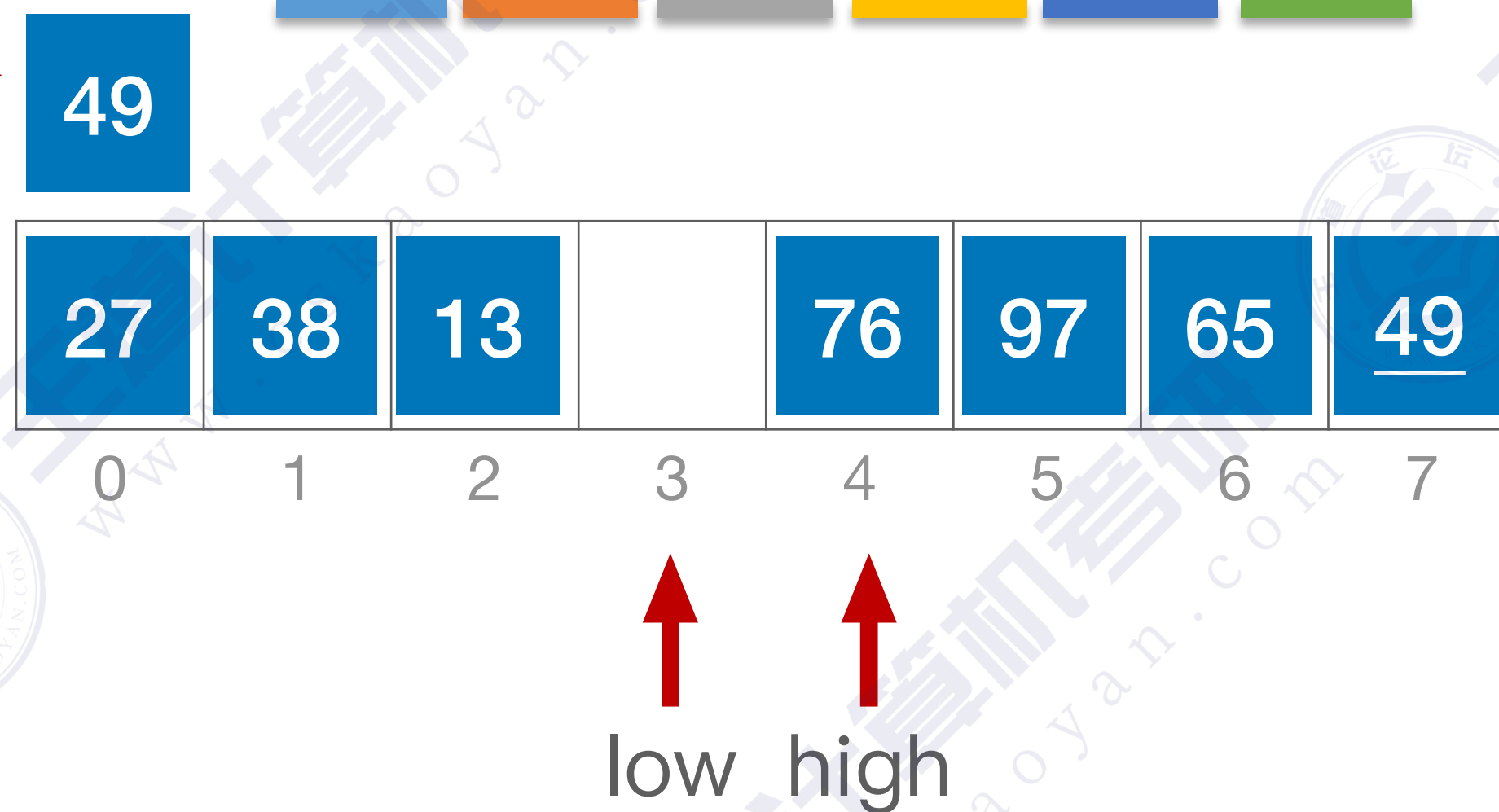
快速排序

更小的元素都交换到左边
更大的元素都交换到右边



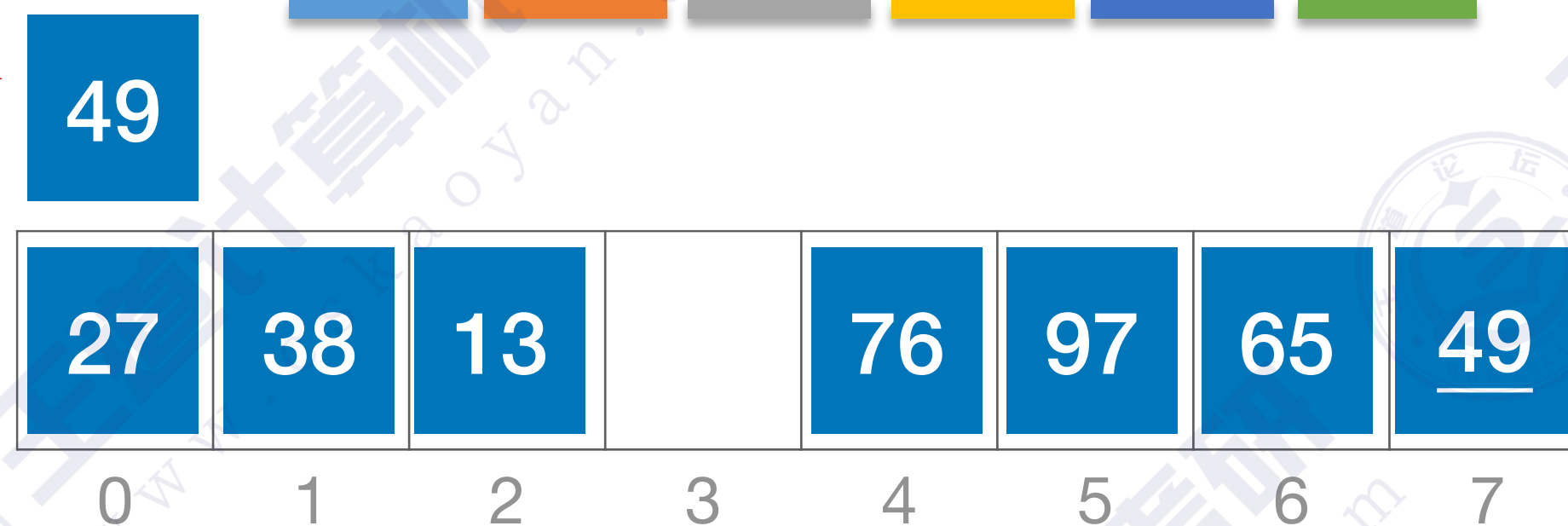
快速排序

更小的元素都交换到左边
更大的元素都交换到右边



快速排序

更小的元素都交换到左边
更大的元素都交换到右边

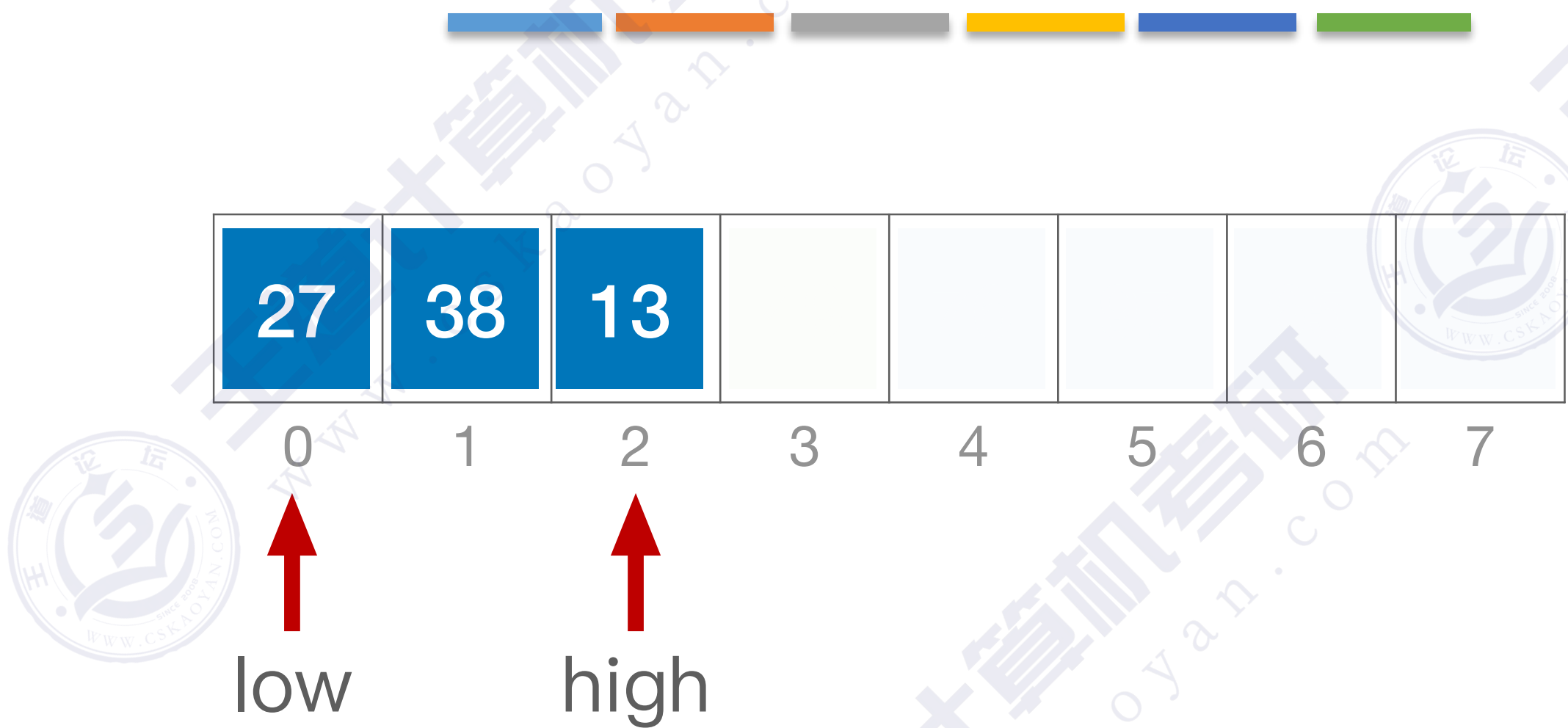


快速排序

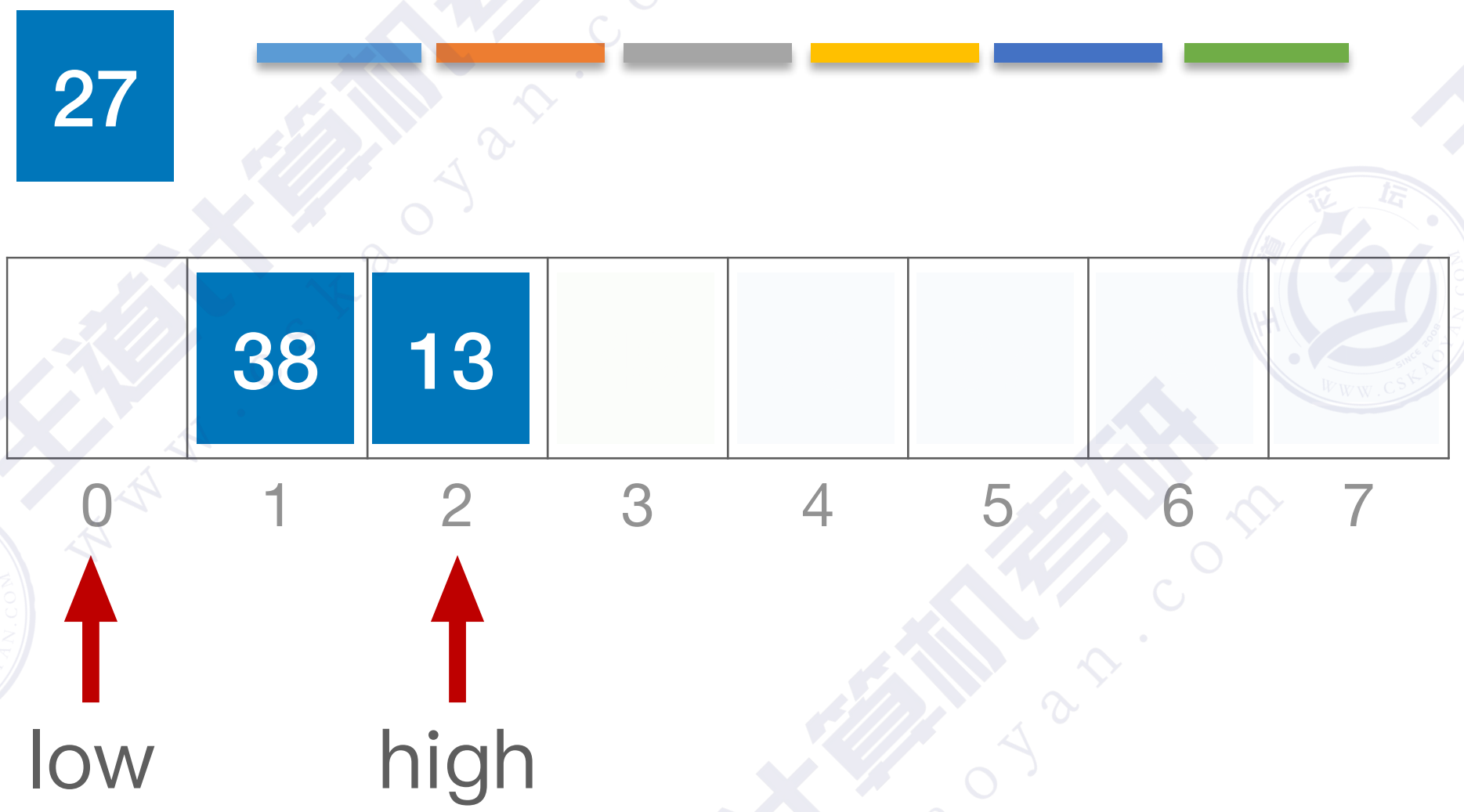


算法思想：在待排序表 $L[1...n]$ 中任取一个元素pivot作为枢轴（或基准，通常取首元素），通过一趟排序将待排序表划分为独立的两部分 $L[1...k-1]$ 和 $L[k+1...n]$ ，使得 $L[1...k-1]$ 中的所有元素小于pivot， $L[k+1...n]$ 中的所有元素大于等于pivot，则pivot放在了其最终位置 $L(k)$ 上，这个过程称为一次“划分”。然后分别递归地对两个子表重复上述过程，直至每部分内只有一个元素或空为止，即所有元素放在了其最终位置上。

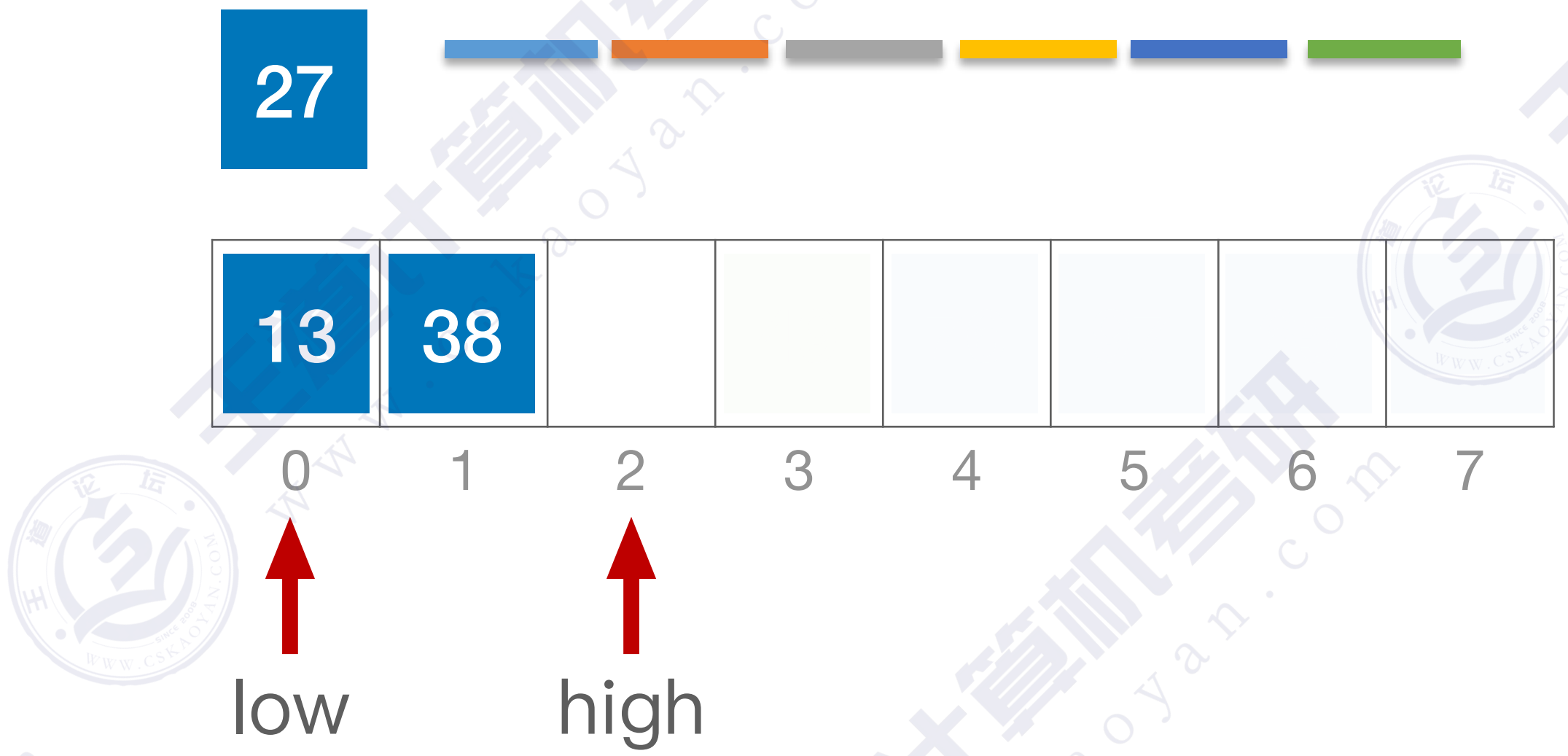
快速排序



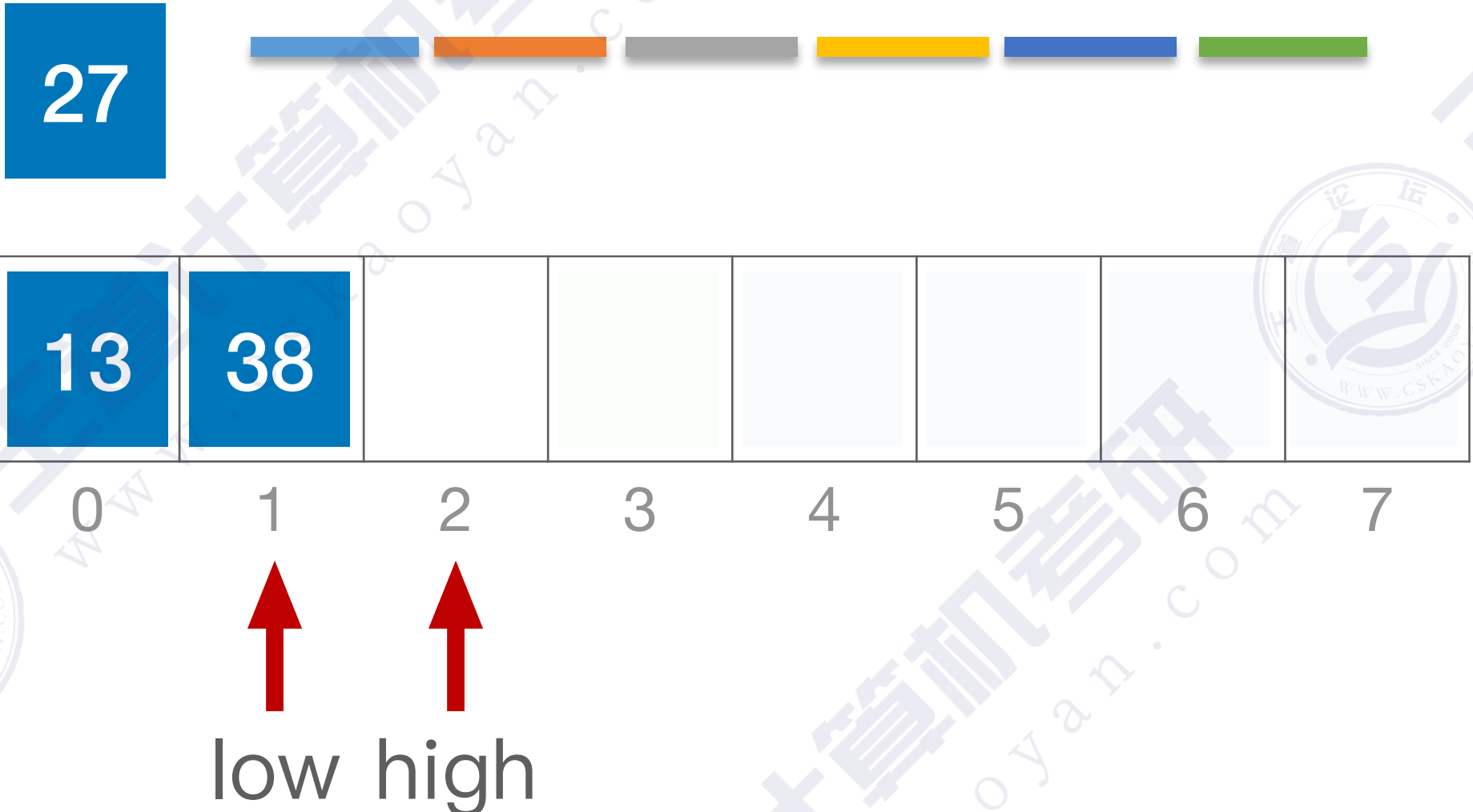
快速排序



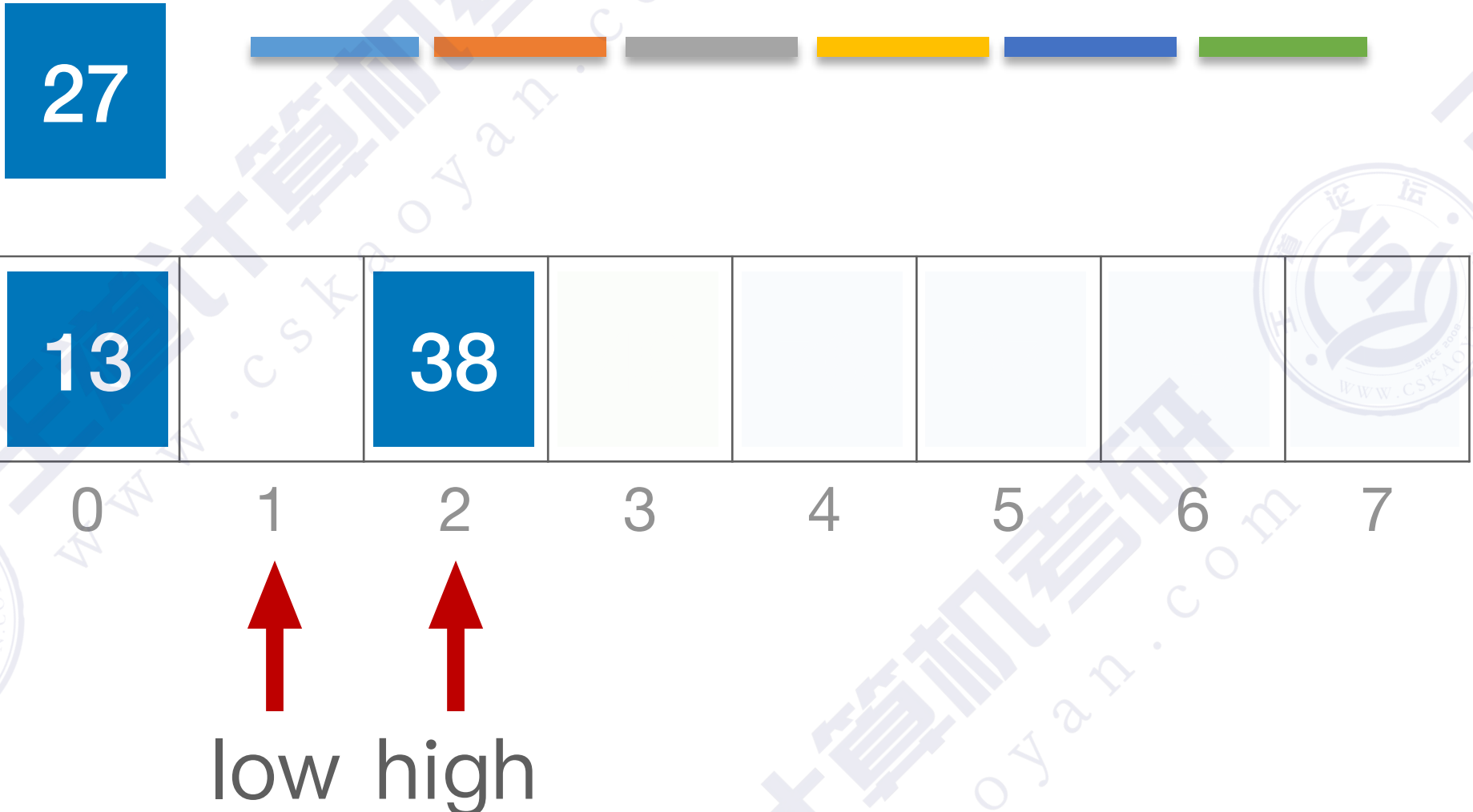
快速排序



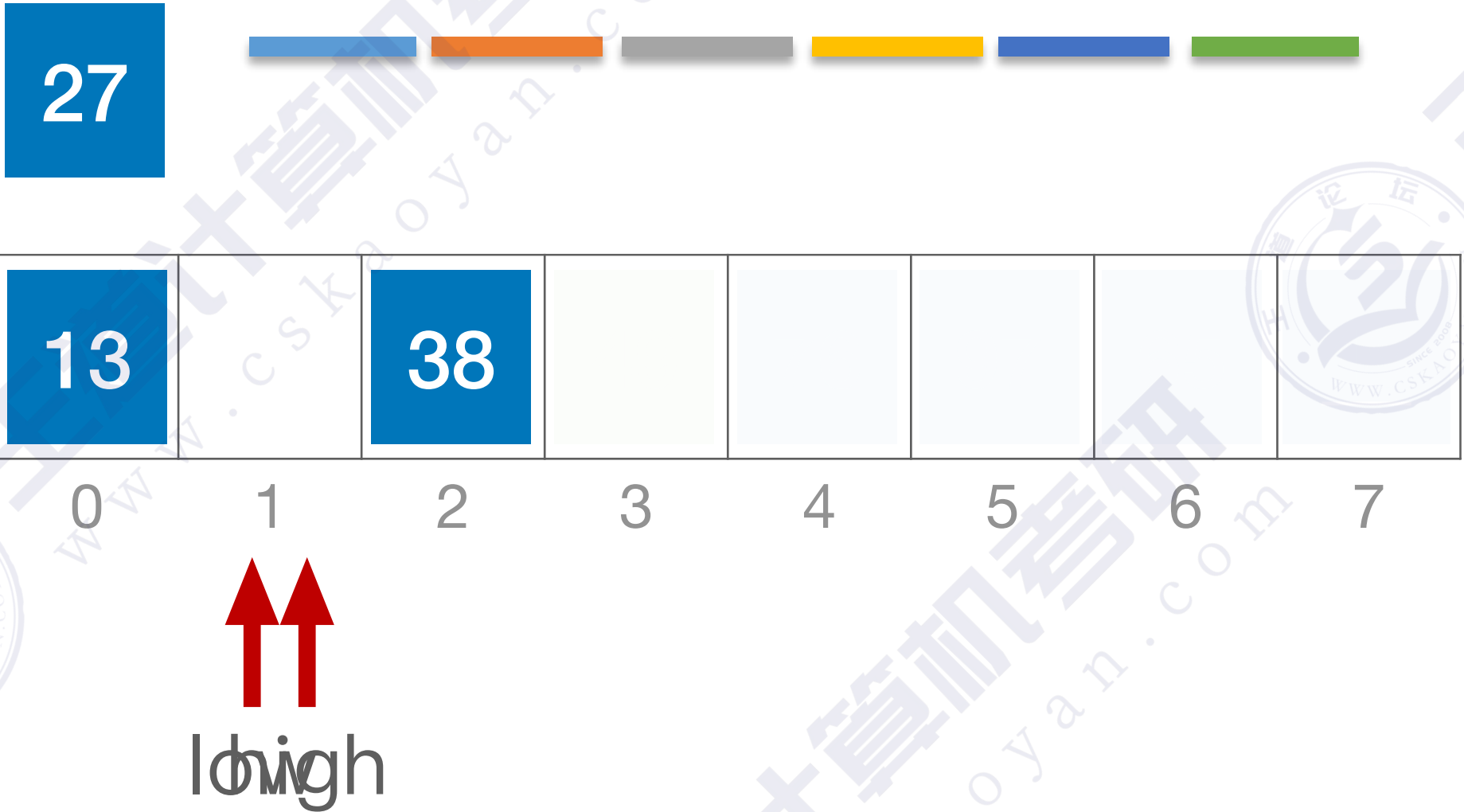
快速排序



快速排序



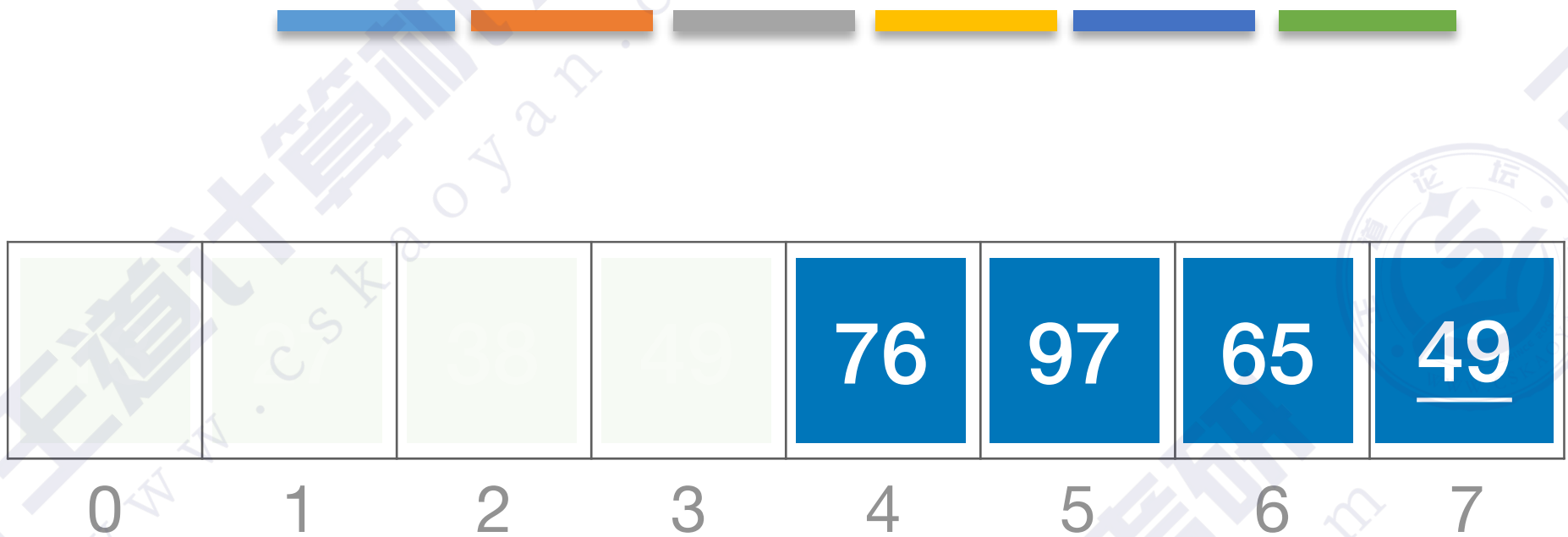
快速排序



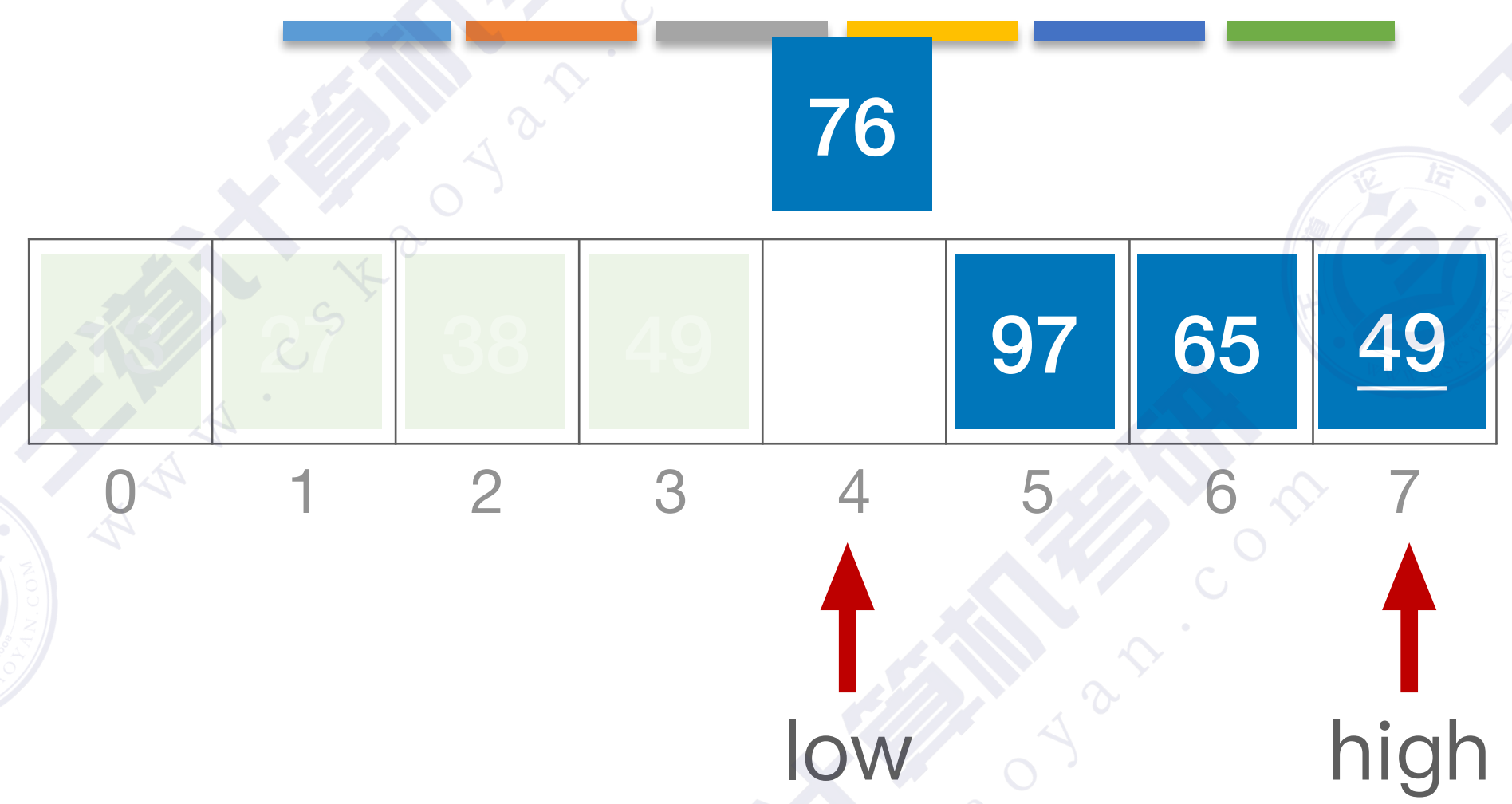
快速排序



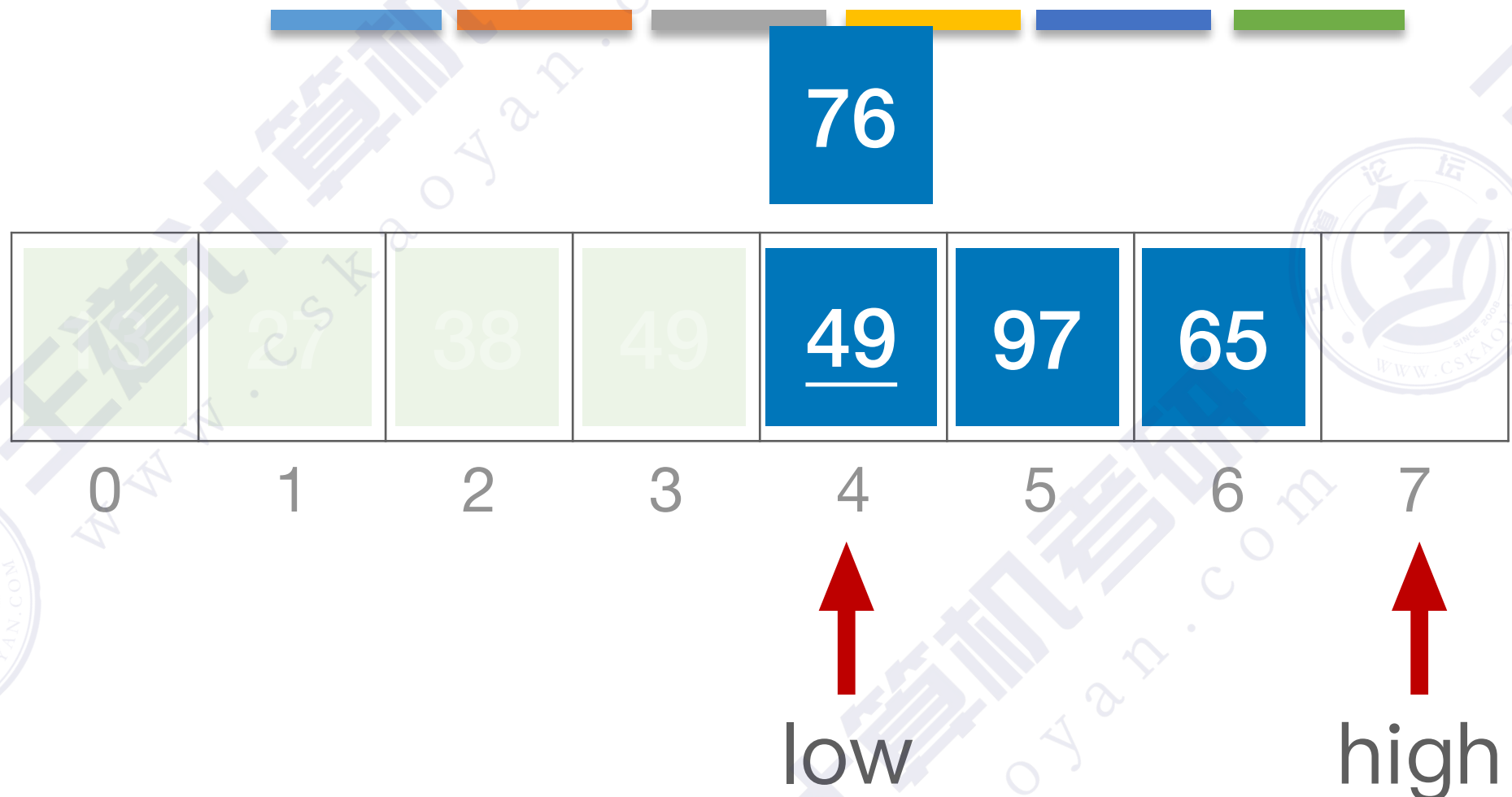
快速排序



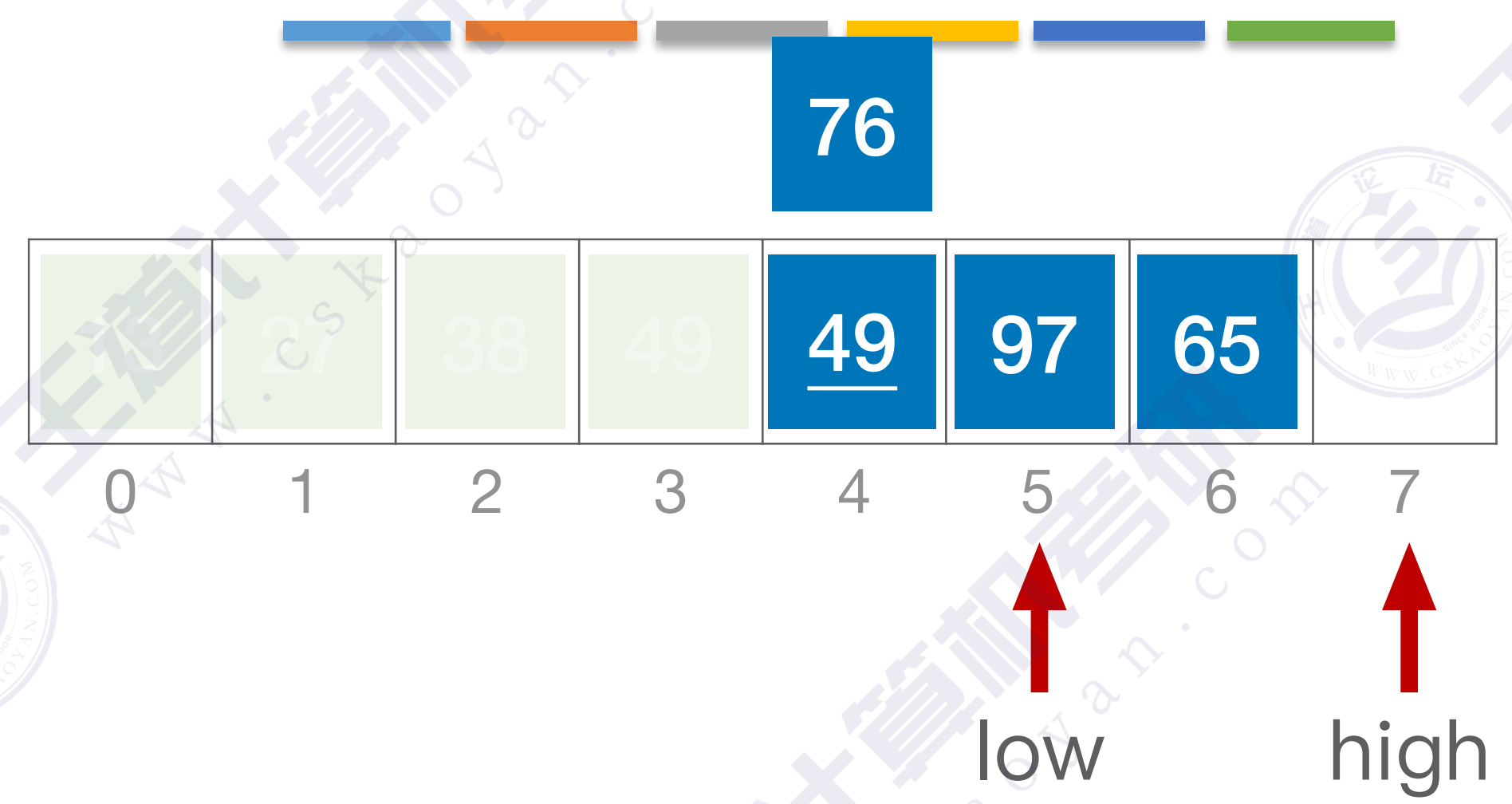
快速排序



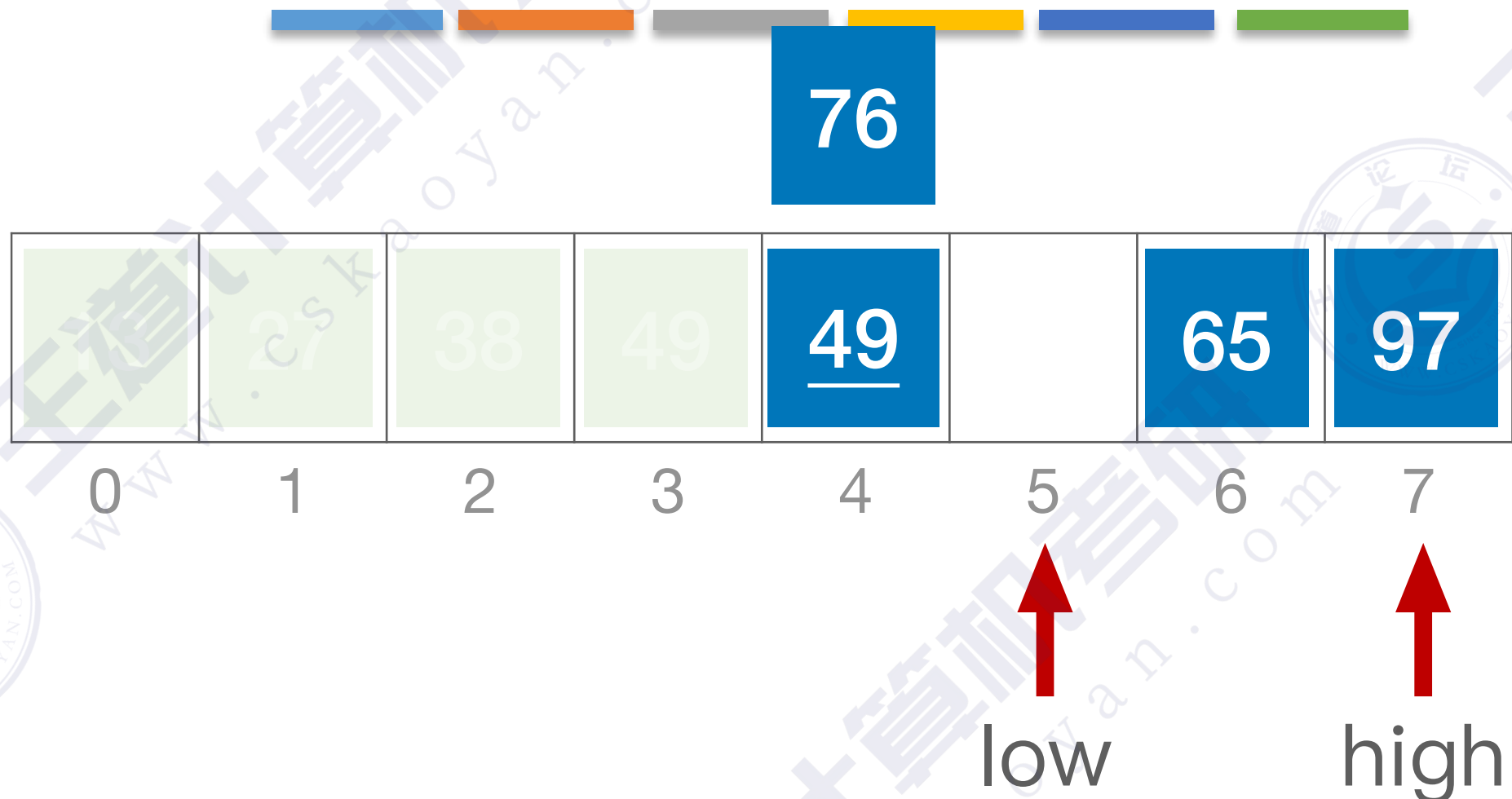
快速排序



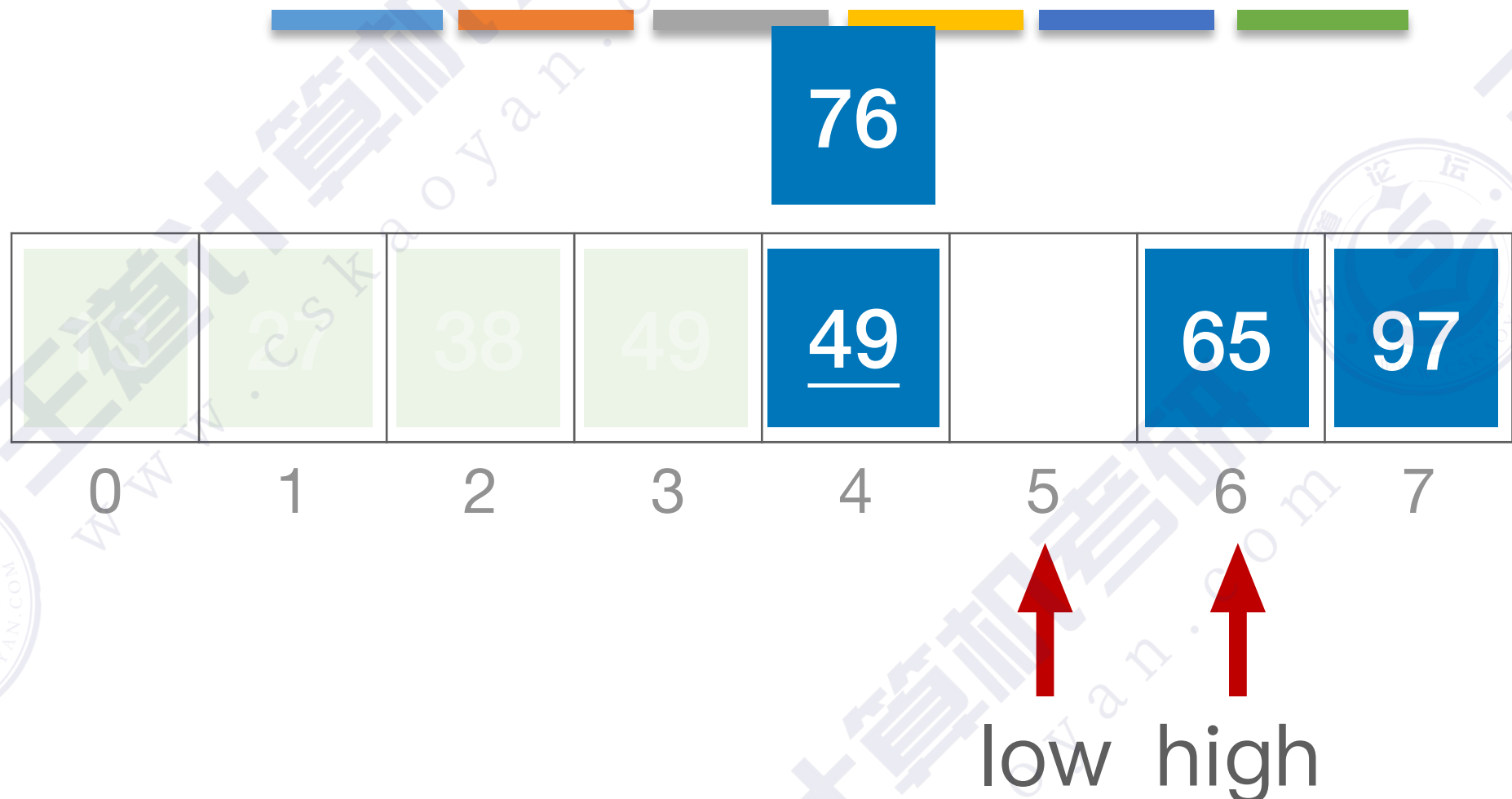
快速排序



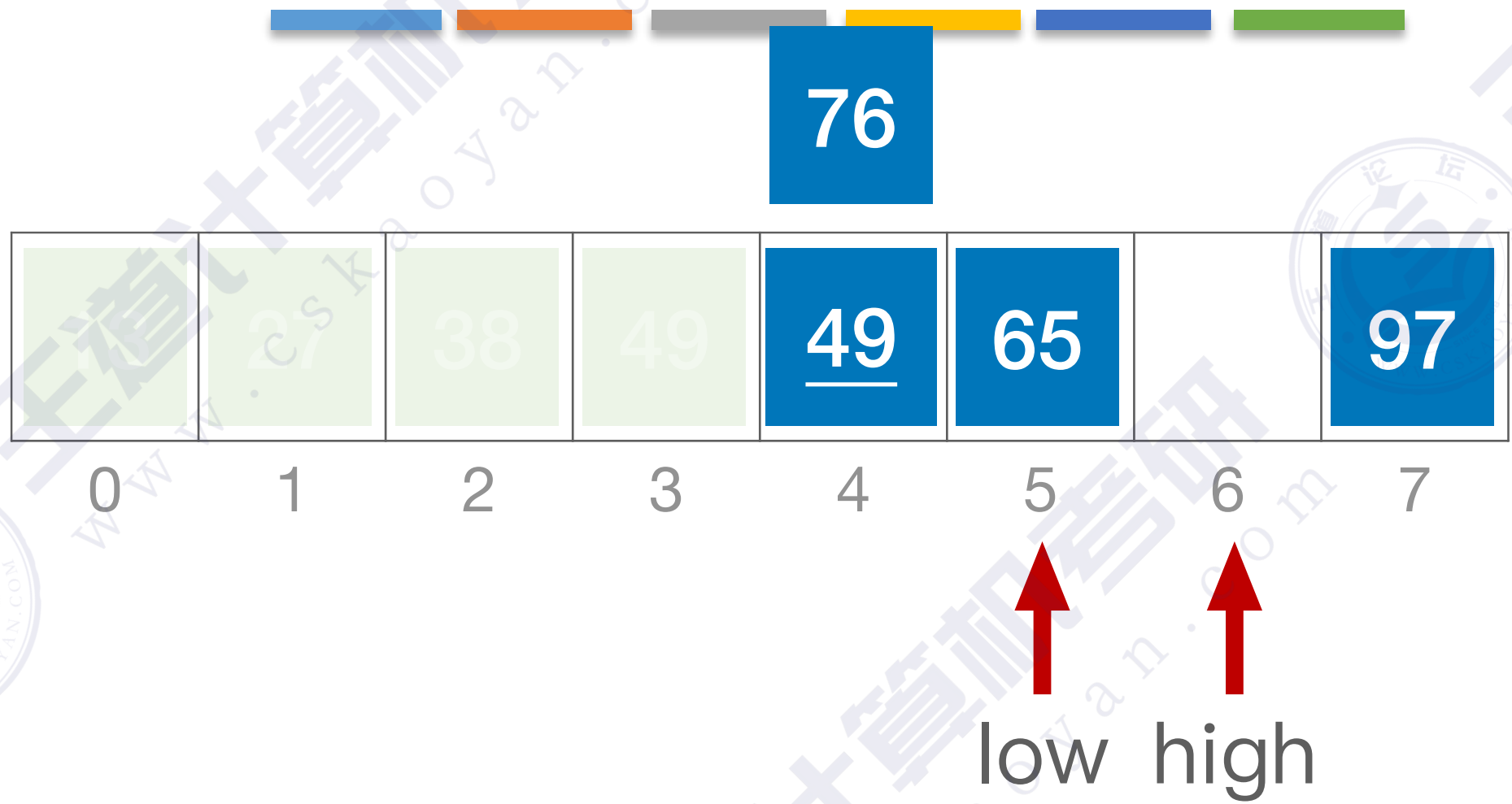
快速排序



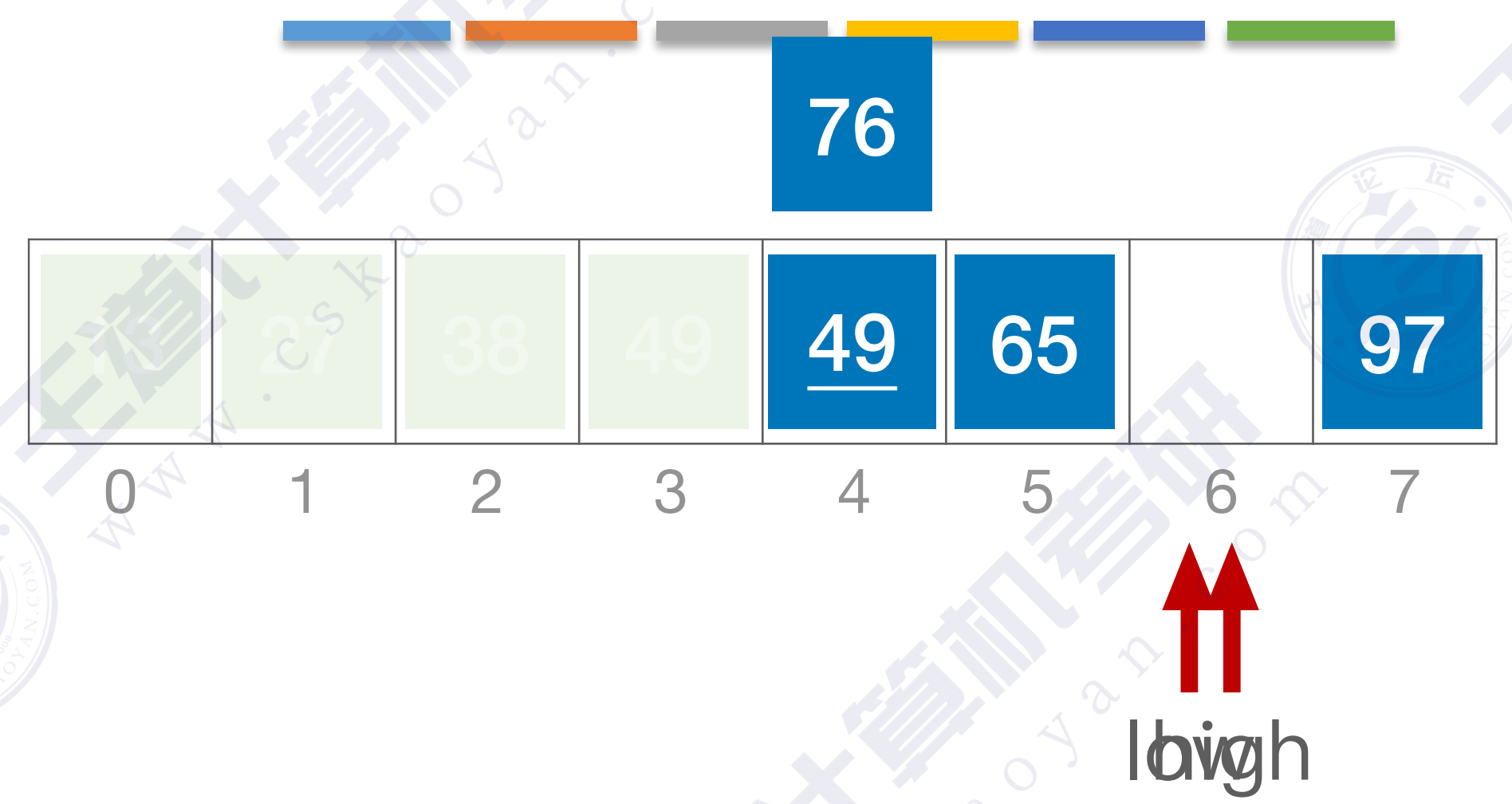
快速排序



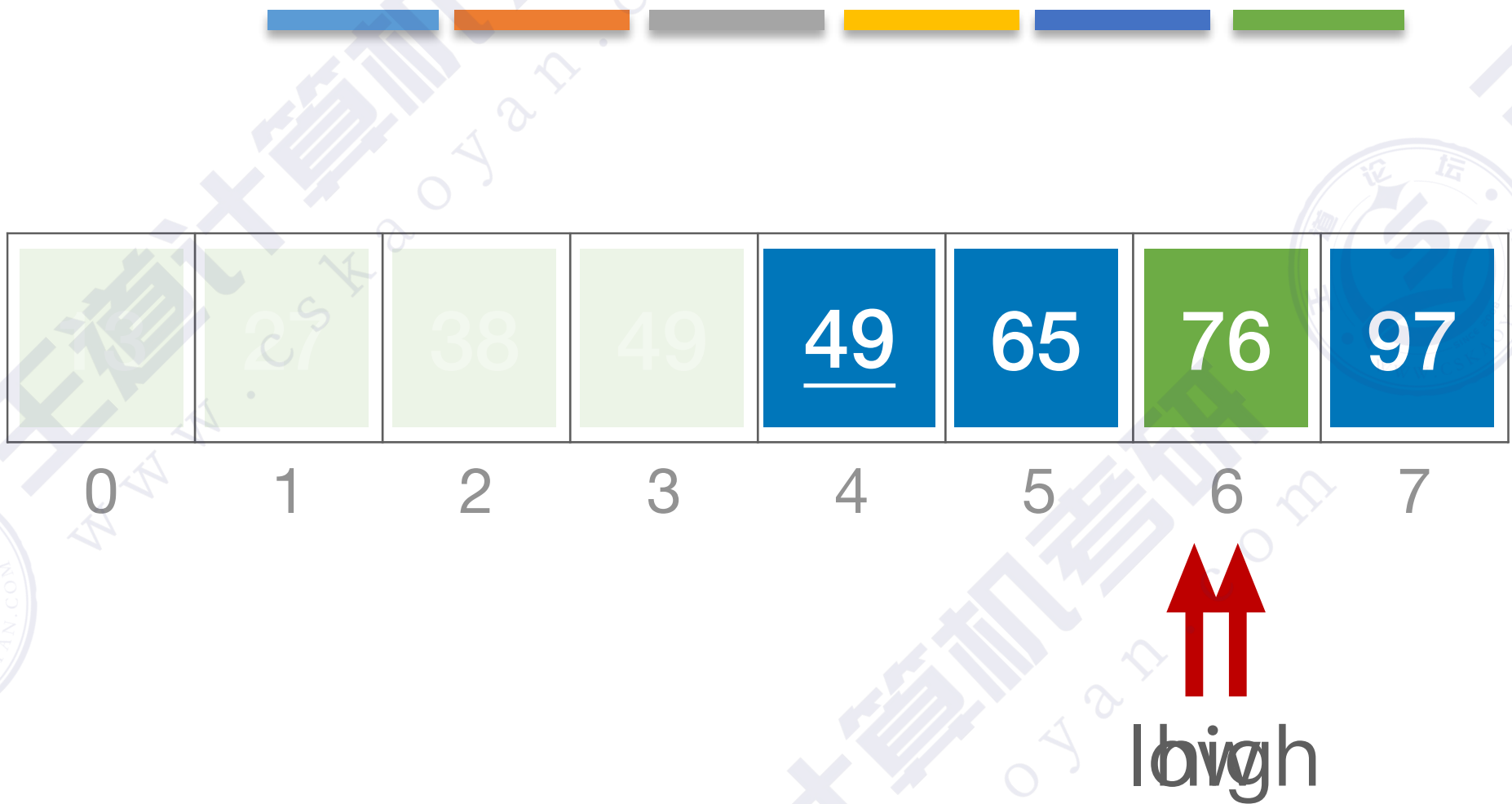
快速排序



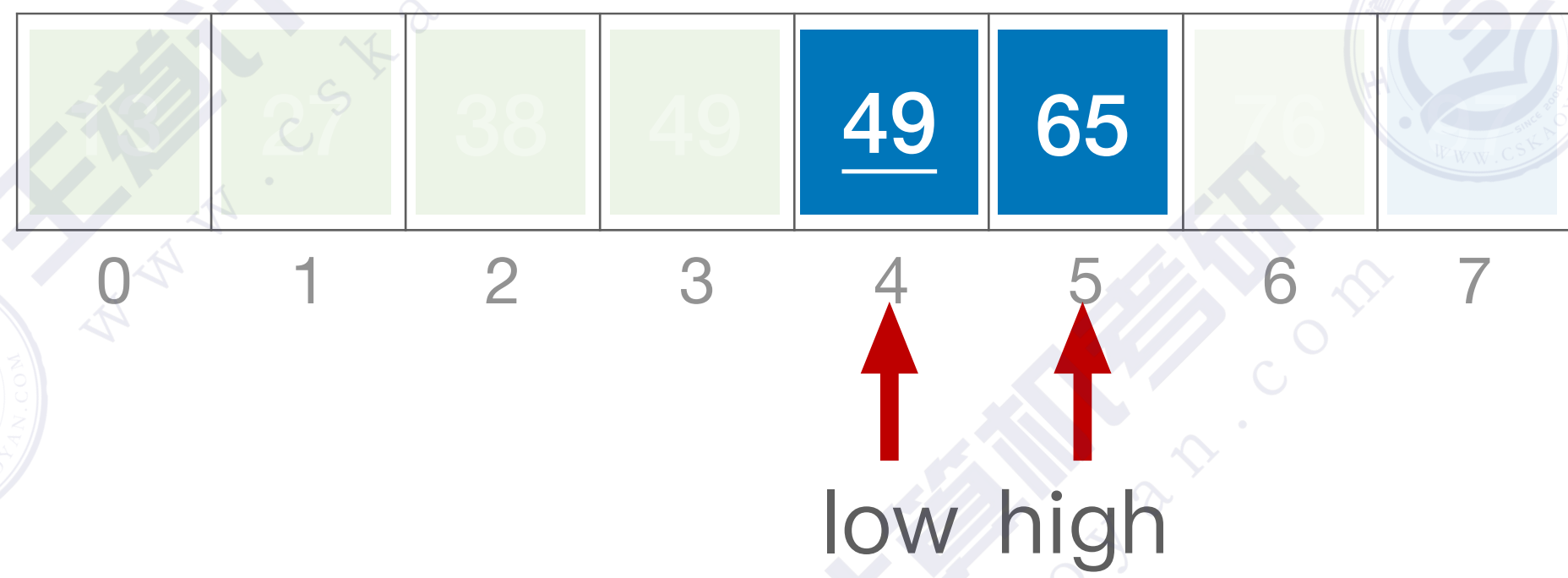
快速排序



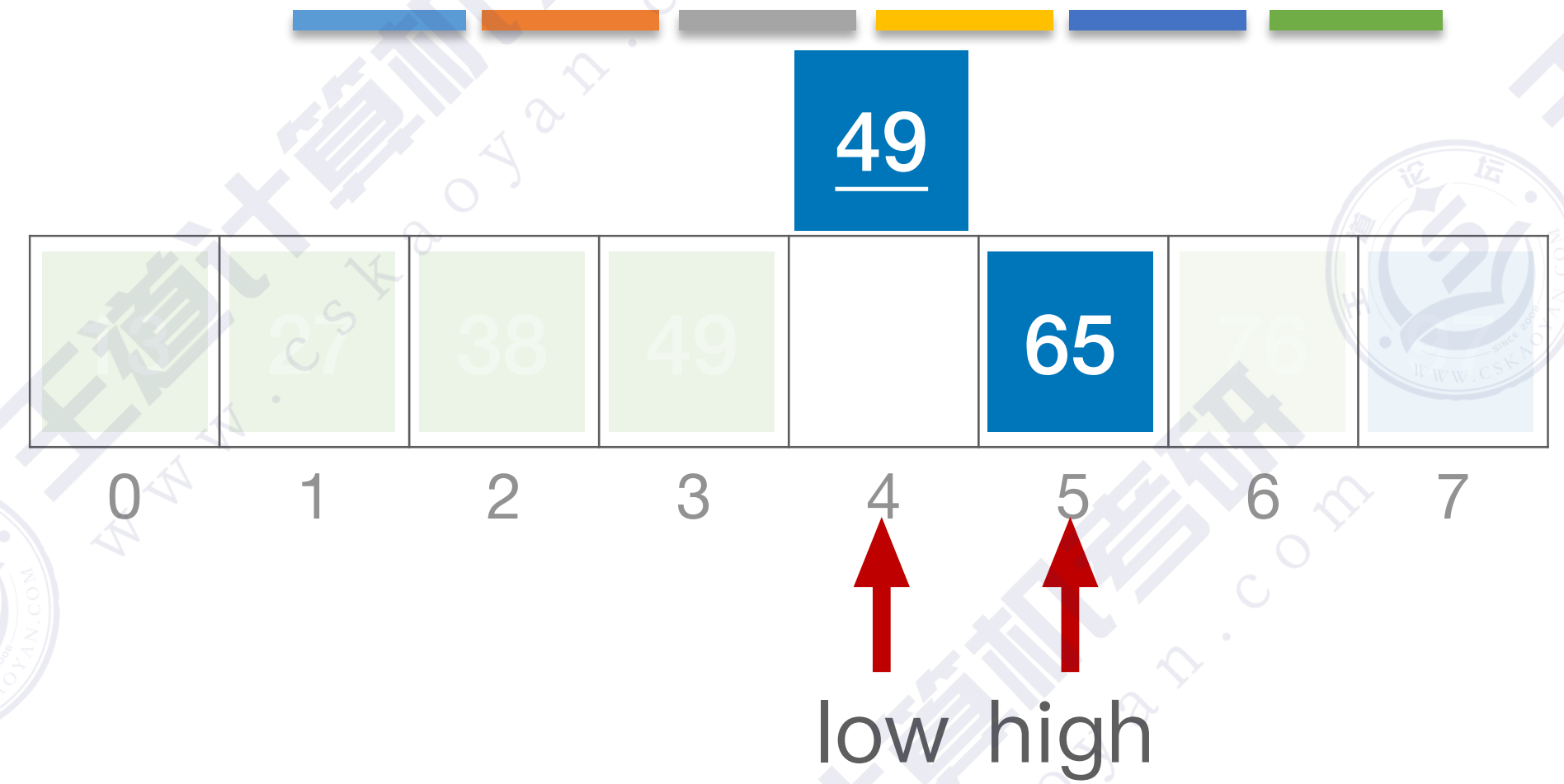
快速排序



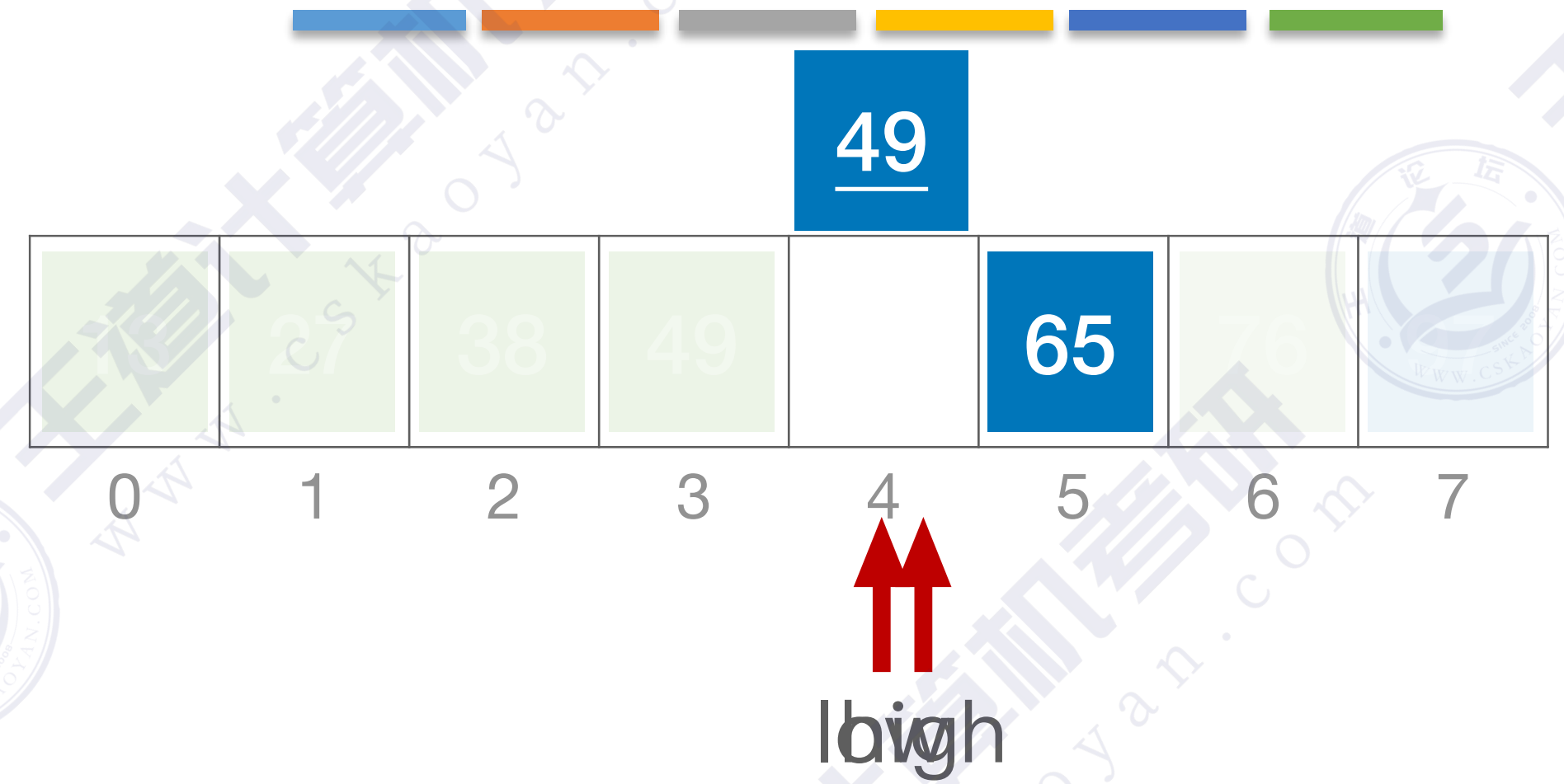
快速排序



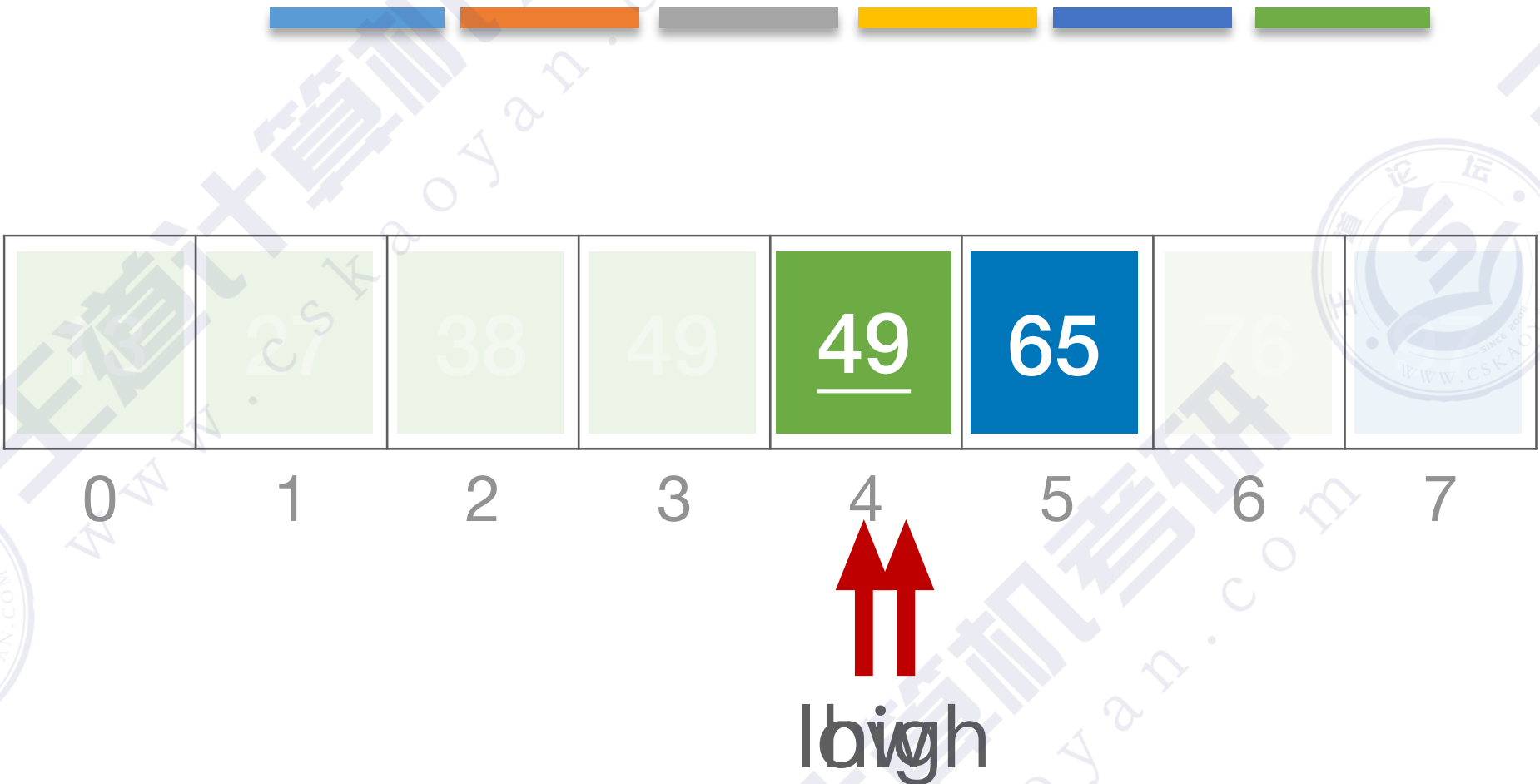
快速排序



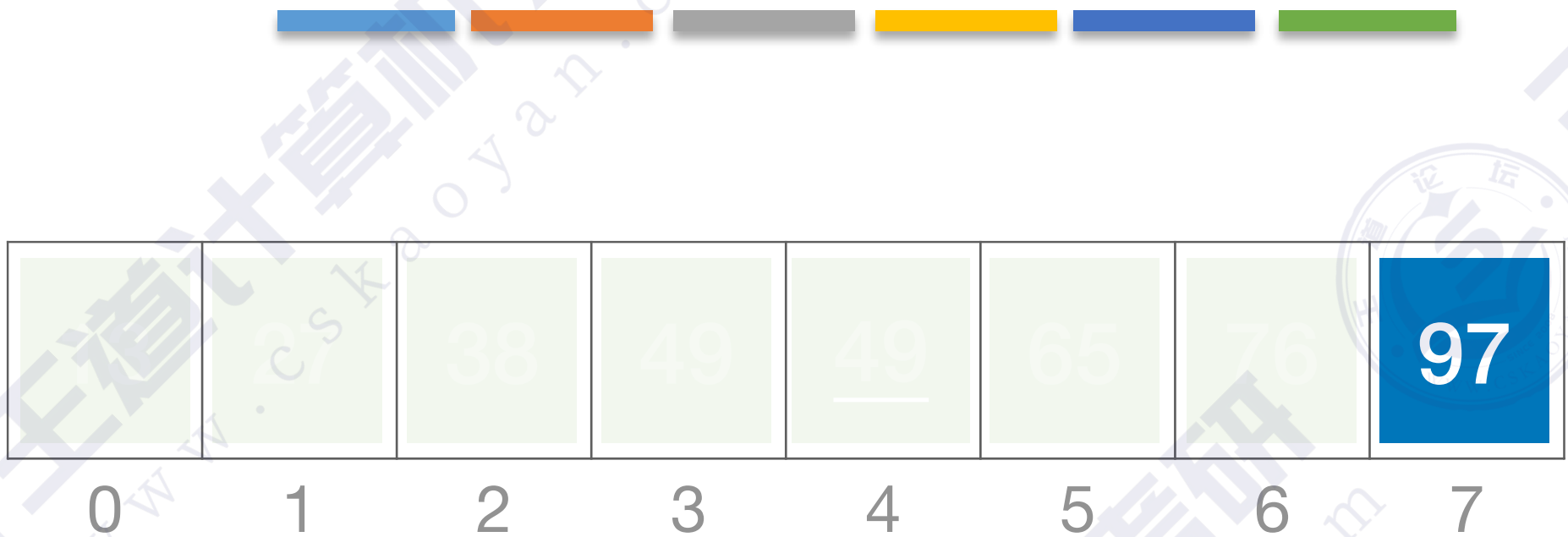
快速排序



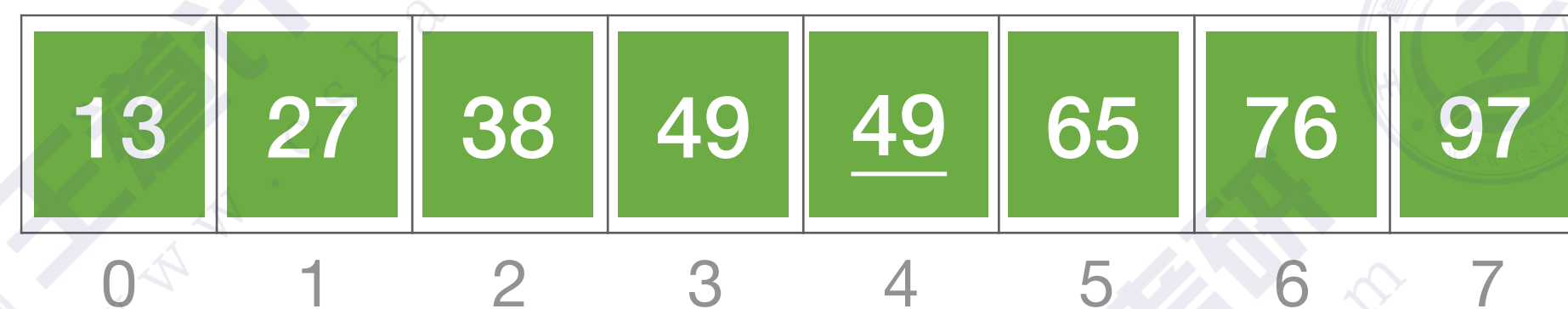
快速排序



快速排序

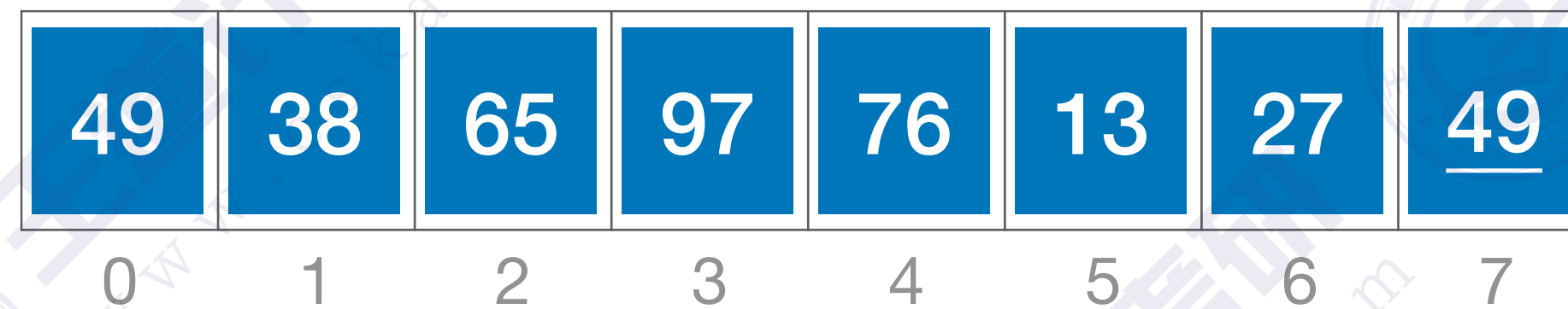


快速排序



算法思想：在待排序表 $L[1...n]$ 中任取一个元素pivot作为枢轴（或基准，通常取首元素），通过一趟排序将待排序表划分为独立的两部分 $L[1...k-1]$ 和 $L[k+1...n]$ ，使得 $L[1...k-1]$ 中的所有元素小于pivot， $L[k+1...n]$ 中的所有元素大于等于pivot，则pivot放在了其最终位置 $L(k)$ 上，这个过程称为一次“划分”。然后分别递归地对两个子表重复上述过程，直至每部分内只有一个元素或空为止，即所有元素放在了其最终位置上。

快速排序



QuickSort
函数

l=0, h=7

递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```

快速排序

49	38	65	97	76	13	27	49
----	----	----	----	----	----	----	----

0

1

2

3

4

5

6

7



low



high

QuickSort
函数

Partition 函数
(处理0~7)

#96, l=0, h=7

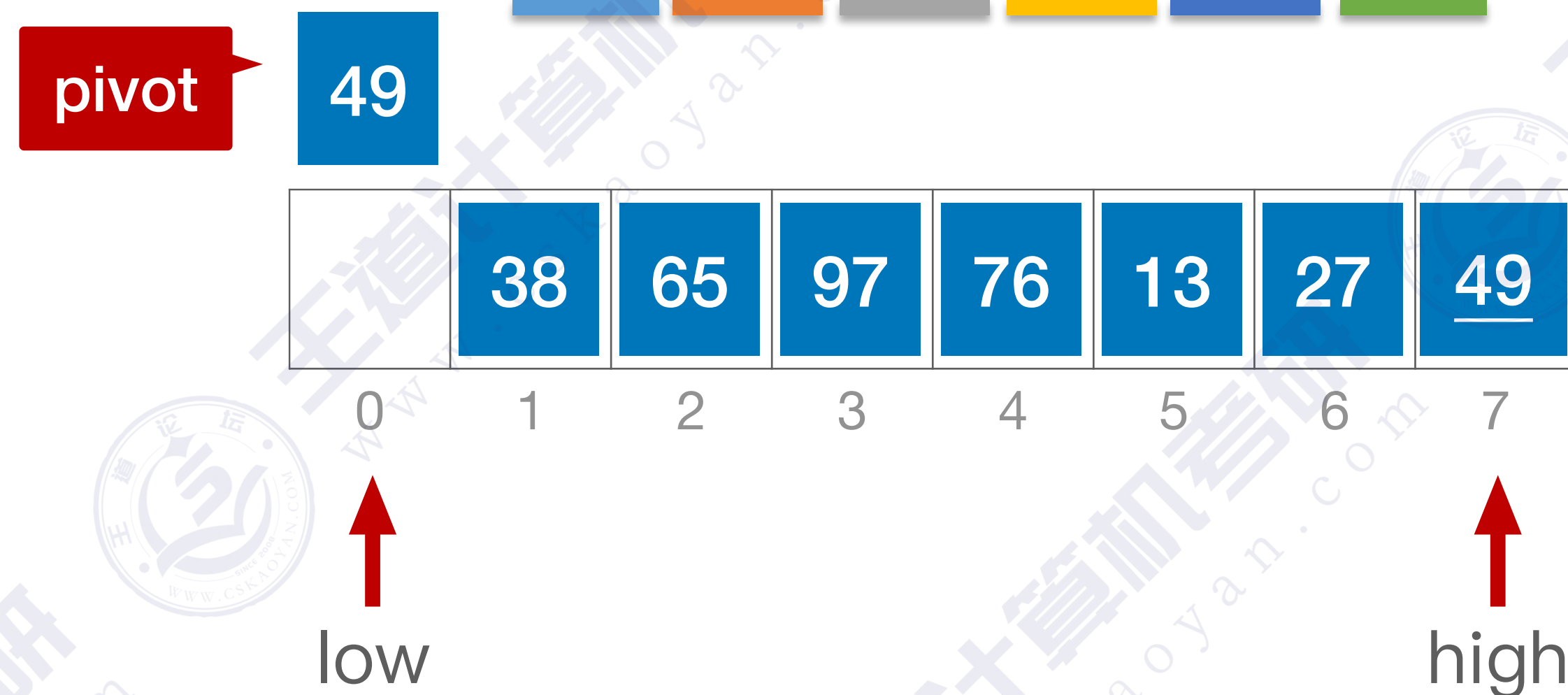
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



Partition 函数
(处理0~7)

#96, l=0, h=7

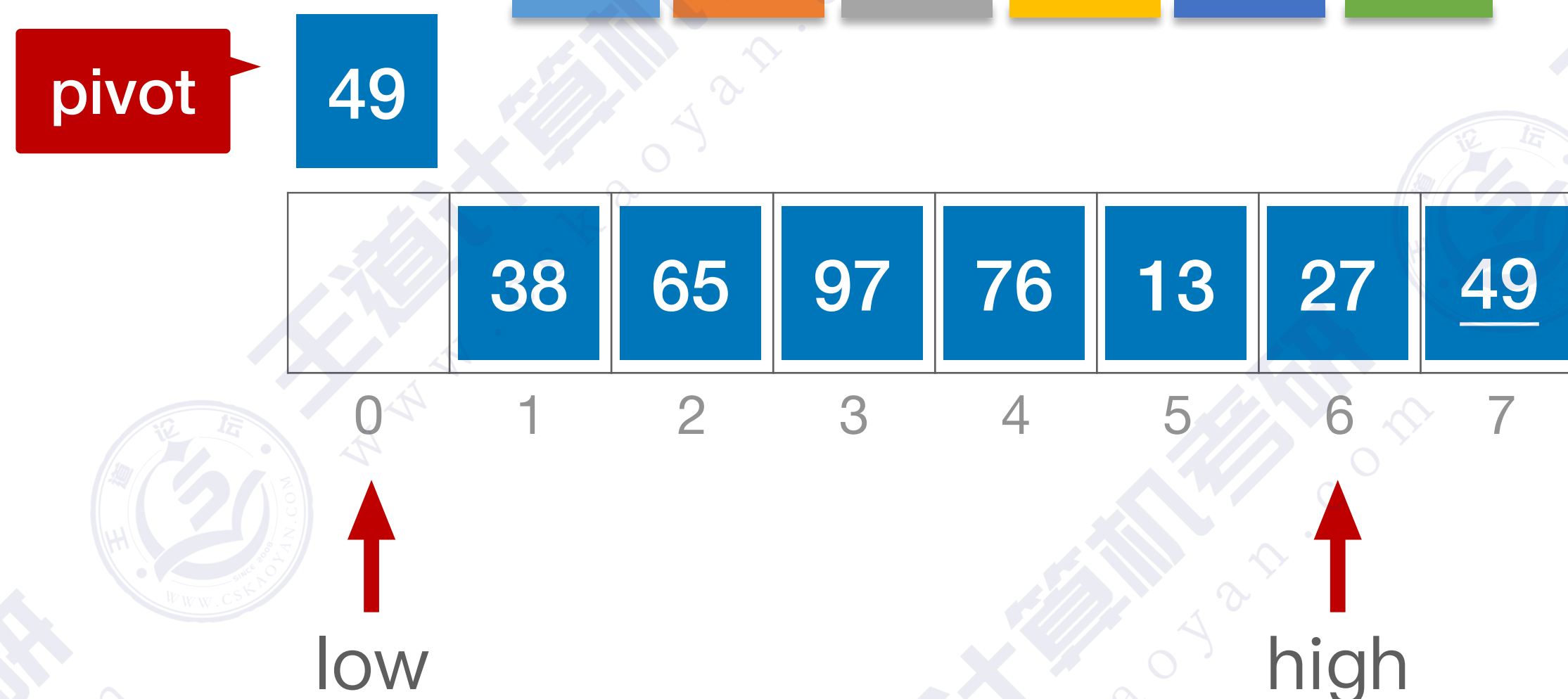
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){  
    int pivot=A[low];           //第一个元素作为枢轴  
    while(low<high){           //用low、high搜索枢轴的最终位置  
        while(low<high&&A[high]>=pivot) --high;  
        A[low]=A[high];         //比枢轴小的元素移动到左端  
        while(low<high&&A[low]<=pivot) ++low;  
        A[high]=A[low];         //比枢轴大的元素移动到右端  
    }  
    A[low]=pivot;               //枢轴元素存放到最后位置  
    return low;                 //返回存放枢轴的最终位置  
}
```

```
93 //快速排序  
94 void QuickSort(int A[],int low,int high){  
95     if(low<high){           //递归跳出的条件  
96         int pivotpos=Partition(A,low,high); //划分  
97         QuickSort(A,low,pivotpos-1);       //划分左子表  
98         QuickSort(A,pivotpos+1,high);      //划分右子表  
99     }  
100 }
```


快速排序



Partition 函数
(处理0~7)

#96, l=0, h=7

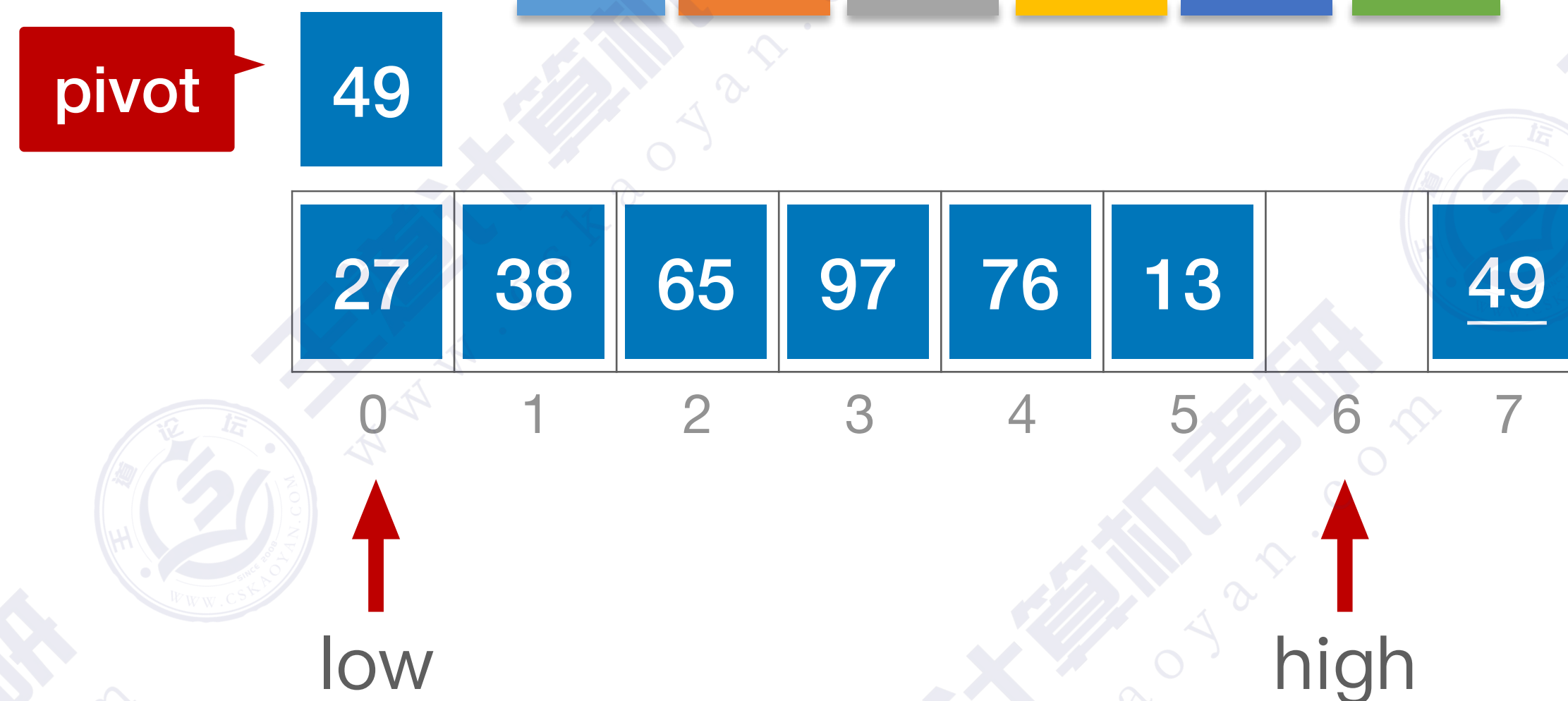
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){           //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1);        //划分左子表
98         QuickSort(A,pivotpos+1,high);        //划分右子表
99     }
100 }
```

快速排序



Partition 函数
(处理0~7)

#96, l=0, h=7

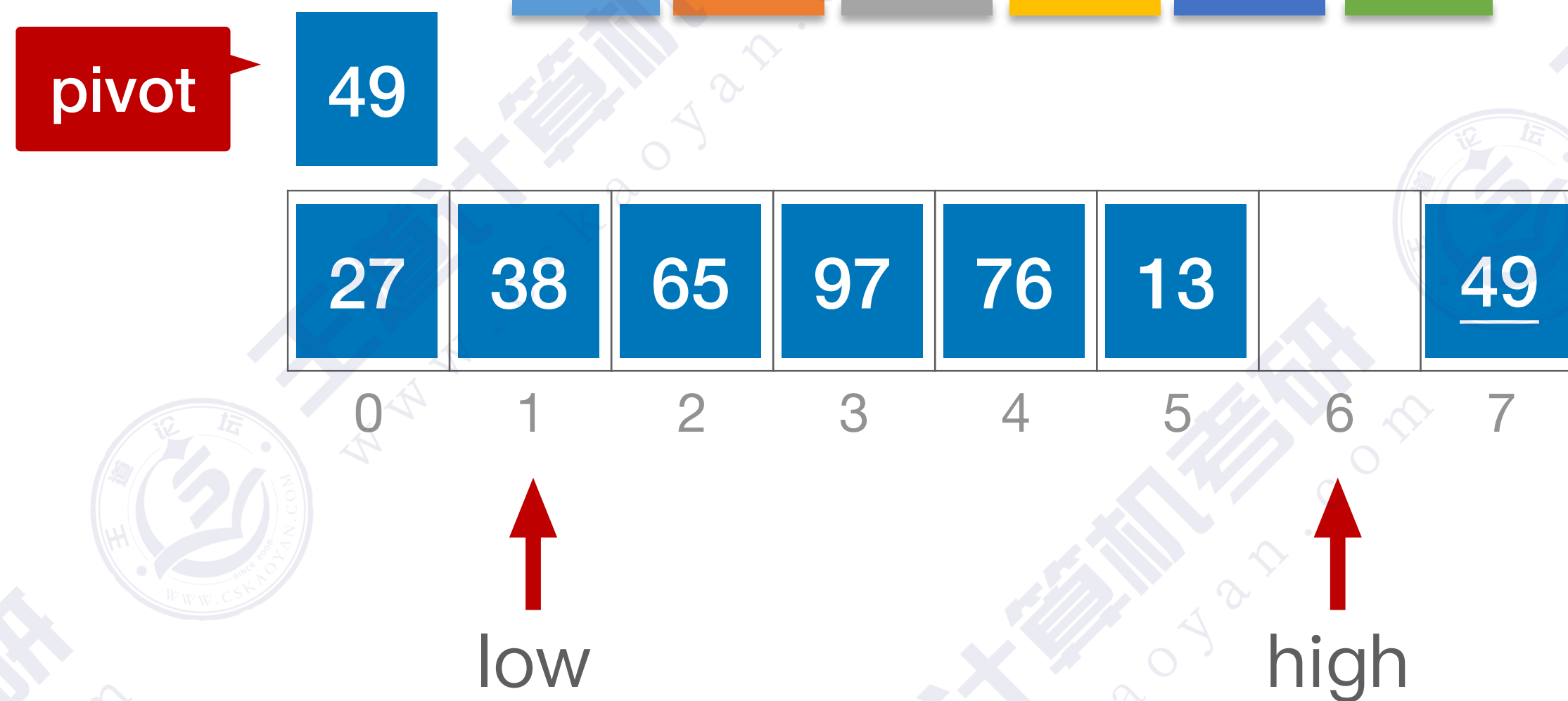
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



Partition 函数
(处理0~7)

#96, l=0, h=7

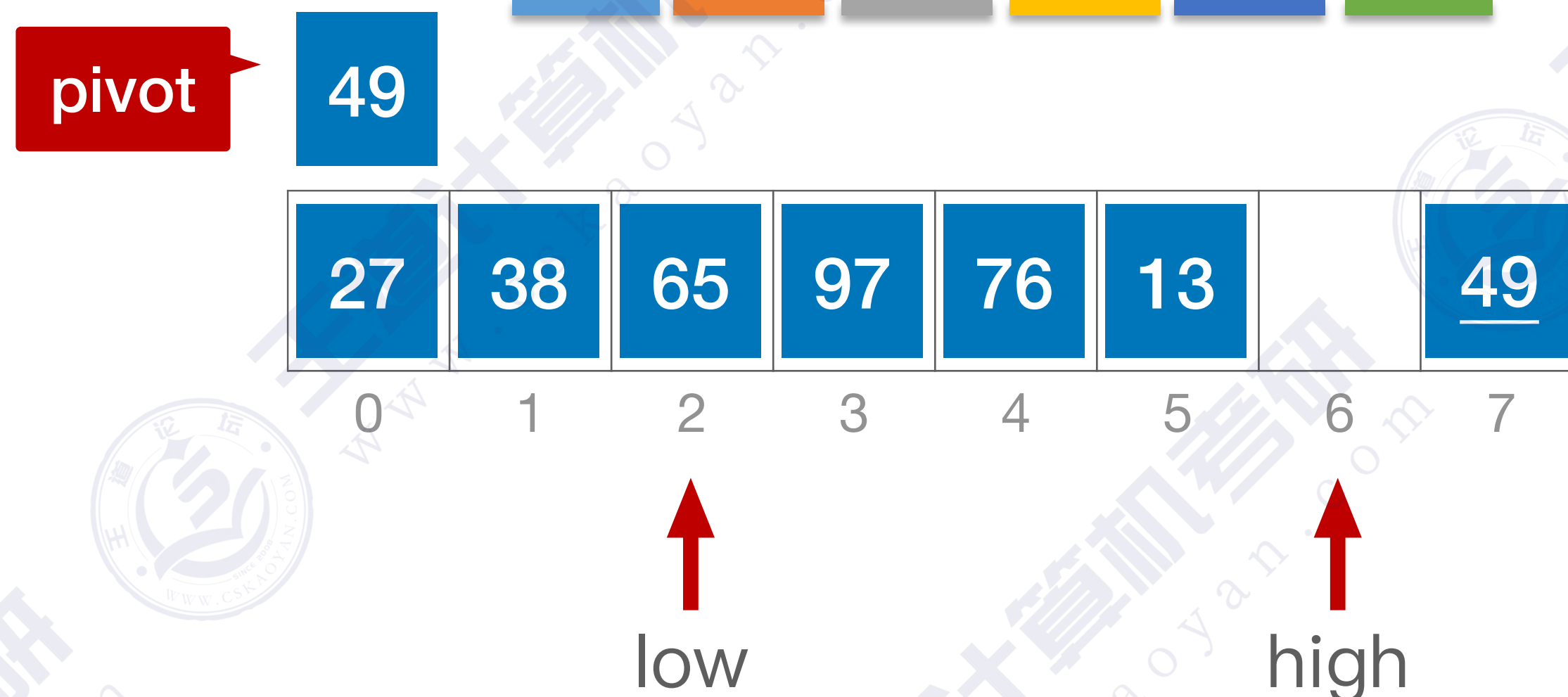
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){            //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1);        //划分左子表
98         QuickSort(A,pivotpos+1,high);        //划分右子表
99     }
100 }
```


快速排序



Partition 函数
(处理0~7)

#96, l=0, h=7

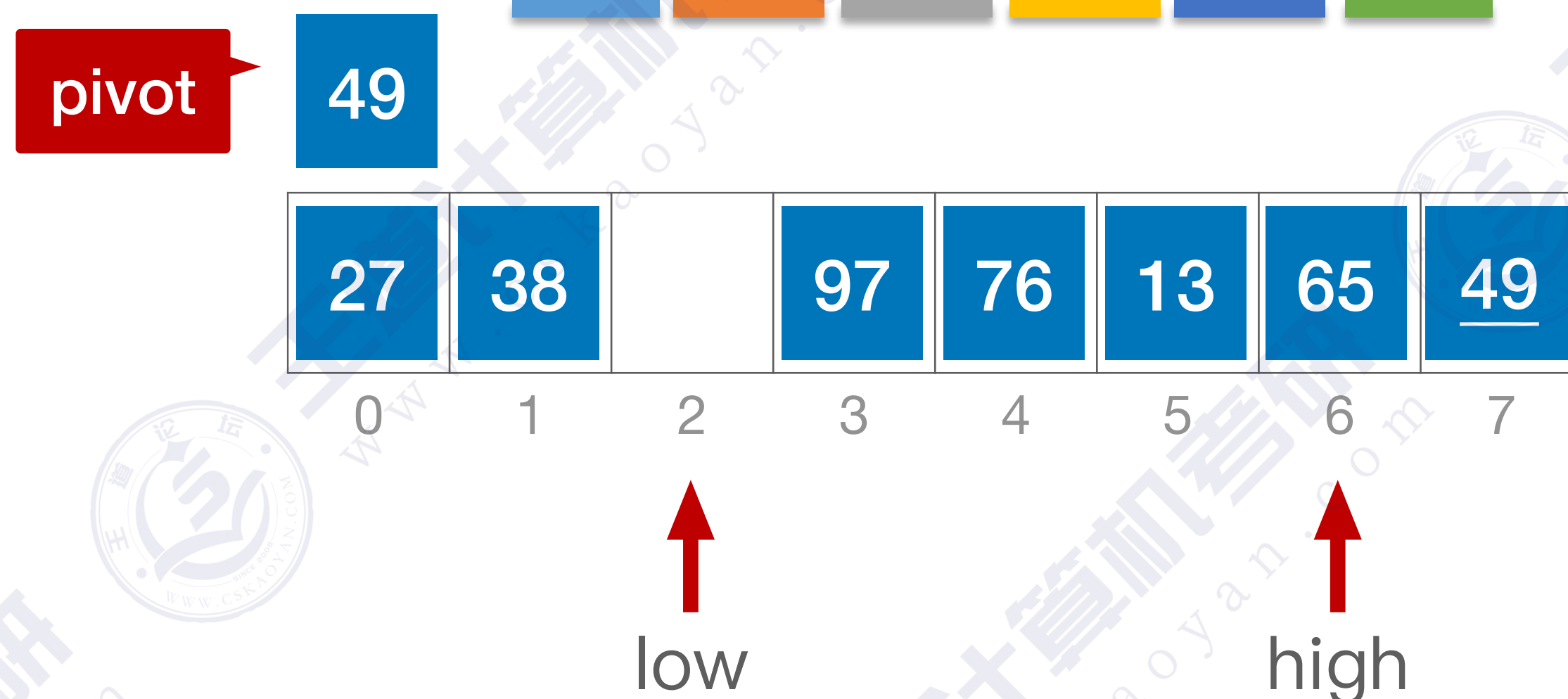
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){            //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1);        //划分左子表
98         QuickSort(A,pivotpos+1,high);        //划分右子表
99     }
100 }
```

快速排序



Partition 函数
(处理0~7)

#96, l=0, h=7

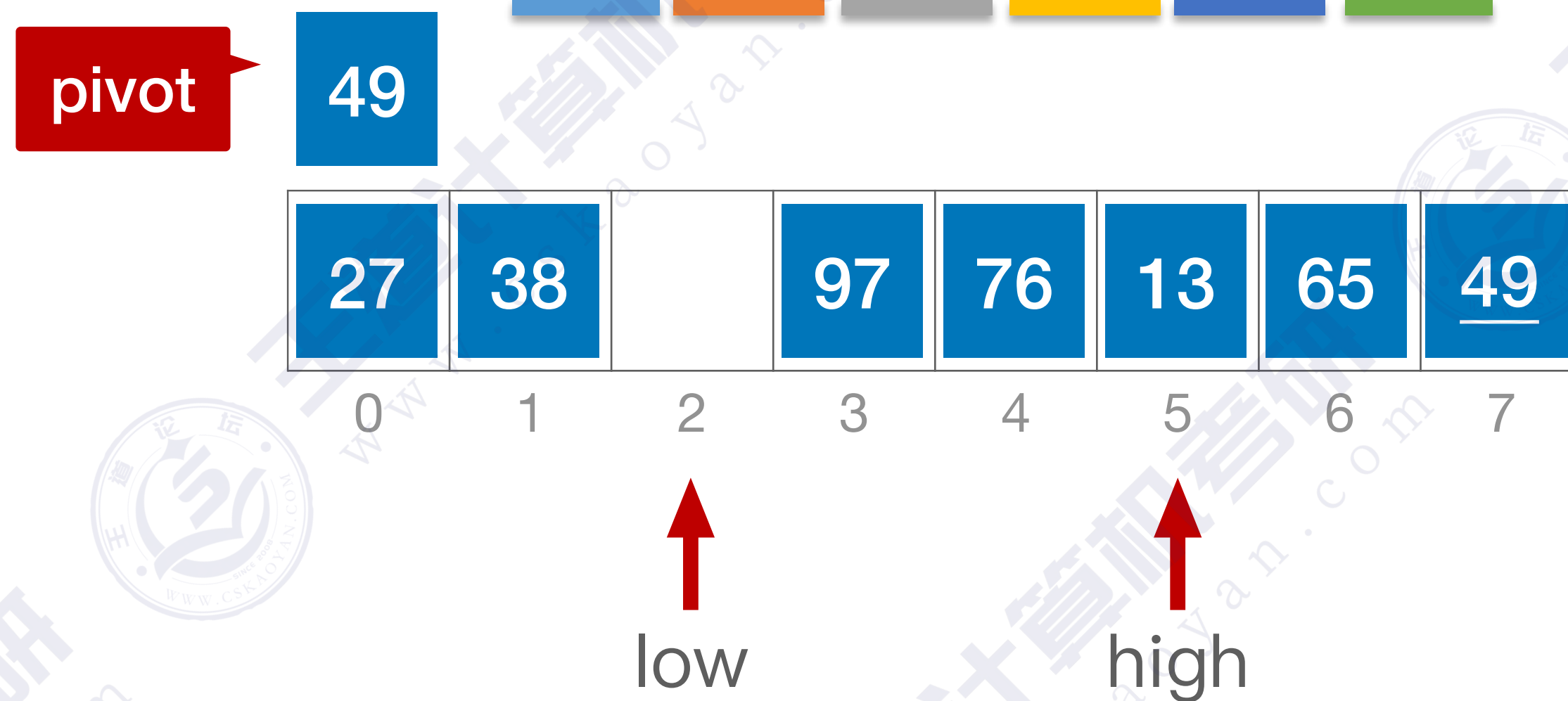
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



Partition 函数
(处理0~7)

#96, l=0, h=7

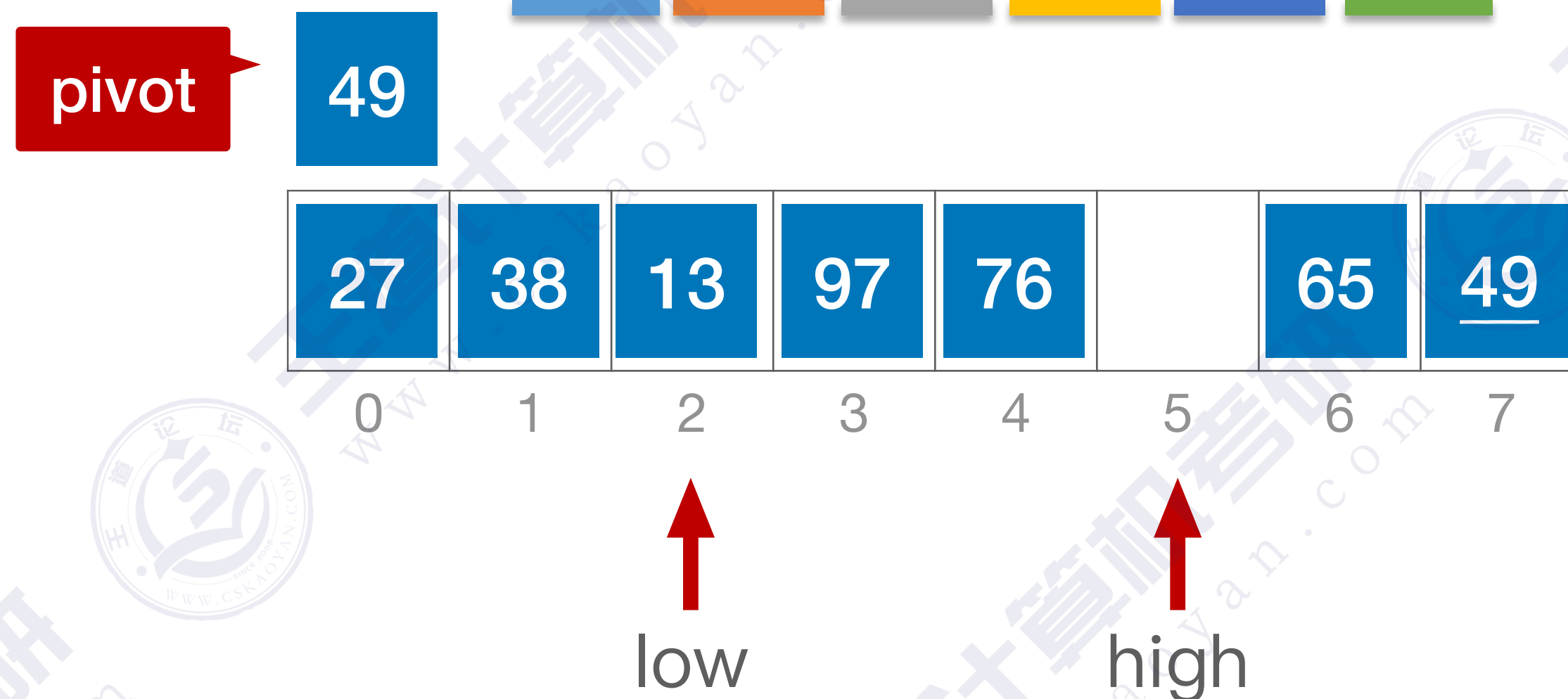
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



Partition 函数
(处理0~7)

#96, l=0, h=7

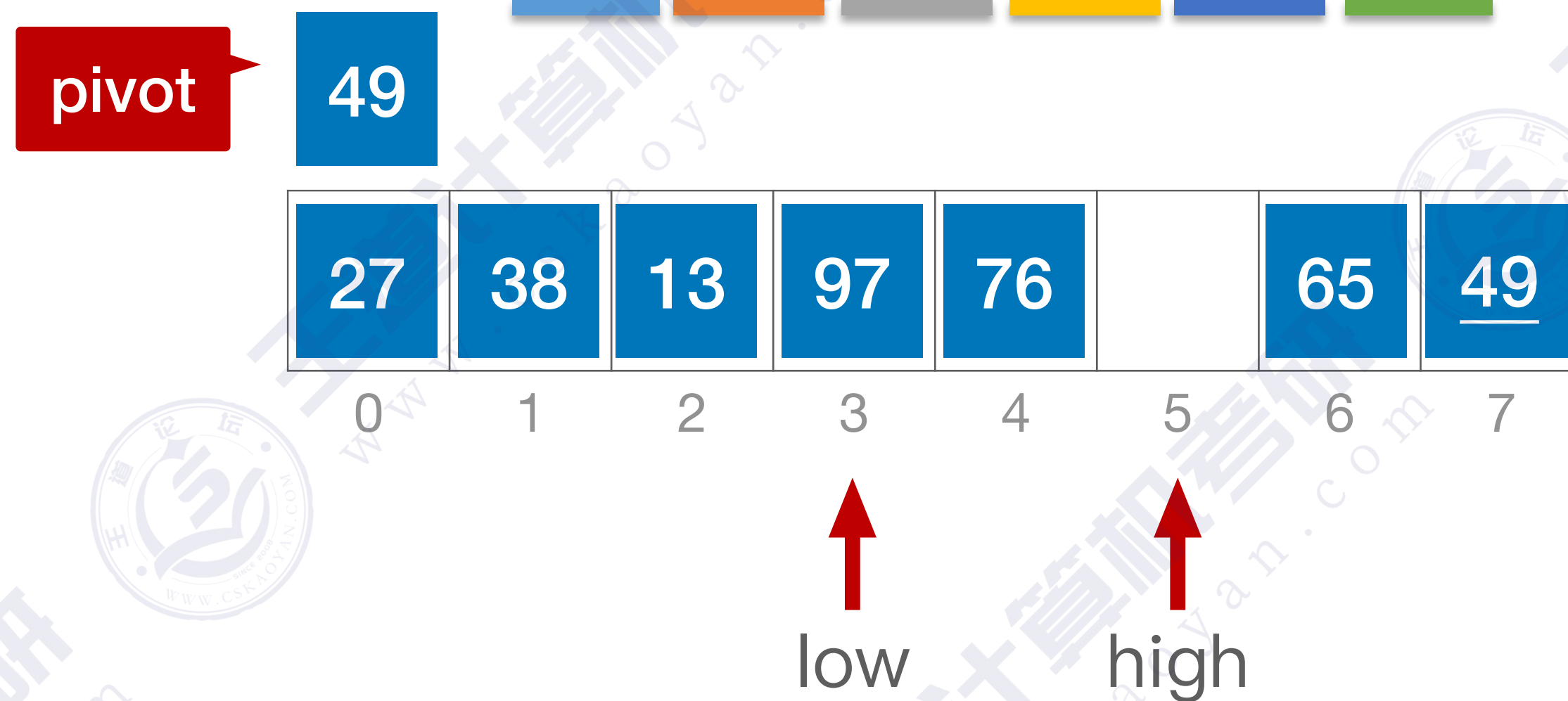
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        → A[low]=A[high];        //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```

快速排序



Partition 函数
(处理0~7)

#96, l=0, h=7

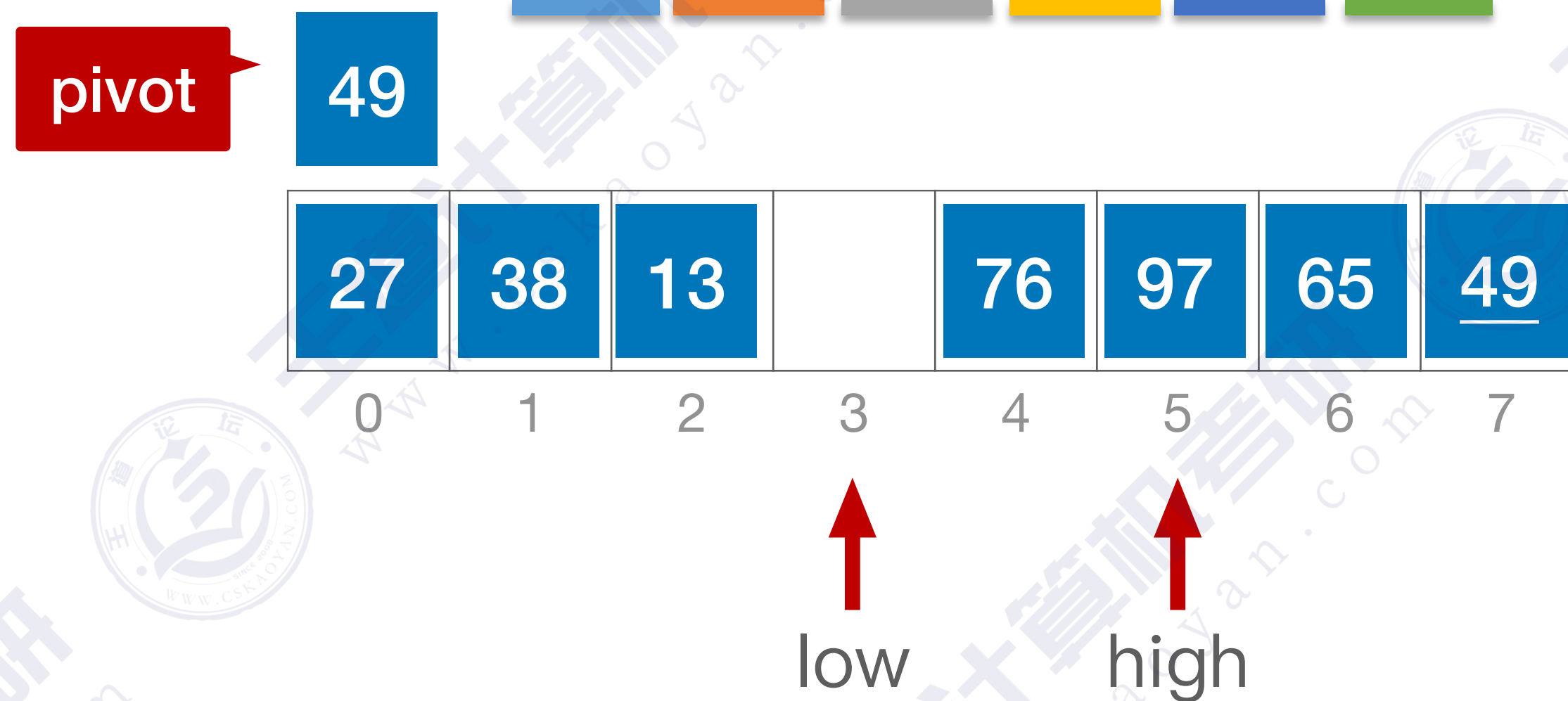
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



Partition 函数
(处理0~7)

#96, l=0, h=7

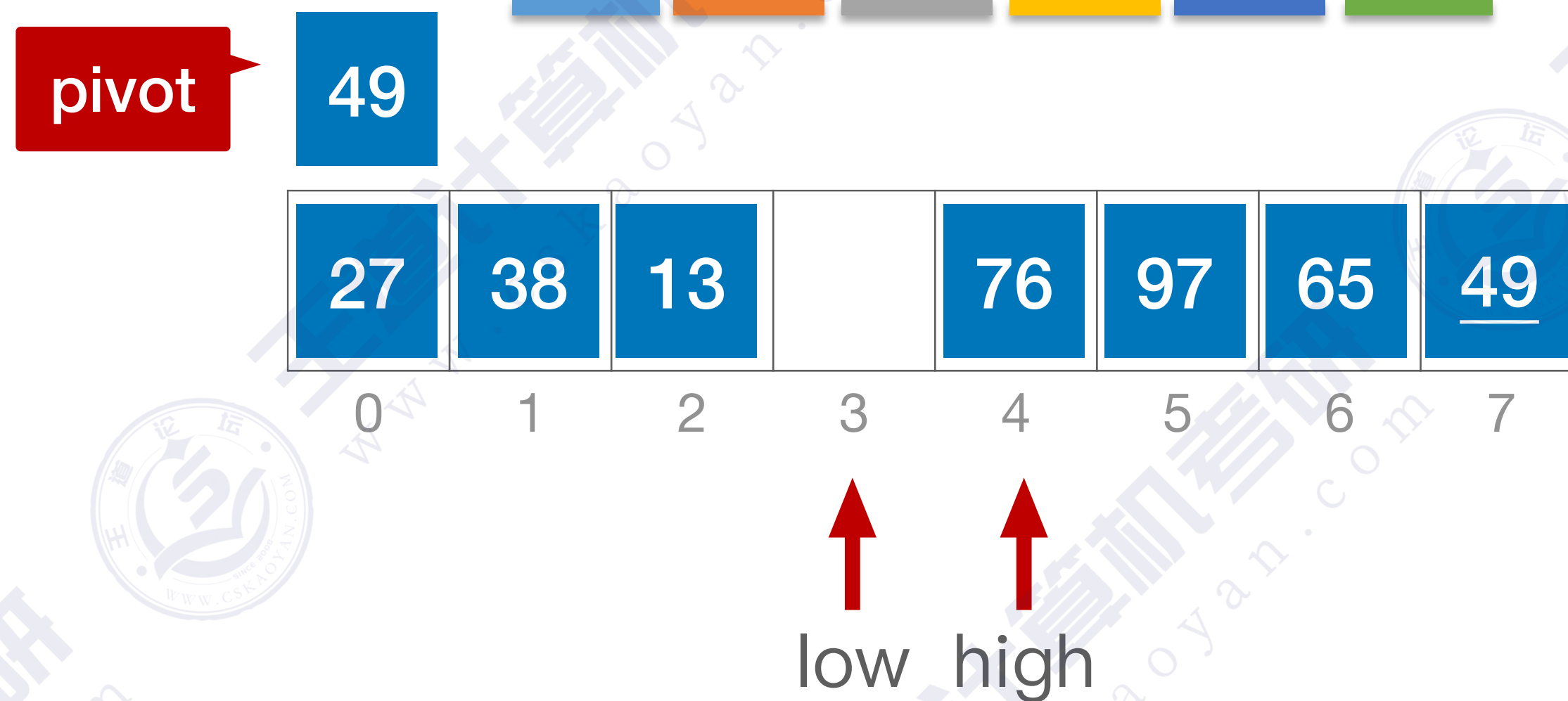
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



Partition 函数
(处理0~7)

#96, l=0, h=7

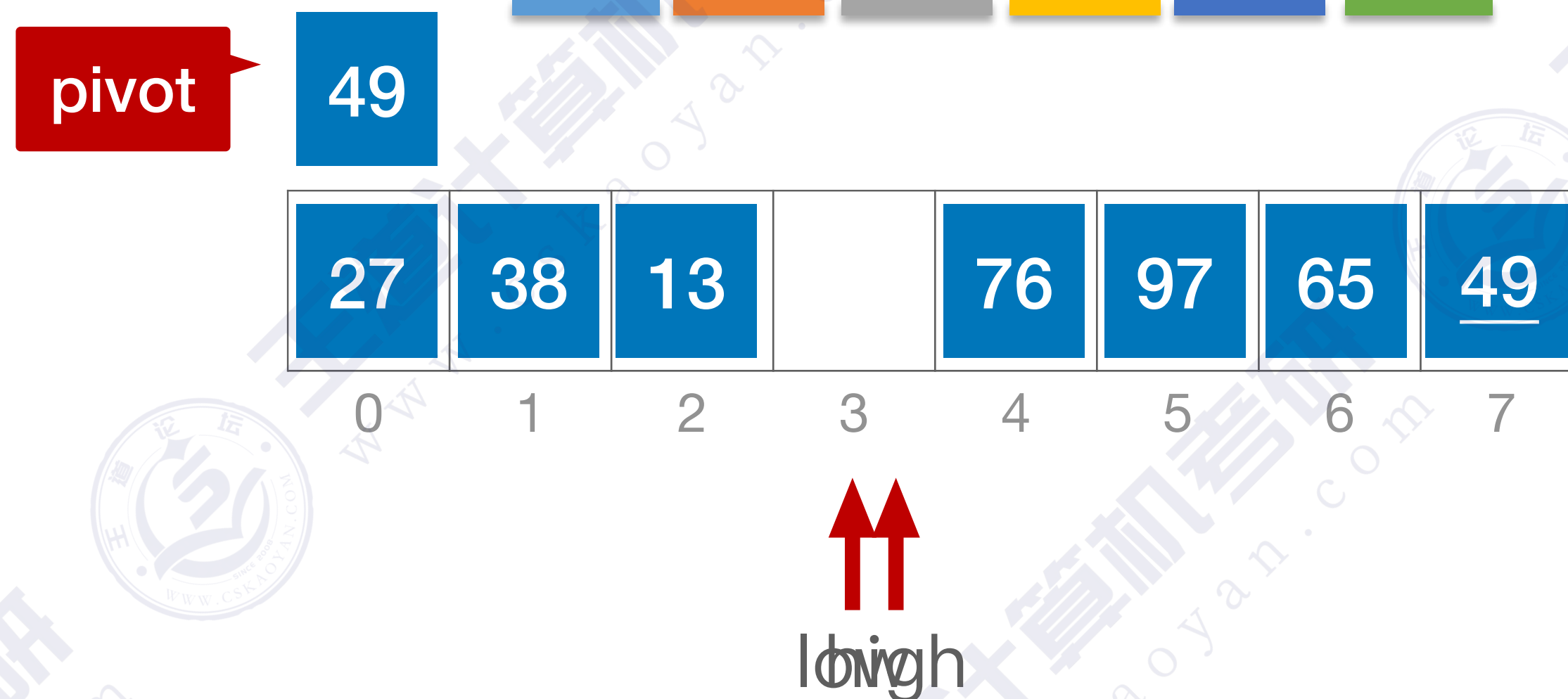
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){           //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1);        //划分左子表
98         QuickSort(A,pivotpos+1,high);        //划分右子表
99     }
100 }
```

快速排序



Partition 函数
(处理0~7)

#96, l=0, h=7

递归工作栈

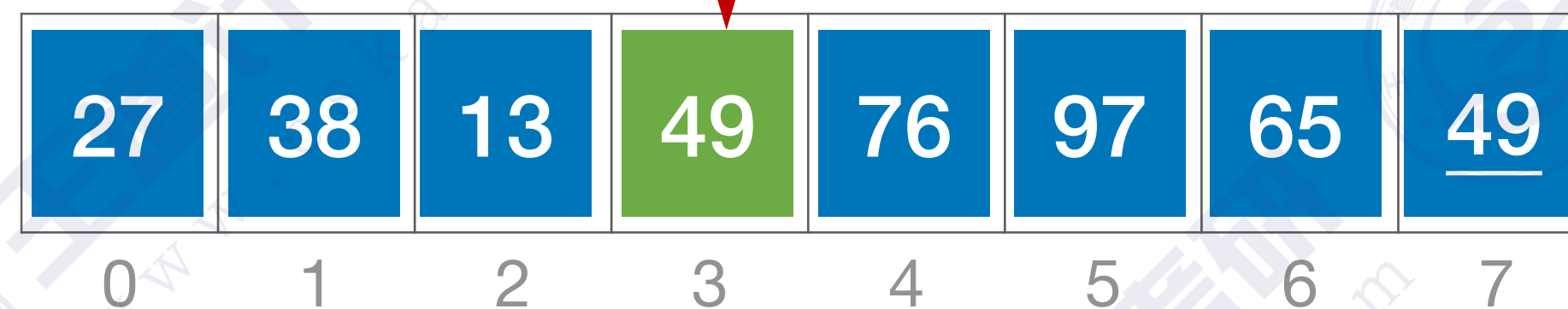
//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];          //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];          //比枢轴大的元素移动到右端
    }
    A[low]=pivot;                //枢轴元素存放到最后位置
    return low;                  //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序

枢轴（基准）元素



用第一个元素把待排序序列“划分”为两个部分。左边更小，右边更大。该元素的最终位置已确定

↑↑
low high

Partition 函数
(处理0~7)

#96, l=0, h=7

递归工作栈

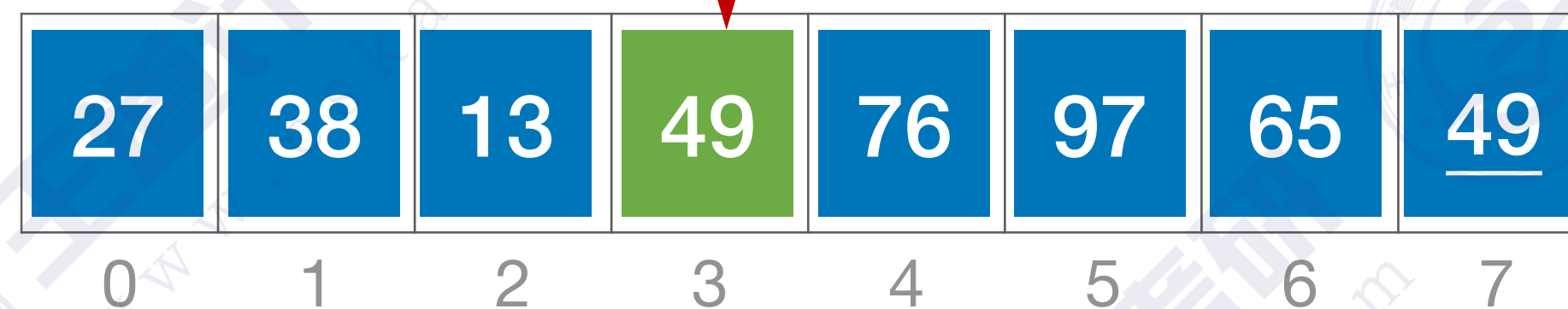
//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){             //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];          //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];          //比枢轴大的元素移动到右端
    }
    A[low]=pivot;                //枢轴元素存放到最后位置
    return low;                  //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序

枢轴（基准）元素



#96, l=0, h=7
p=3

递归工作栈

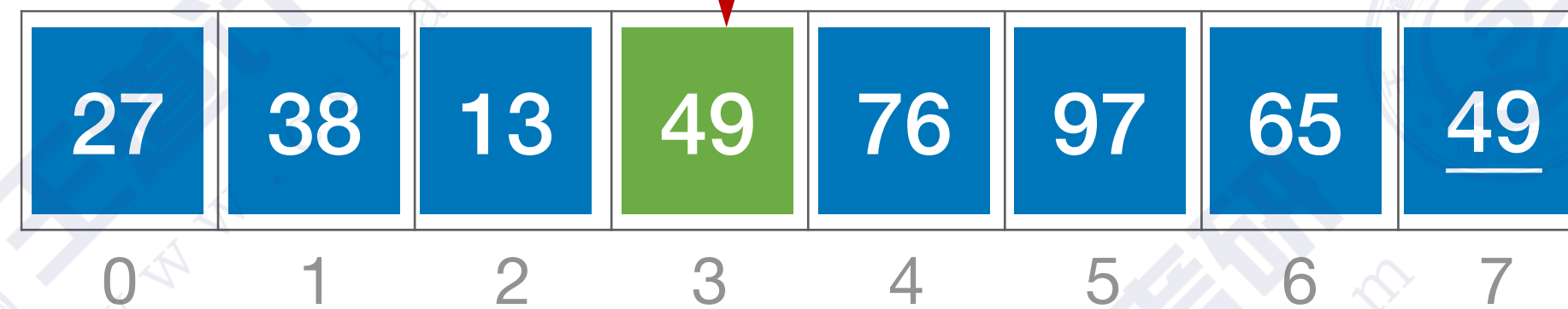
//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         → int pivotpos=Partition(A,low,high); //划分
97             QuickSort(A,low,pivotpos-1); //划分左子表
98             QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```

快速排序

枢轴（基准）元素



#96, l=0, h=7
p=3

递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){  
    int pivot=A[low];           //第一个元素作为枢轴  
    while(low<high){             //用low、high搜索枢轴的最终位置  
        while(low<high&&A[high]>=pivot) --high;  
        A[low]=A[high];          //比枢轴小的元素移动到左端  
        while(low<high&&A[low]<=pivot) ++low;  
        A[high]=A[low];          //比枢轴大的元素移动到右端  
    }  
    A[low]=pivot;                //枢轴元素存放到最后位置  
    return low;                  //返回存放枢轴的最终位置  
}
```

```
93 //快速排序  
94 void QuickSort(int A[],int low,int high){  
95     if(low<high){ //递归跳出的条件  
96         int pivotpos=Partition(A,low,high); //划分  
97         QuickSort(A,low,pivotpos-1); //划分左子表  
98         QuickSort(A,pivotpos+1,high); //划分右子表  
99     }  
100 }
```


快速排序



第二层
QuickSort

$l=0, h=2$

#97, $l=0, h=7$
 $p=3$

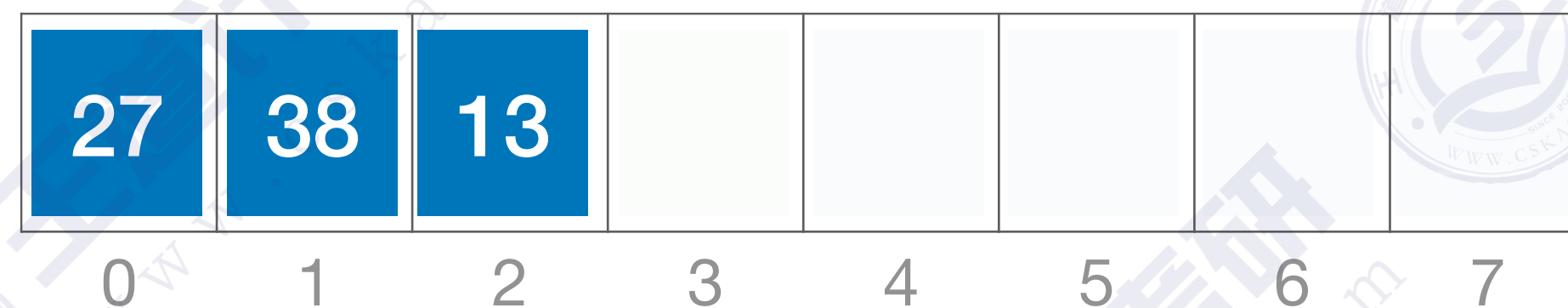
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第二层
QuickSort

#96, l=0, h=2
#97, l=0, h=7
p=3

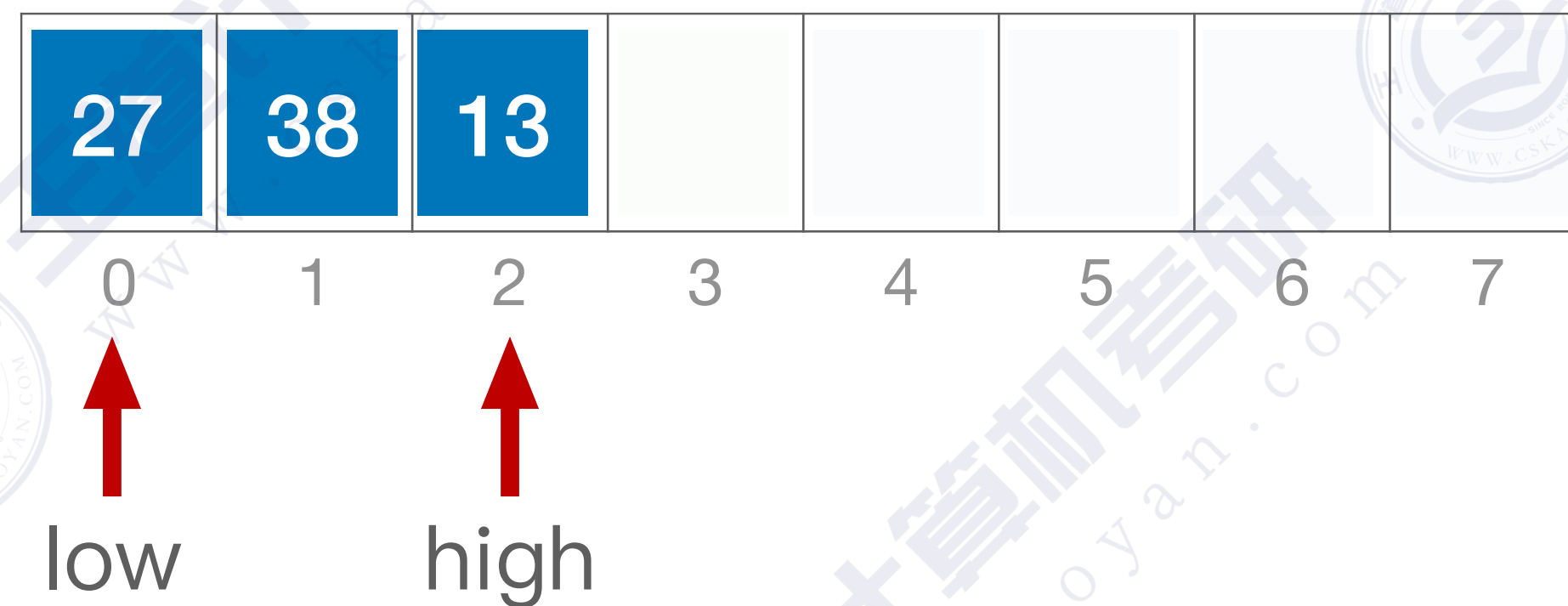
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){           //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){           //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1);         //划分左子表
98         QuickSort(A,pivotpos+1,high);         //划分右子表
99     }
100 }
```

快速排序



第二层
QuickSort

Partition 函数
(处理0~2)

#96, l=0, h=2

#97, l=0, h=7
p=3

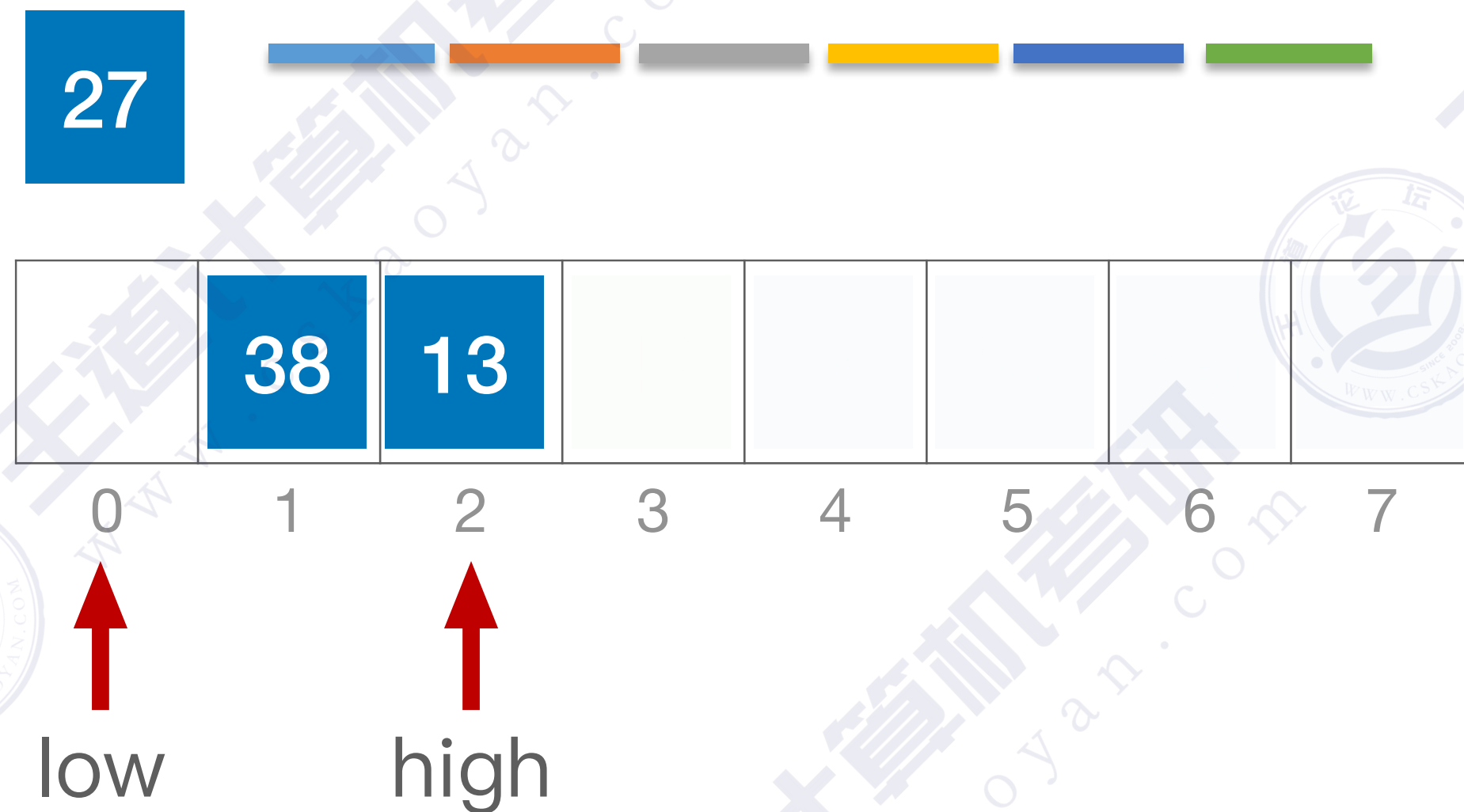
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){             //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];          //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];          //比枢轴大的元素移动到右端
    }
    A[low]=pivot;                //枢轴元素存放到最后位置
    return low;                  //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第二层
QuickSort

Partition 函数
(处理0~2)

#96, l=0, h=2

#97, l=0, h=7
p=3

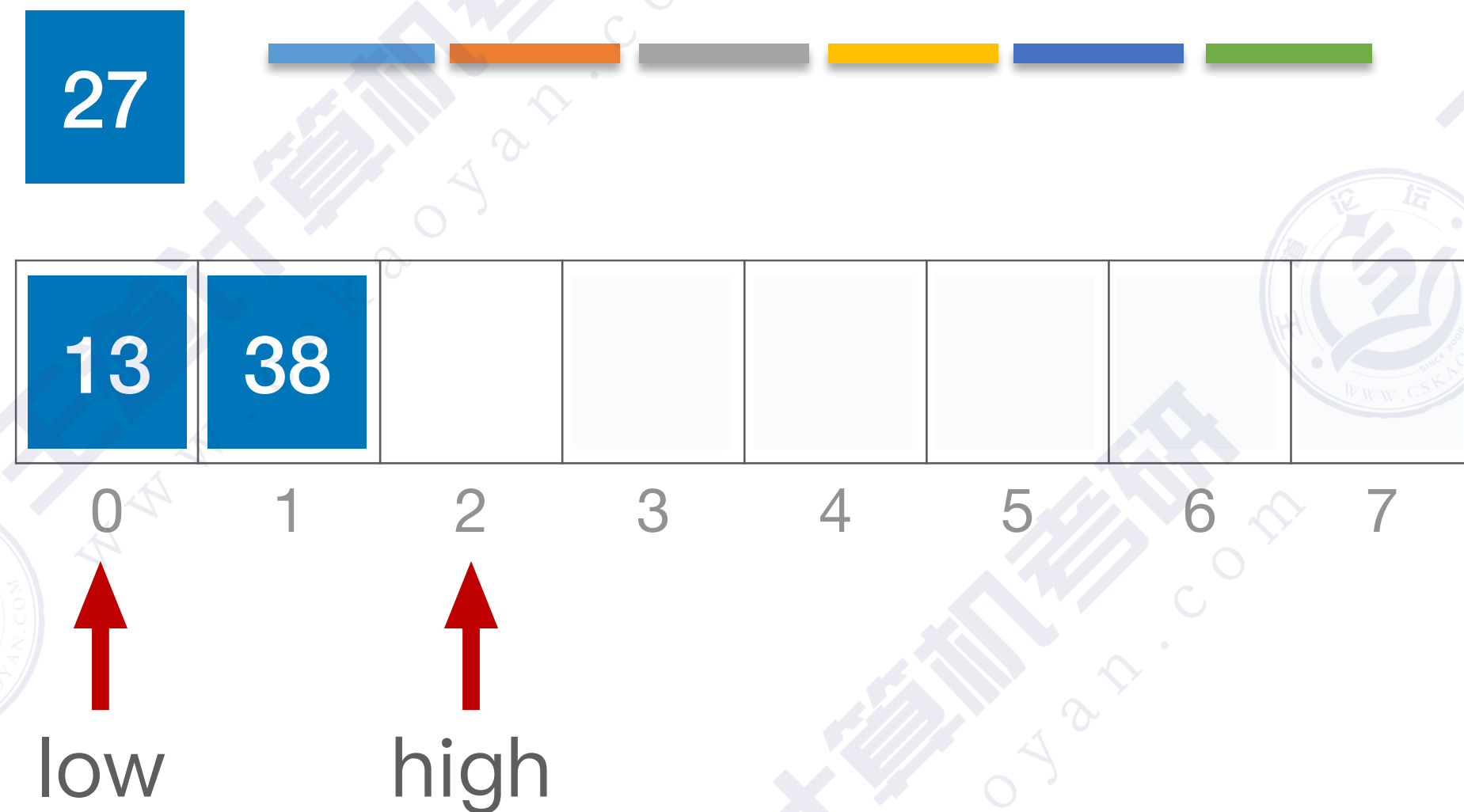
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第二层
QuickSort

Partition 函数
(处理0~2)

#96, l=0, h=2

#97, l=0, h=7
p=3

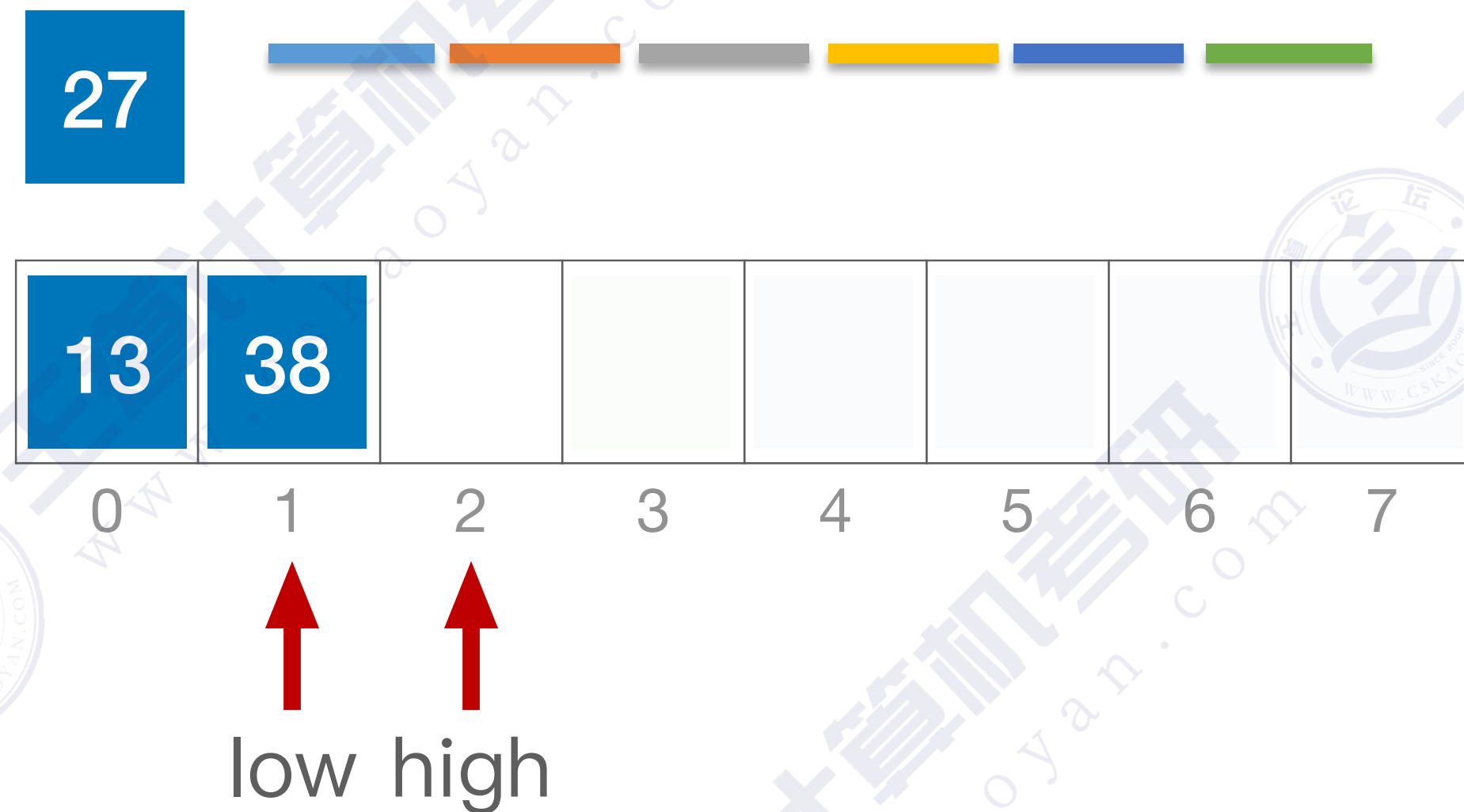
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```

快速排序



第二层
QuickSort

Partition 函数
(处理0~2)

#96, l=0, h=2

#97, l=0, h=7
p=3

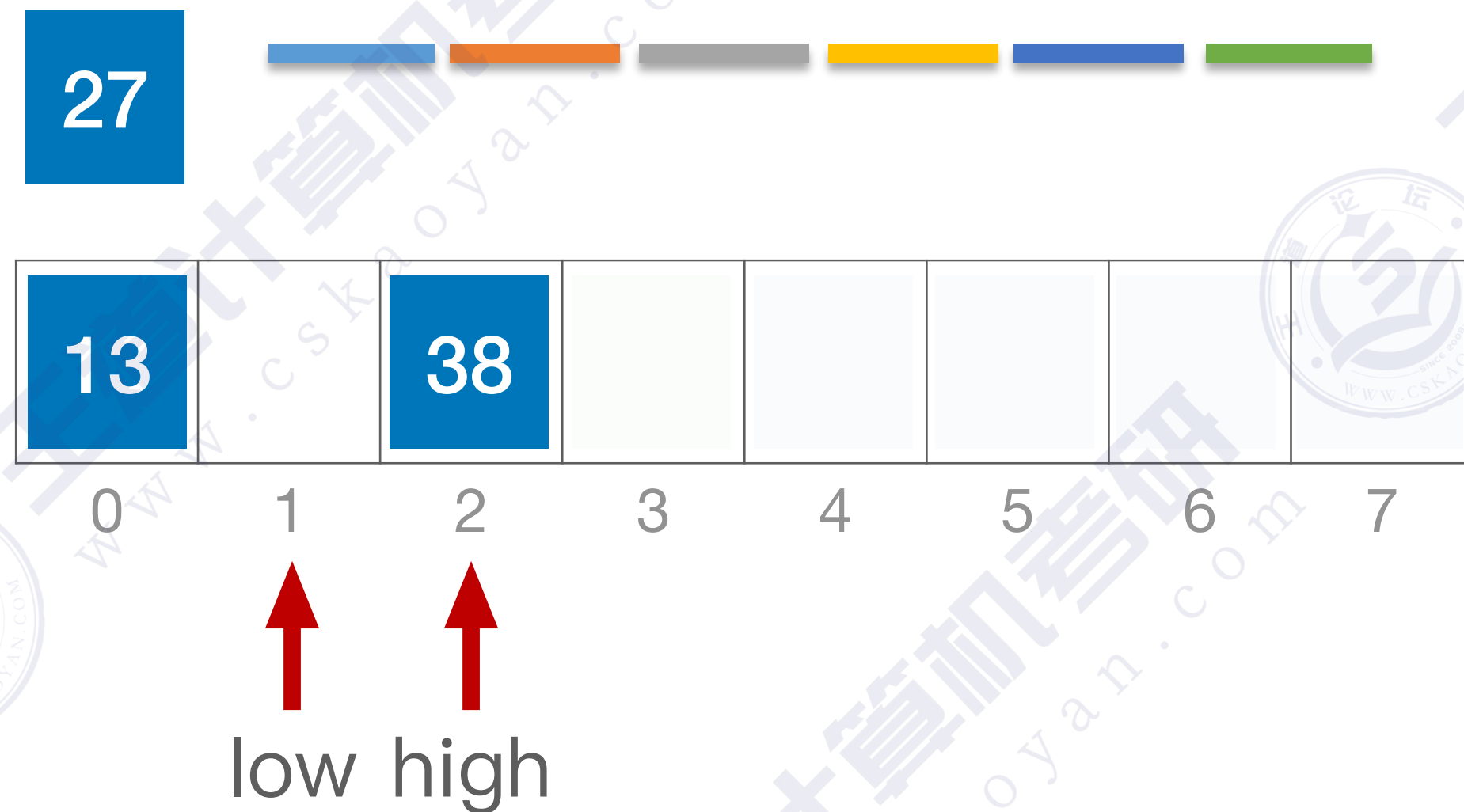
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第二层
QuickSort

Partition 函数
(处理0~2)

#96, l=0, h=2

#97, l=0, h=7
p=3

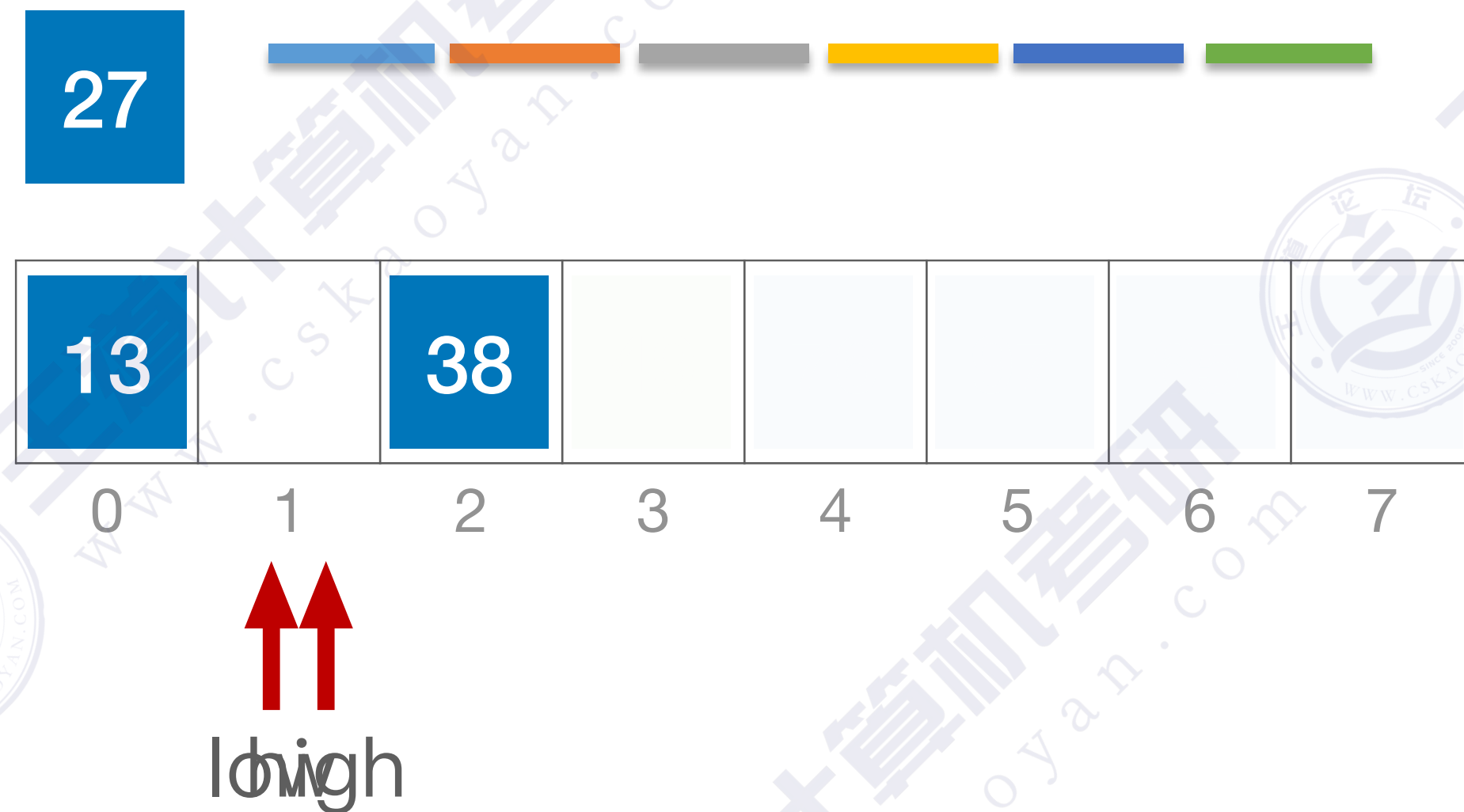
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第二层
QuickSort

Partition 函数
(处理0~2)

#96, l=0, h=2

#97, l=0, h=7
p=3

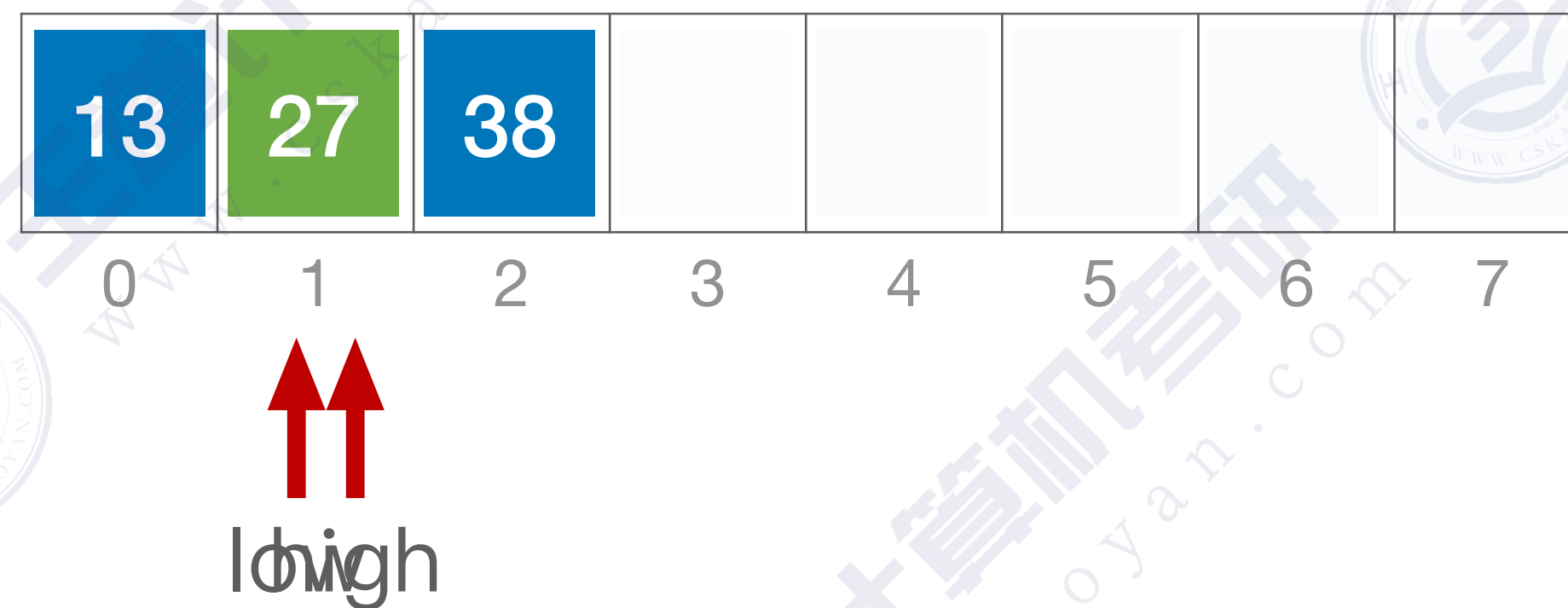
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```

快速排序



第二层
QuickSort

Partition 函数
(处理0~2)

#96, l=0, h=2

#97, l=0, h=7
p=3

递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){            //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1);        //划分左子表
98         QuickSort(A,pivotpos+1,high);        //划分右子表
99     }
100 }
```


快速排序



第二层
QuickSort

#96, l=0, h=2 p=1
#97, l=0, h=7 p=3

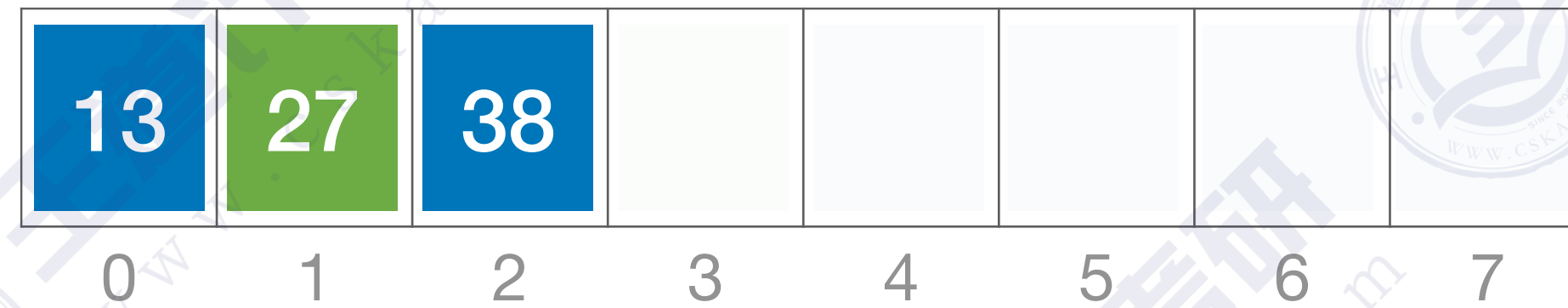
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第二层
QuickSort

#96, l=0, h=2 p=1
#97, l=0, h=7 p=3

递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){             //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];          //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;                //枢轴元素存放到最后位置
    return low;                  //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```

快速排序



第三层
QuickSort

$l=0, h=0$
#97, $l=0, h=2$ $p=1$
#97, $l=0, h=7$ $p=3$

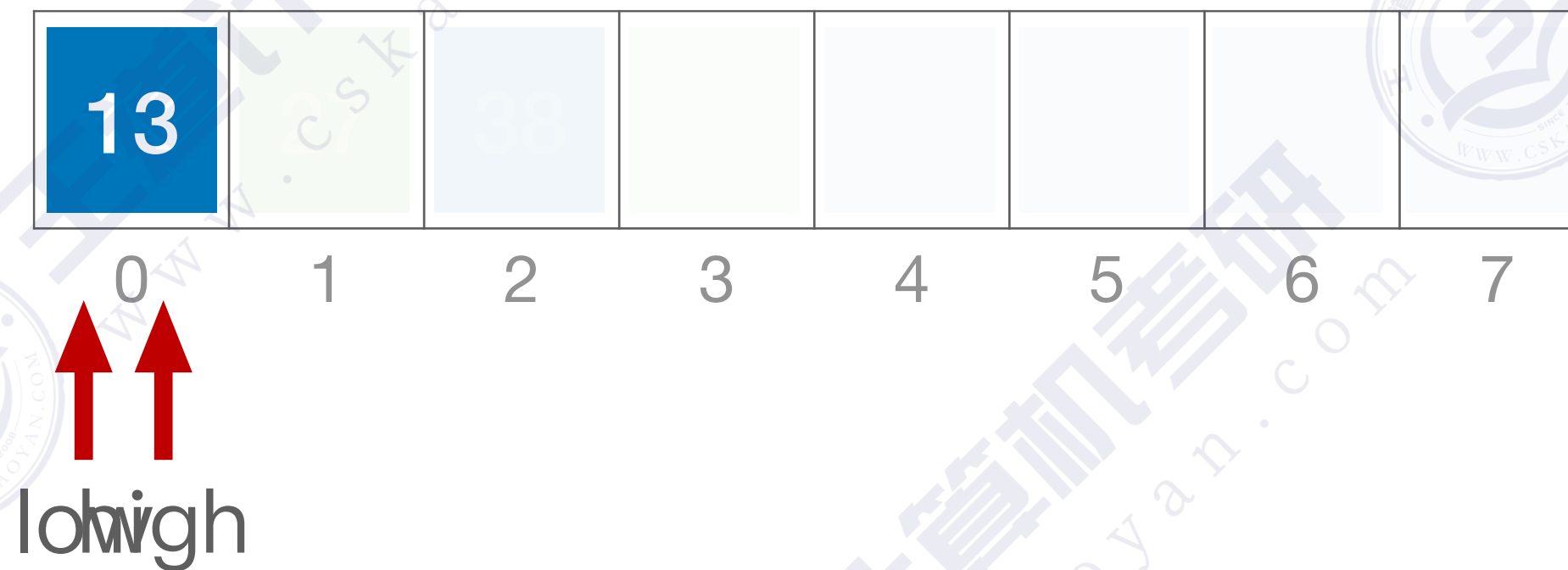
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){             //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];          //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];          //比枢轴大的元素移动到右端
    }
    A[low]=pivot;                //枢轴元素存放到最后位置
    return low;                  //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第三层
QuickSort

$l=0, h=0$
#97, $l=0, h=2$ $p=1$
#97, $l=0, h=7$ $p=3$

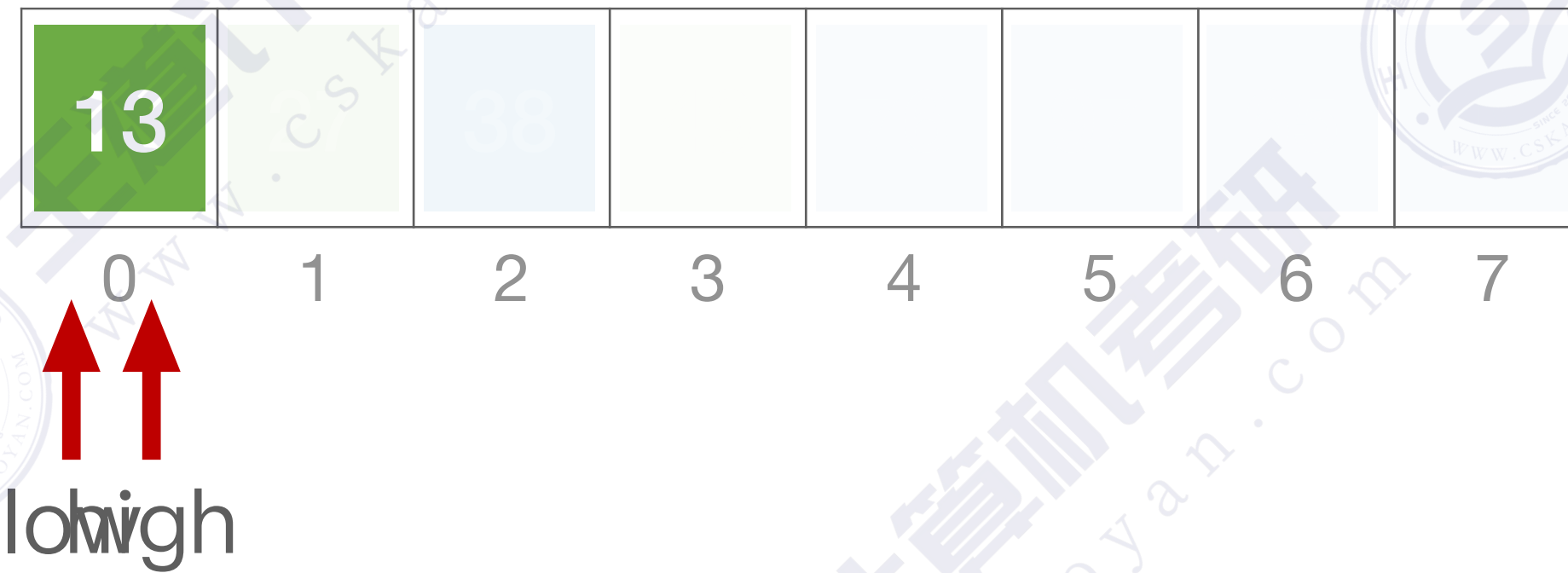
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){           //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1);        //划分左子表
98         QuickSort(A,pivotpos+1,high);        //划分右子表
99     }
100 }
```


快速排序



第三层
QuickSort

$l=0, h=0$
#97, $l=0, h=2$ $p=1$
#97, $l=0, h=7$ $p=3$

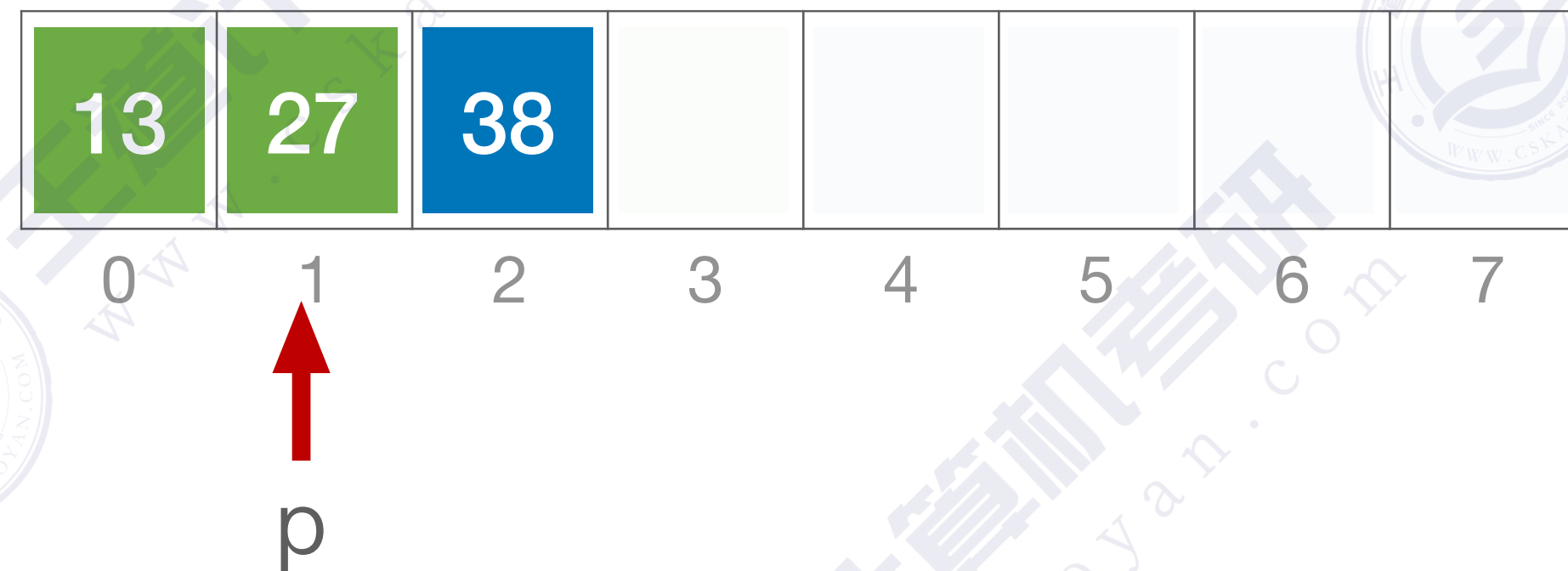
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```

快速排序



第二层
QuickSort

#98, l=0, h=2
p=1
#97, l=0, h=7
p=3

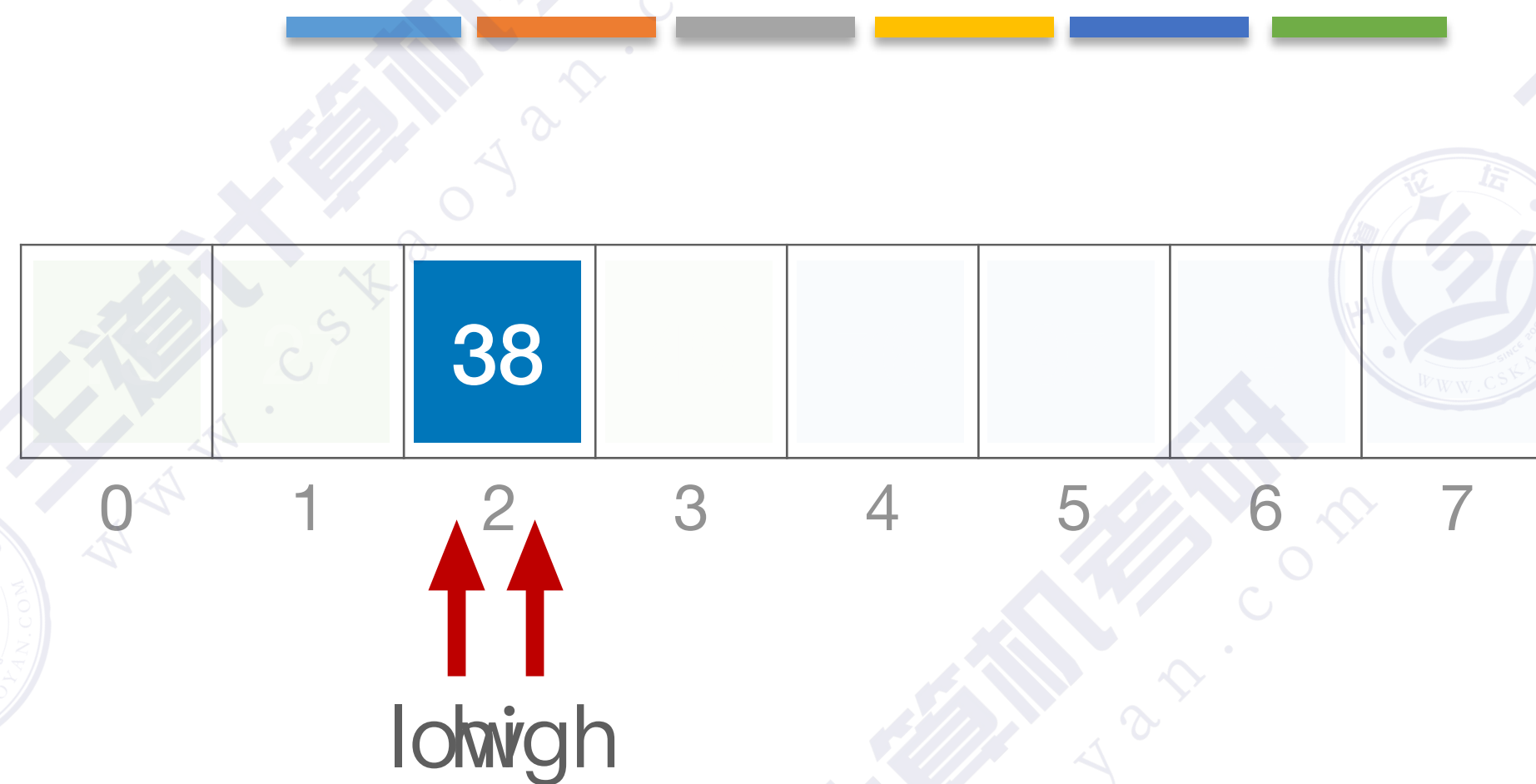
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最终位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第三层
QuickSort

$l=2, h=2$
#98, $l=0, h=2$ $p=1$
#97, $l=0, h=7$ $p=3$

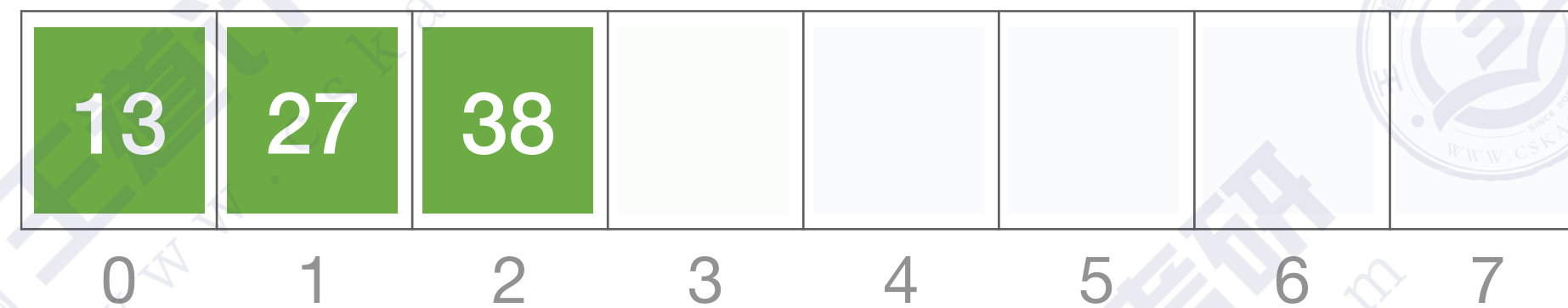
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第二层
QuickSort

#98, l=0, h=2
p=1
#97, l=0, h=7
p=3

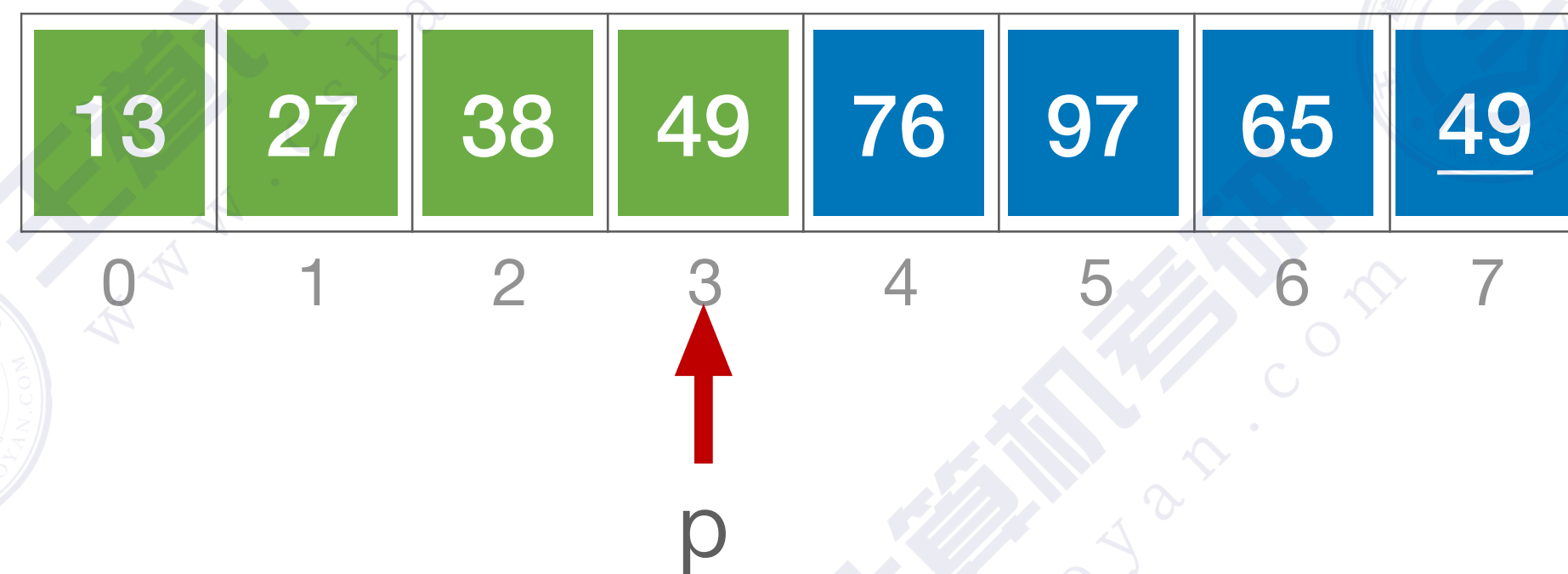
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```

快速排序



第一层
QuickSort

#97, l=0, h=7
p=3

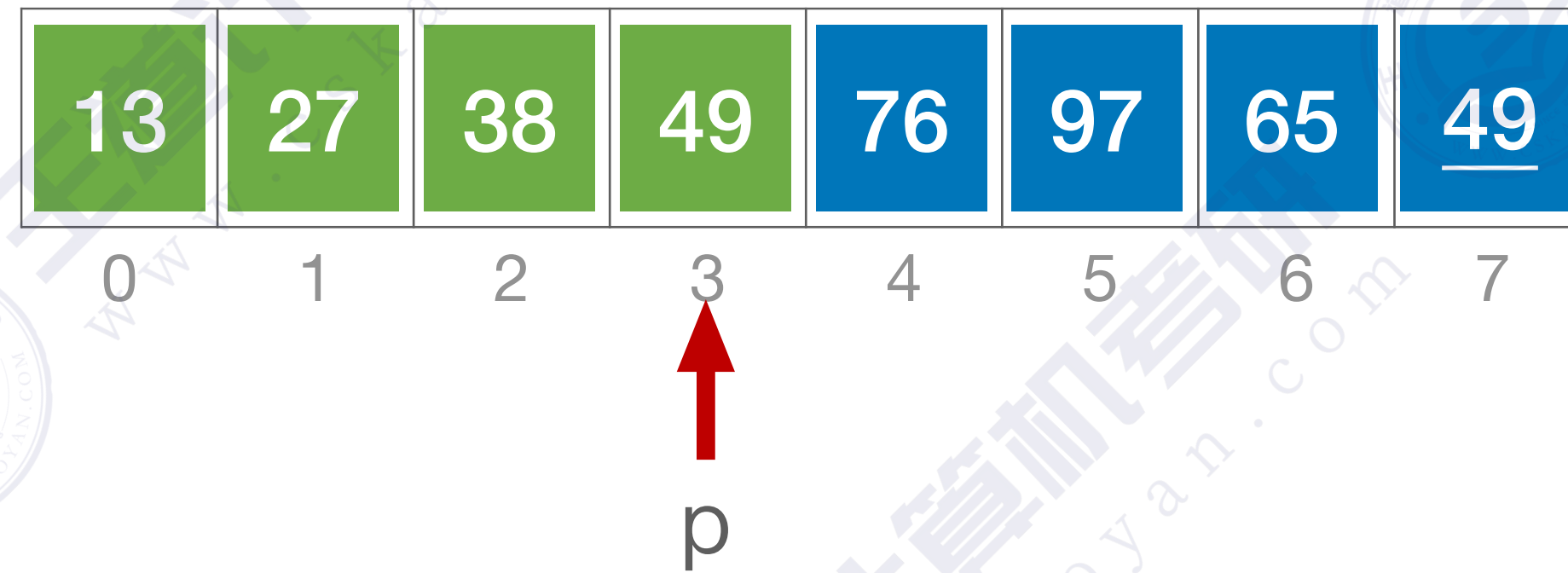
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第一层
QuickSort

#98, l=0, h=7
p=3

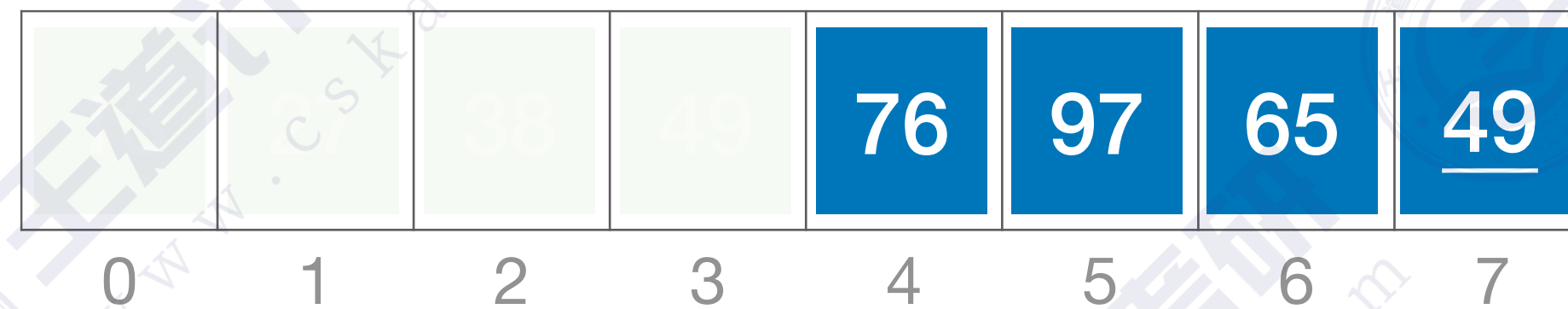
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];        //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第二层
QuickSort

$l=4, h=7$

#98, $l=0, h=7$
 $p=3$

递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```

快速排序



第二层
QuickSort

Partition 函数
(处理4~7)

#96, l=4, h=7

#98, l=0, h=7
p=3

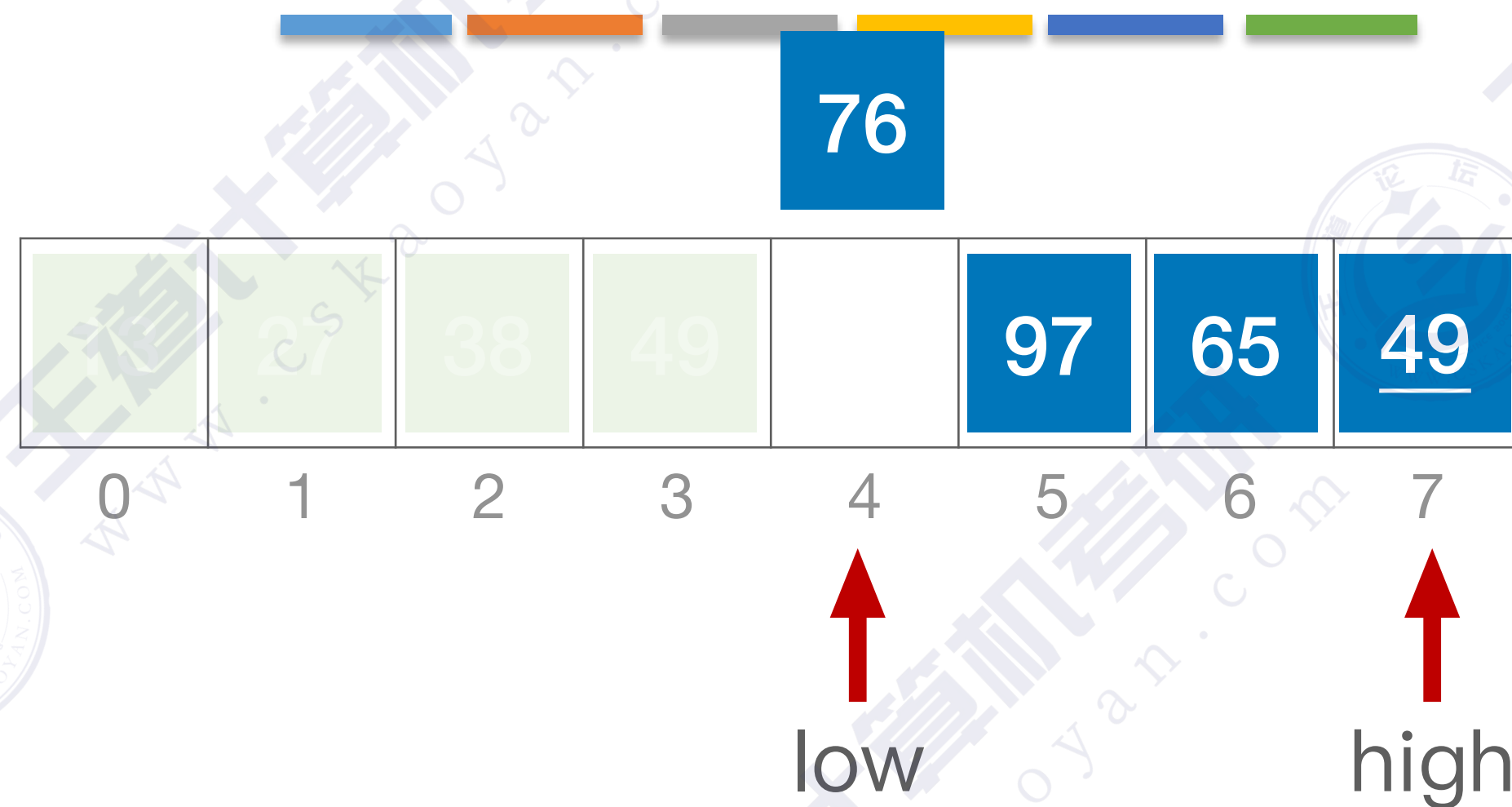
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         → int pivotpos=Partition(A,low,high); //划分
97             QuickSort(A,low,pivotpos-1); //划分左子表
98             QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第二层
QuickSort

Partition 函数
(处理4~7)

#96, l=4, h=7

#98, l=0, h=7
p=3

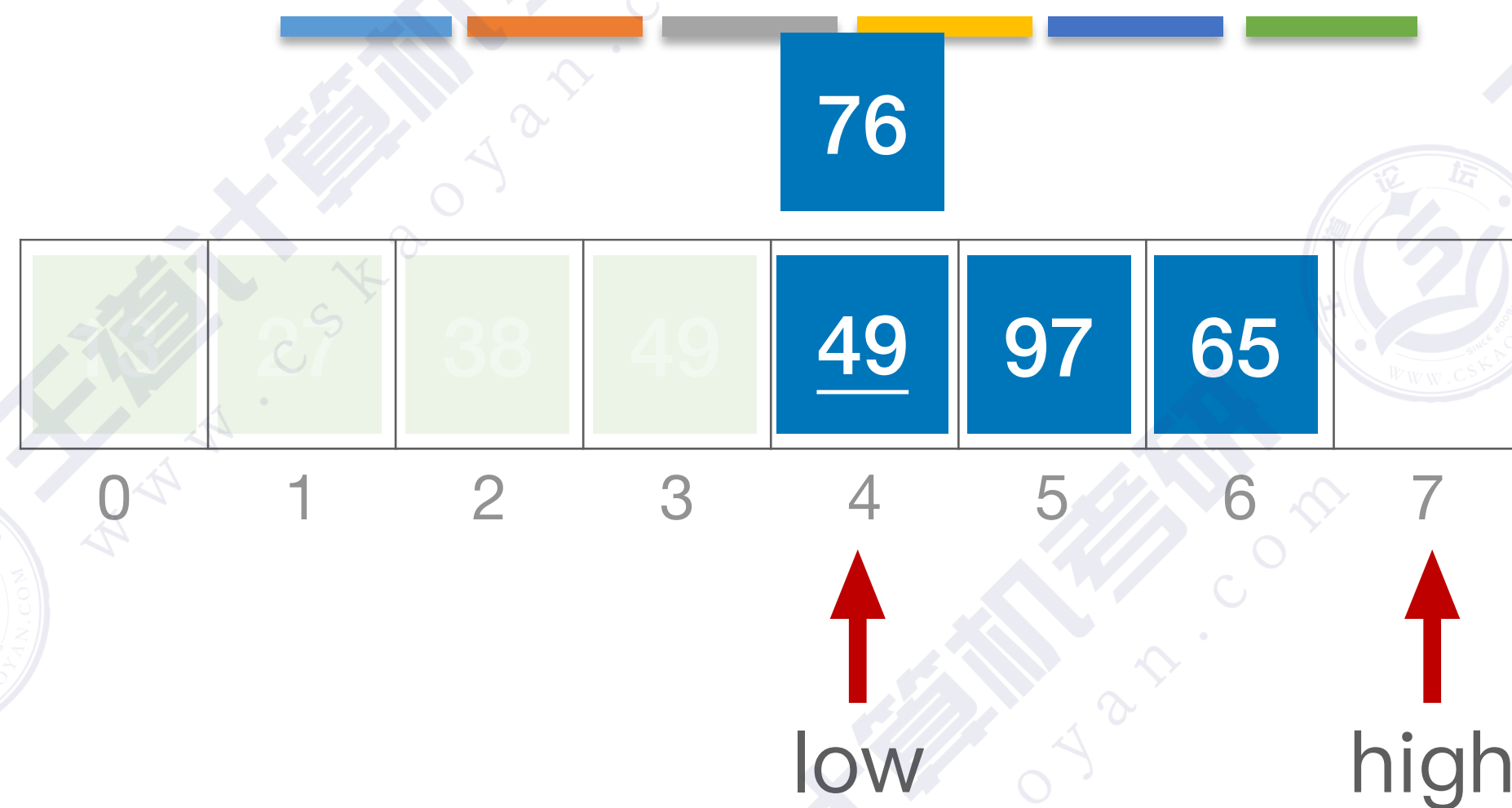
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){             //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];          //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];          //比枢轴大的元素移动到右端
    }
    A[low]=pivot;                //枢轴元素存放到最后位置
    return low;                  //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第二层
QuickSort

Partition 函数
(处理4~7)

#96, l=4, h=7

#98, l=0, h=7
p=3

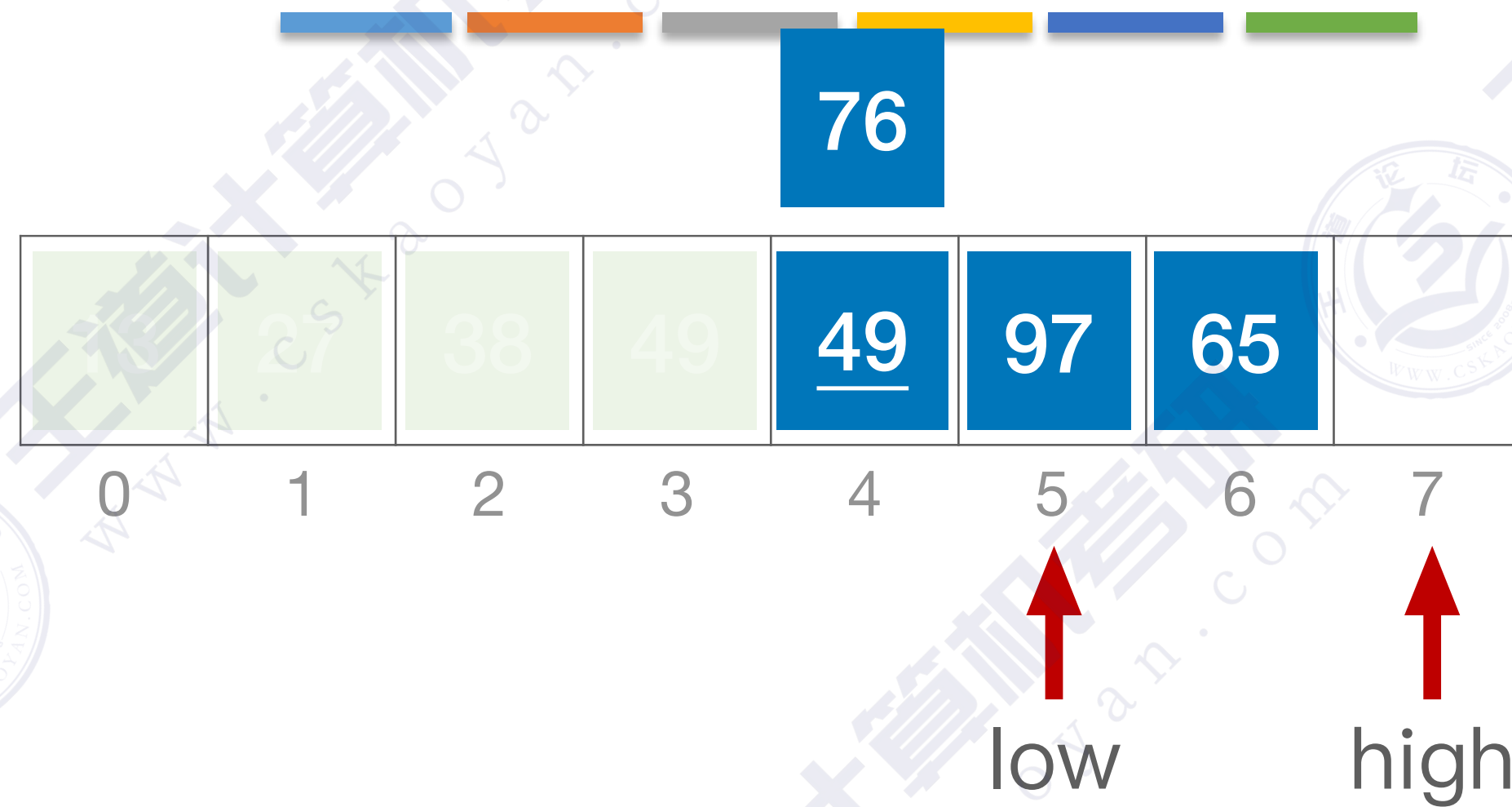
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){           //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){           //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1);         //划分左子表
98         QuickSort(A,pivotpos+1,high);         //划分右子表
99     }
100 }
```

快速排序



第二层
QuickSort

Partition 函数
(处理4~7)

#96, l=4, h=7

#98, l=0, h=7
p=3

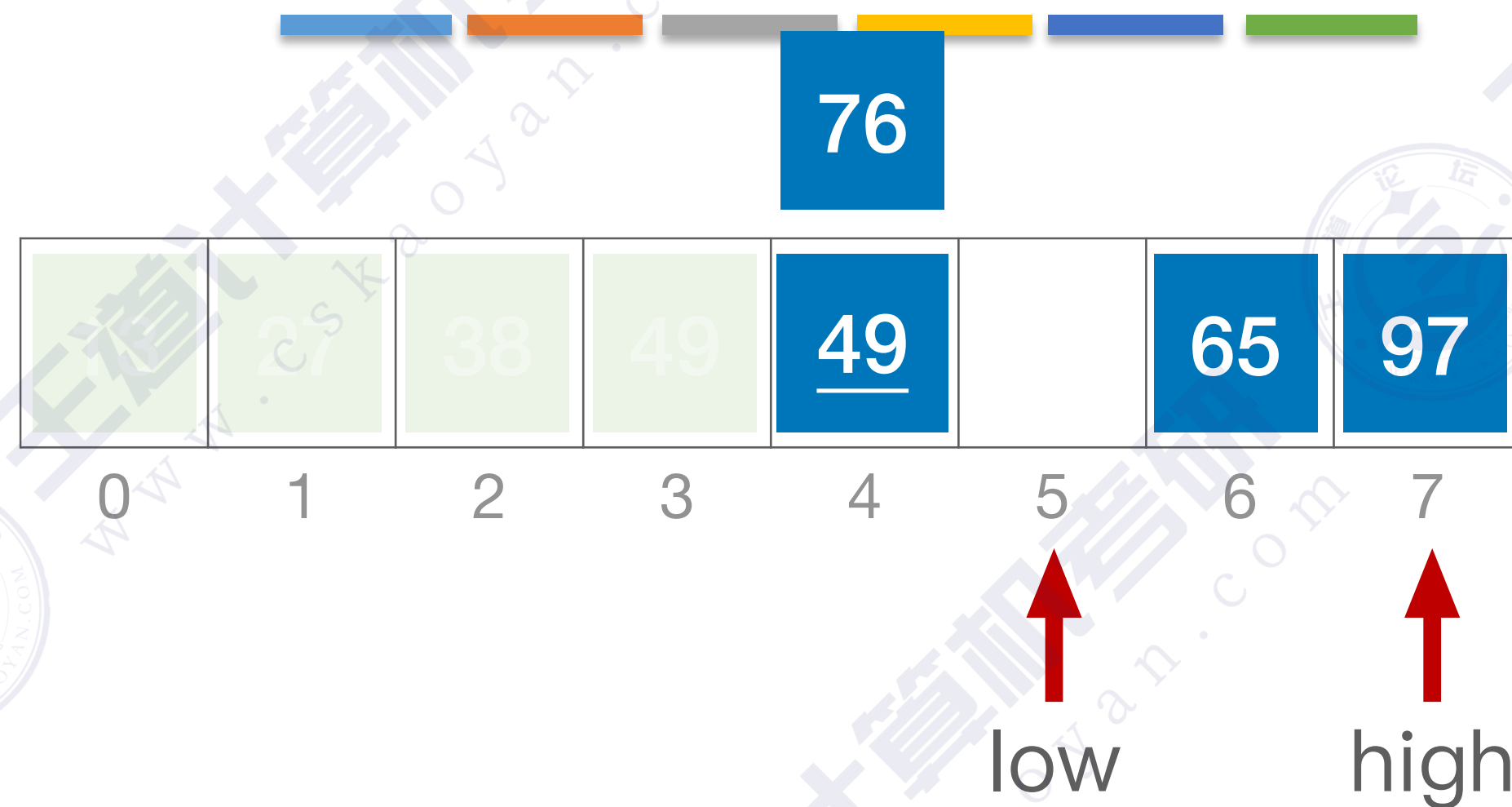
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){             //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];          //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];          //比枢轴大的元素移动到右端
    }
    A[low]=pivot;                //枢轴元素存放到最后位置
    return low;                  //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第二层
QuickSort

Partition 函数
(处理4~7)

#96, l=4, h=7

#98, l=0, h=7
p=3

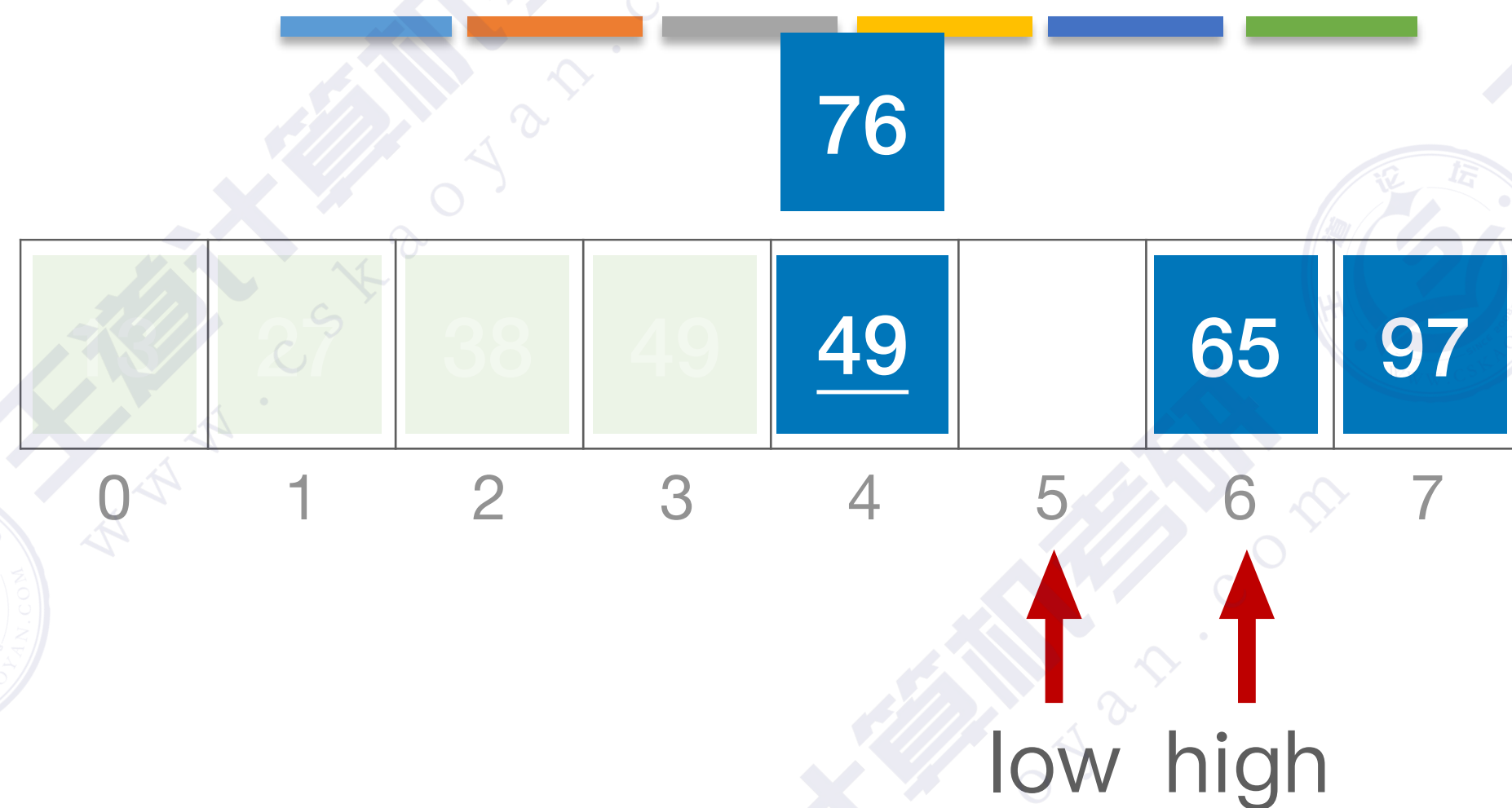
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];          //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];          //比枢轴大的元素移动到右端
    }
    A[low]=pivot;                //枢轴元素存放到最后位置
    return low;                  //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第二层
QuickSort

Partition 函数
(处理4~7)

#96, l=4, h=7

#98, l=0, h=7
p=3

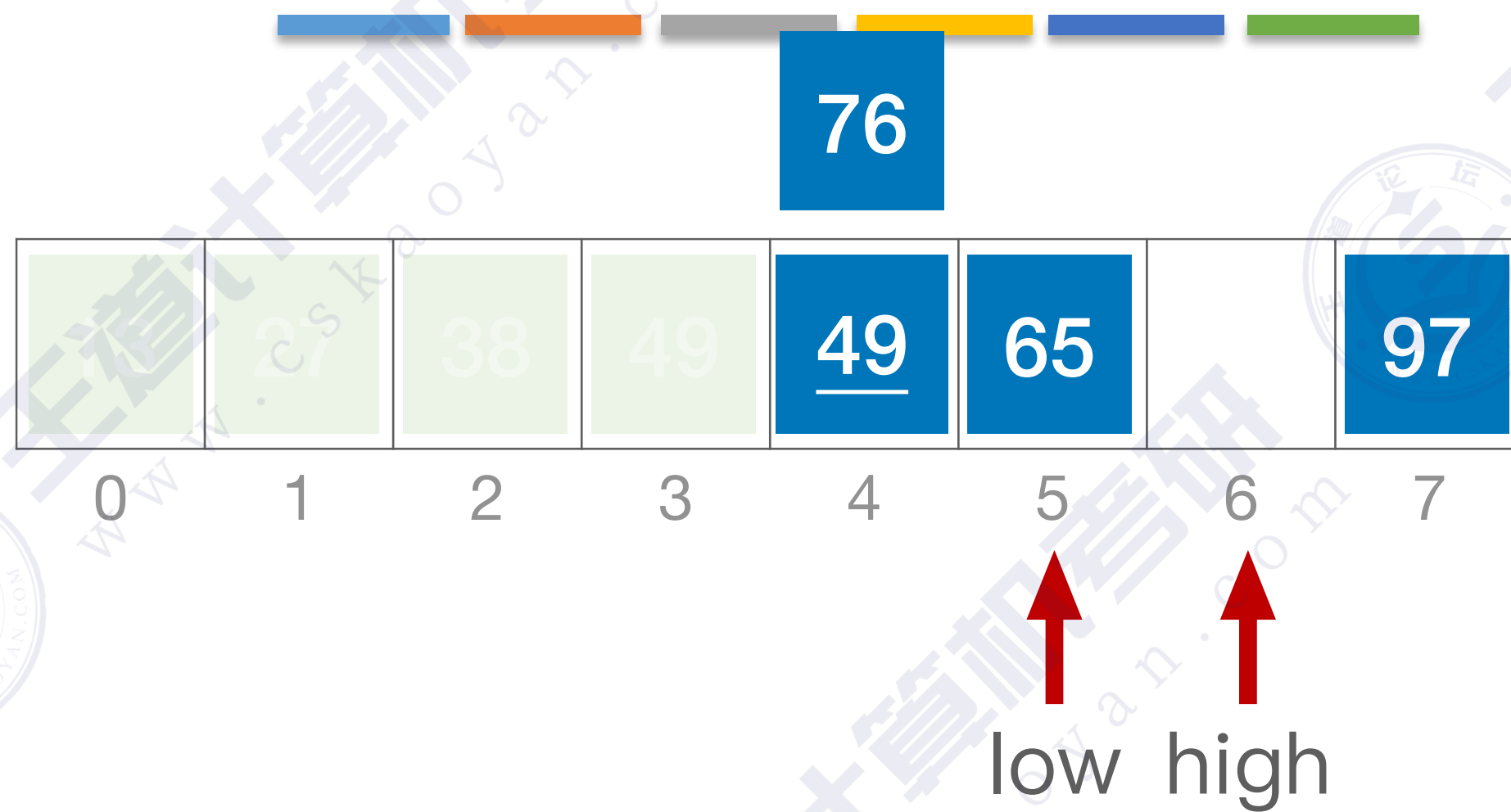
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```

快速排序



第二层 QuickSort

Partition 函数 (处理4~7)

#96, l=4, h=7

#98, l=0, h=7
p=3

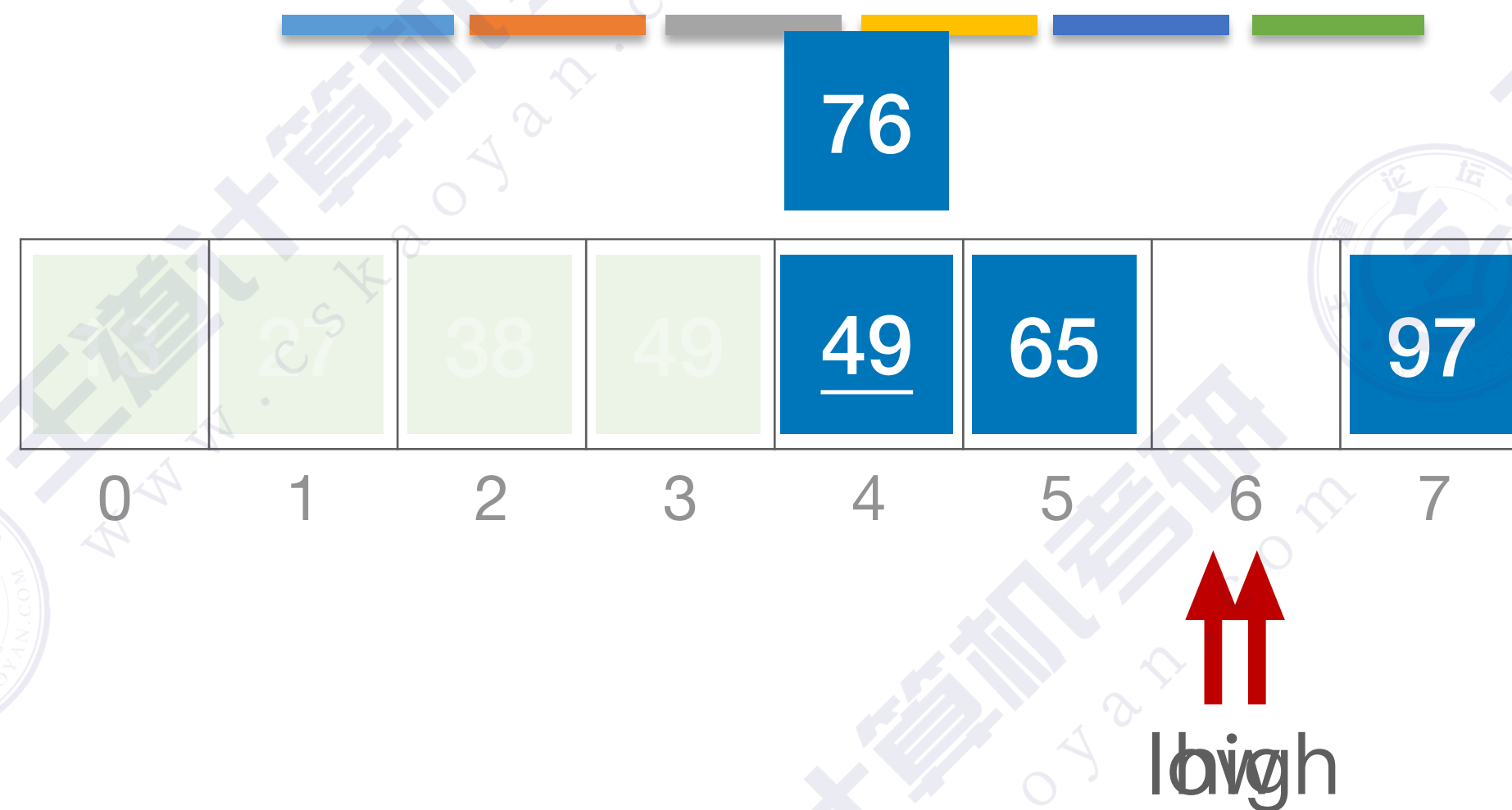
递归工作栈

```
//用第一个元素将待排序序列划分成左右两个部分
```

```
int Partition(int A[], int low, int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){             //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];          //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];          //比枢轴大的元素移动到右端
    }
    A[low]=pivot;                //枢轴元素存放到最终位置
    return low;                  //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         → int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第二层
QuickSort

Partition 函数
(处理4~7)

#96, l=4, h=7

#98, l=0, h=7
p=3

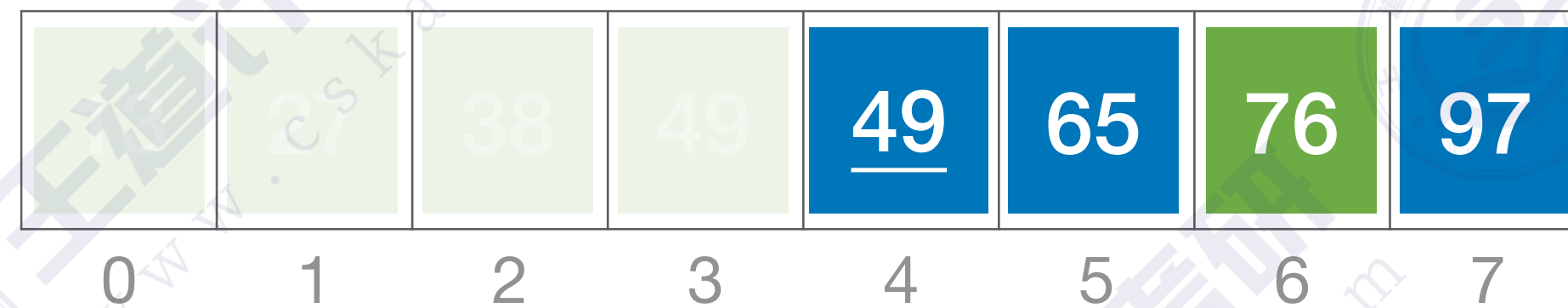
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



↑↑
low high

第二层
QuickSort

Partition 函数
(处理4~7)

#96, l=4, h=7

#98, l=0, h=7
p=3

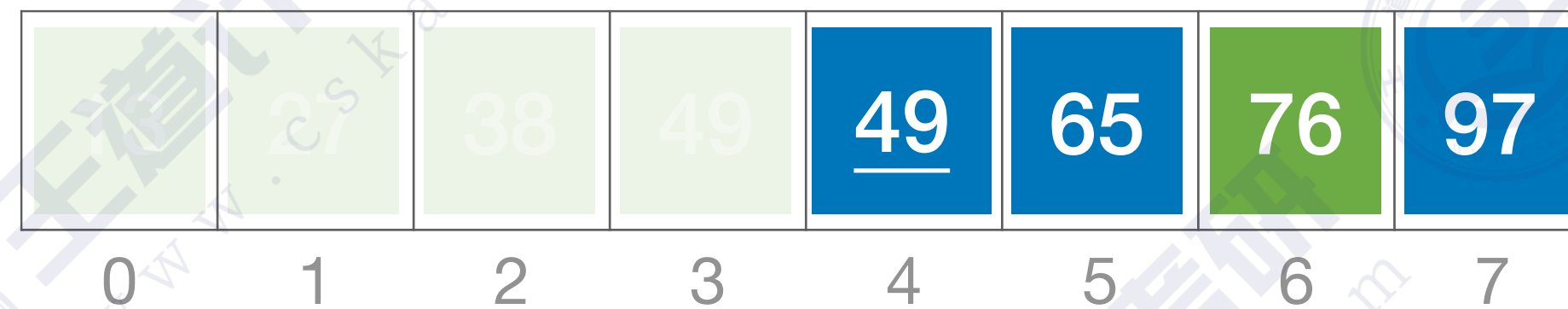
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```

快速排序



第二层
QuickSort

#96, l=4, h=7
p=6
#98, l=0, h=7
p=3

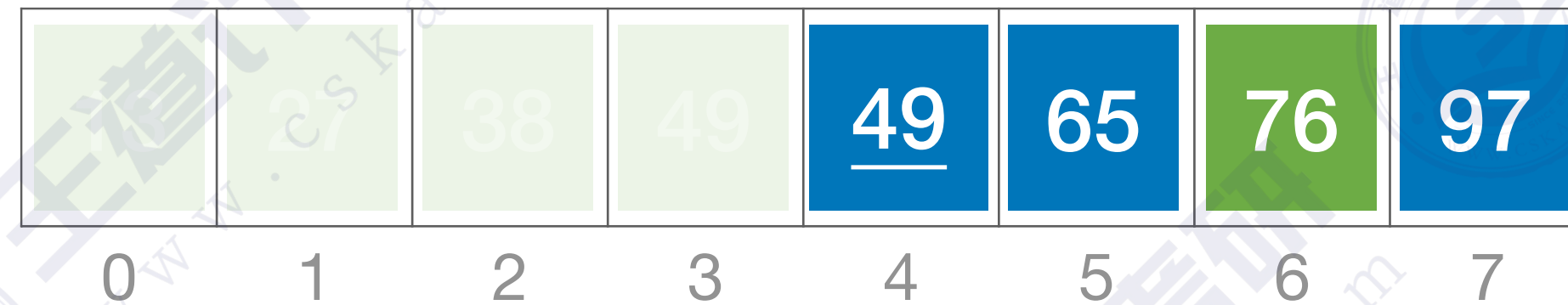
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第二层
QuickSort

#96, l=4, h=7 p=6
#98, l=0, h=7 p=3

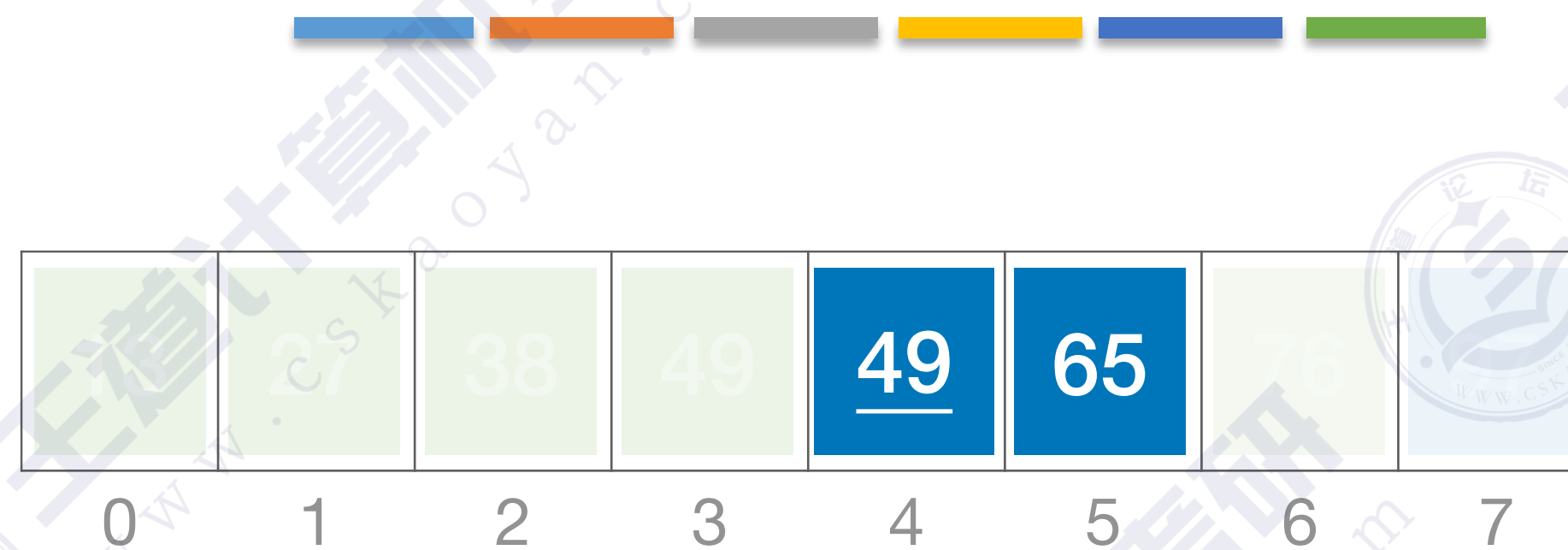
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第三层
QuickSort

$l=4, h=5$
#97, $l=4, h=7$ $p=6$
#98, $l=0, h=7$ $p=3$

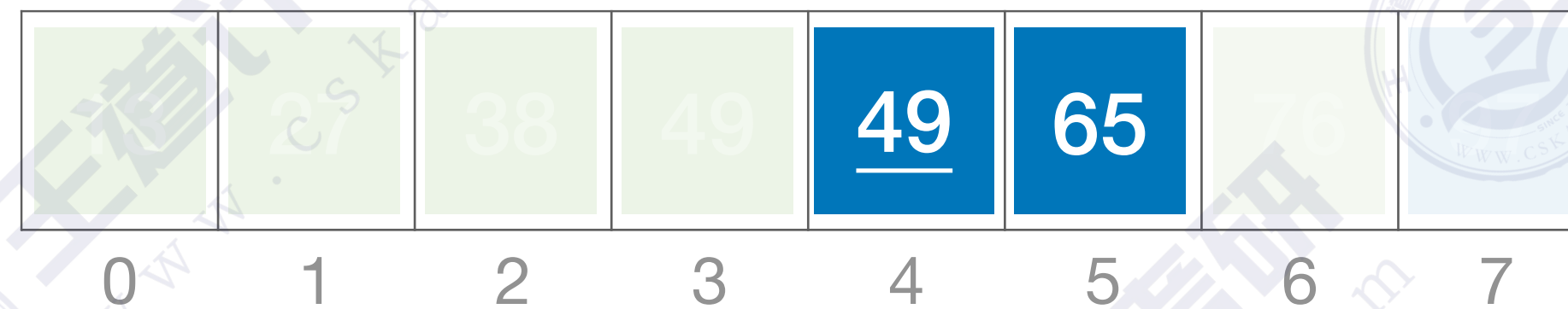
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){             //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];          //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];          //比枢轴大的元素移动到右端
    }
    A[low]=pivot;                //枢轴元素存放到最后位置
    return low;                  //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```

快速排序



第三层
QuickSort

Partition 函数
(处理4~5)

#96, l=4, h=5

#97, l=4, h=7
p=6

#98, l=0, h=7
p=3

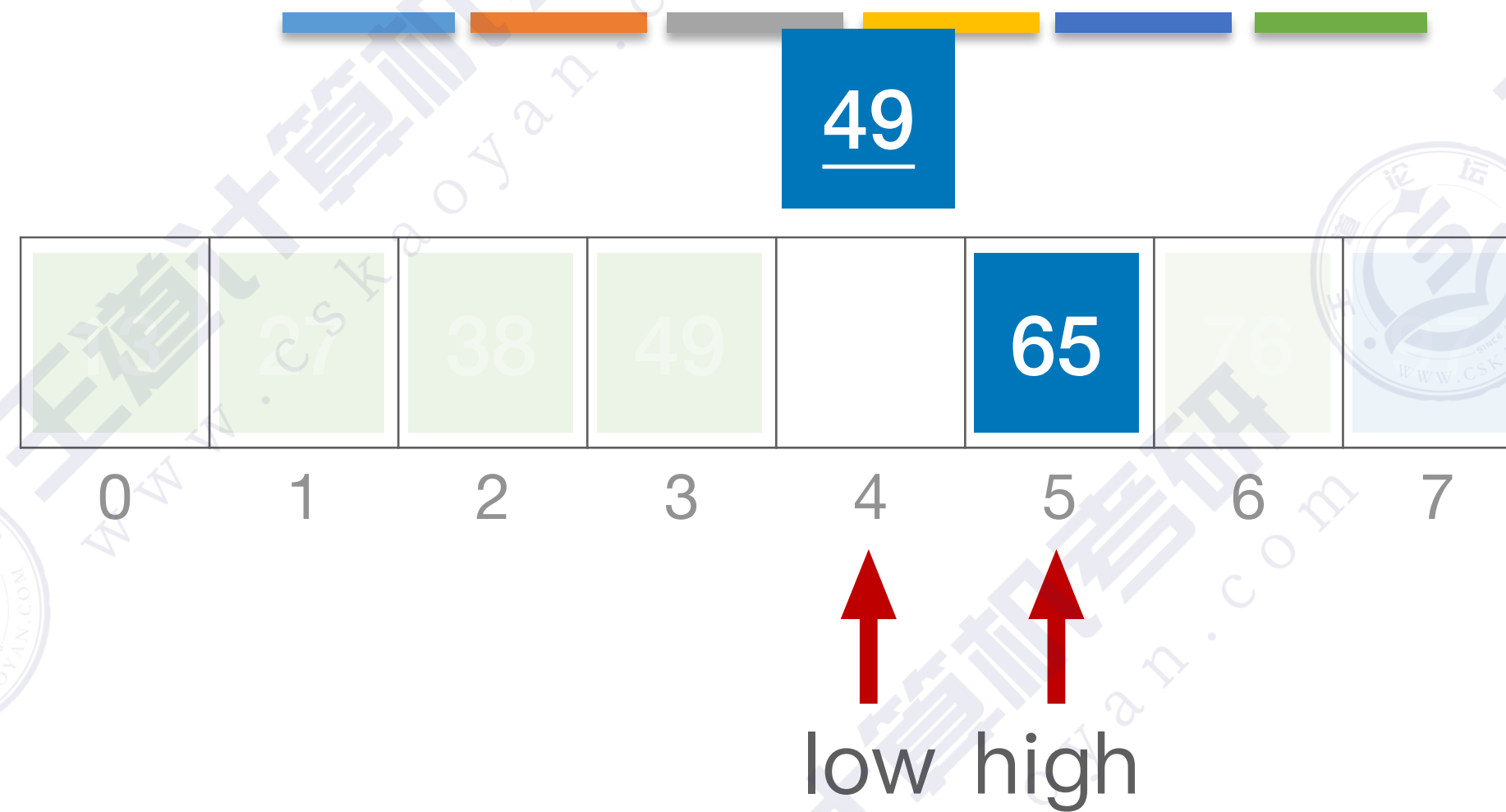
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){             //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];          //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];          //比枢轴大的元素移动到右端
    }
    A[low]=pivot;                //枢轴元素存放到最后位置
    return low;                  //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第三层
QuickSort

Partition 函数 (处理4~5)
#96, l=4, h=5
#97, l=4, h=7 p=6
#98, l=0, h=7 p=3

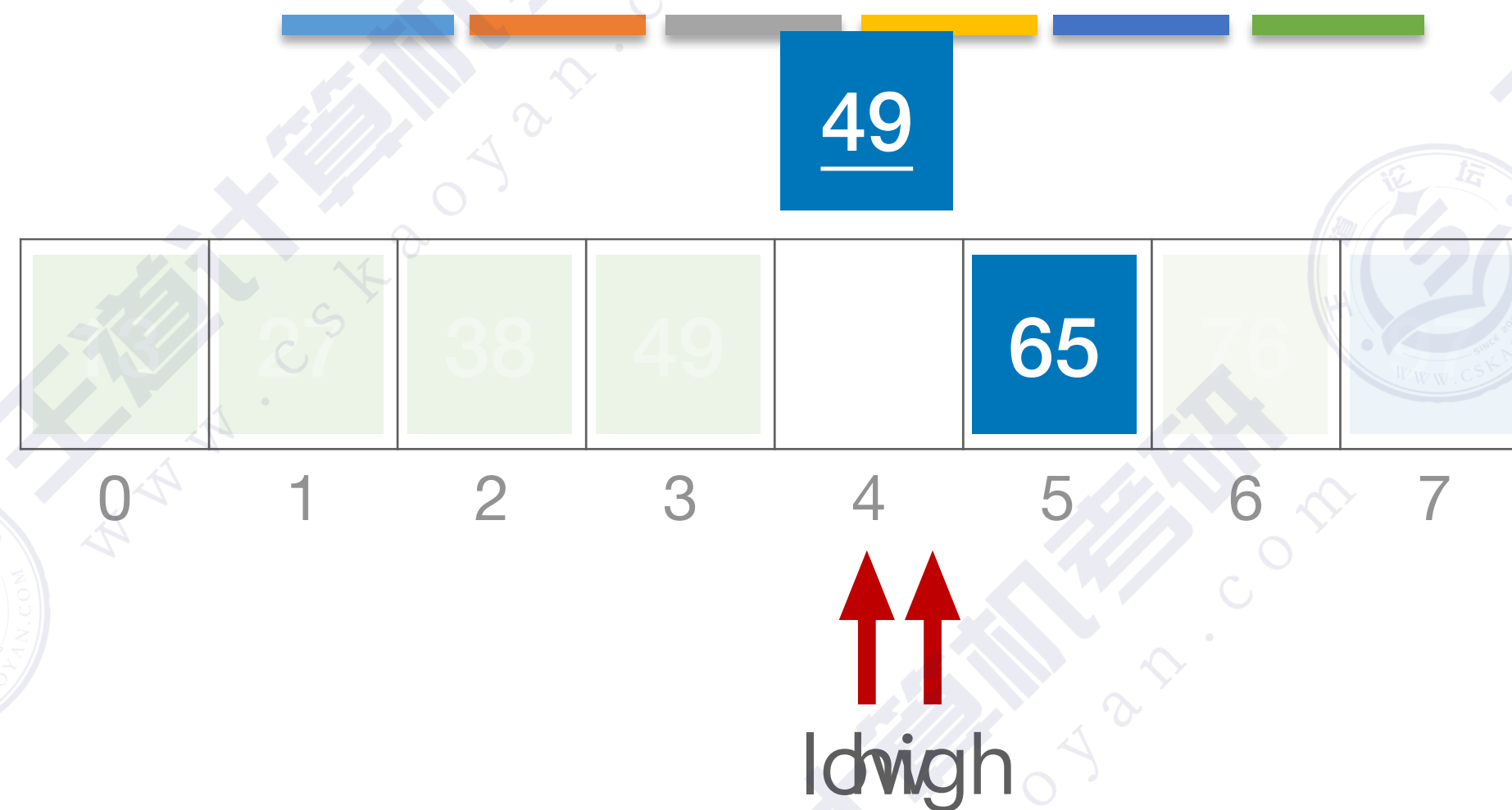
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



Partition 函数
(处理4~5)

#96, l=4, h=5

#97, l=4, h=7
p=6

#98, l=0, h=7
p=3

递归工作栈

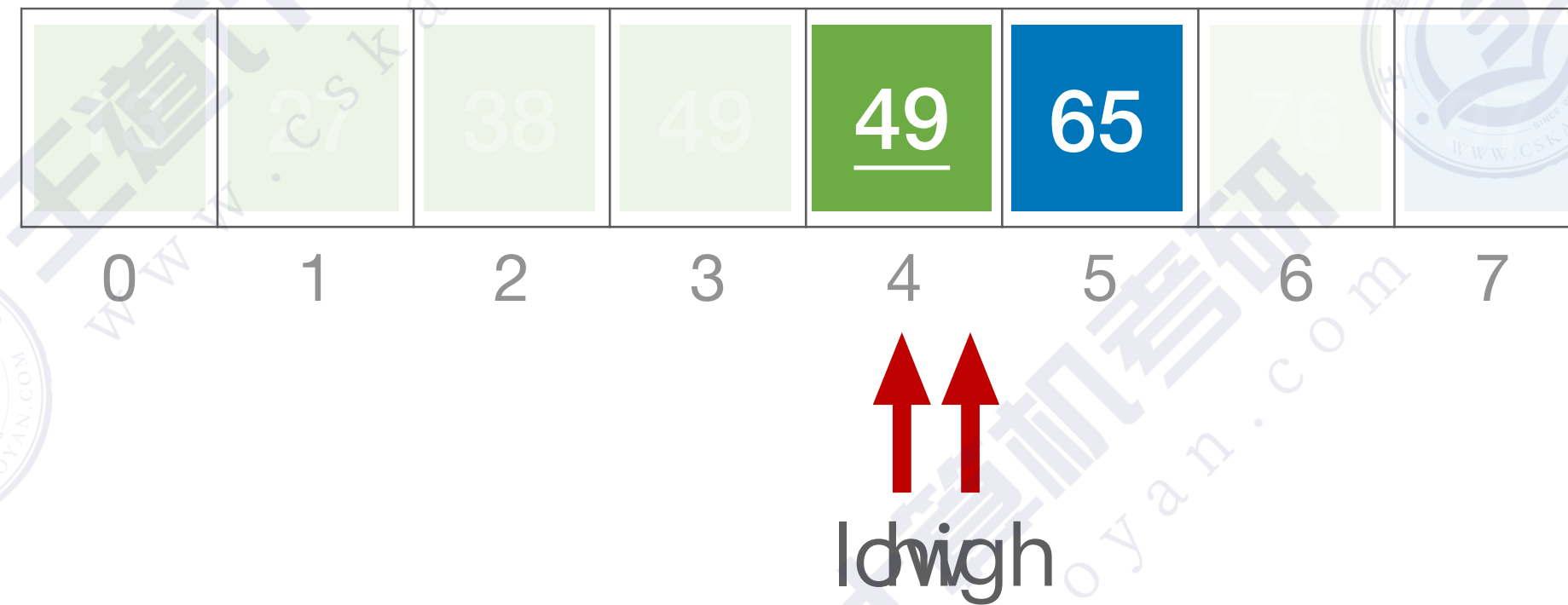
//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

93 //快速排序

```
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```

快速排序



第三层
QuickSort

Partition 函数 (处理4~5)
#96, l=4, h=5
#97, l=4, h=7 p=6
#98, l=0, h=7 p=3

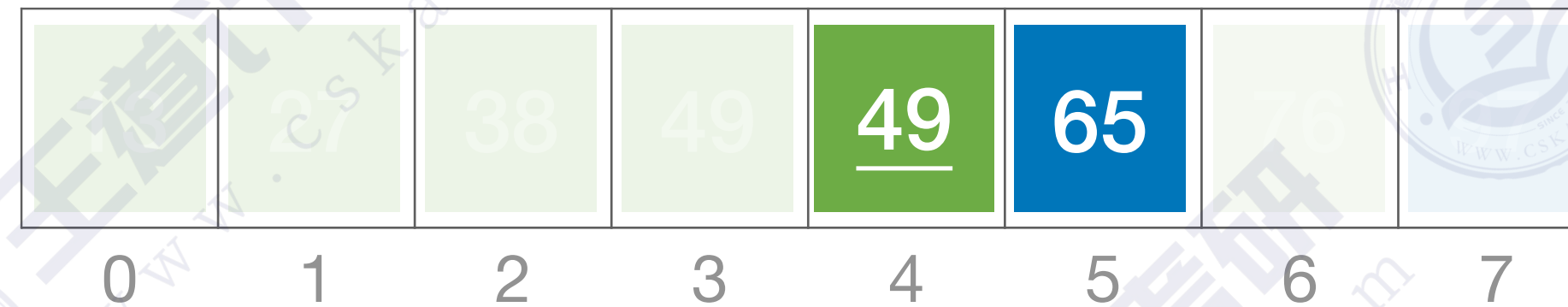
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第三层
QuickSort

#96, l=4, h=5 p=4
#97, l=4, h=7 p=6
#98, l=0, h=7 p=3

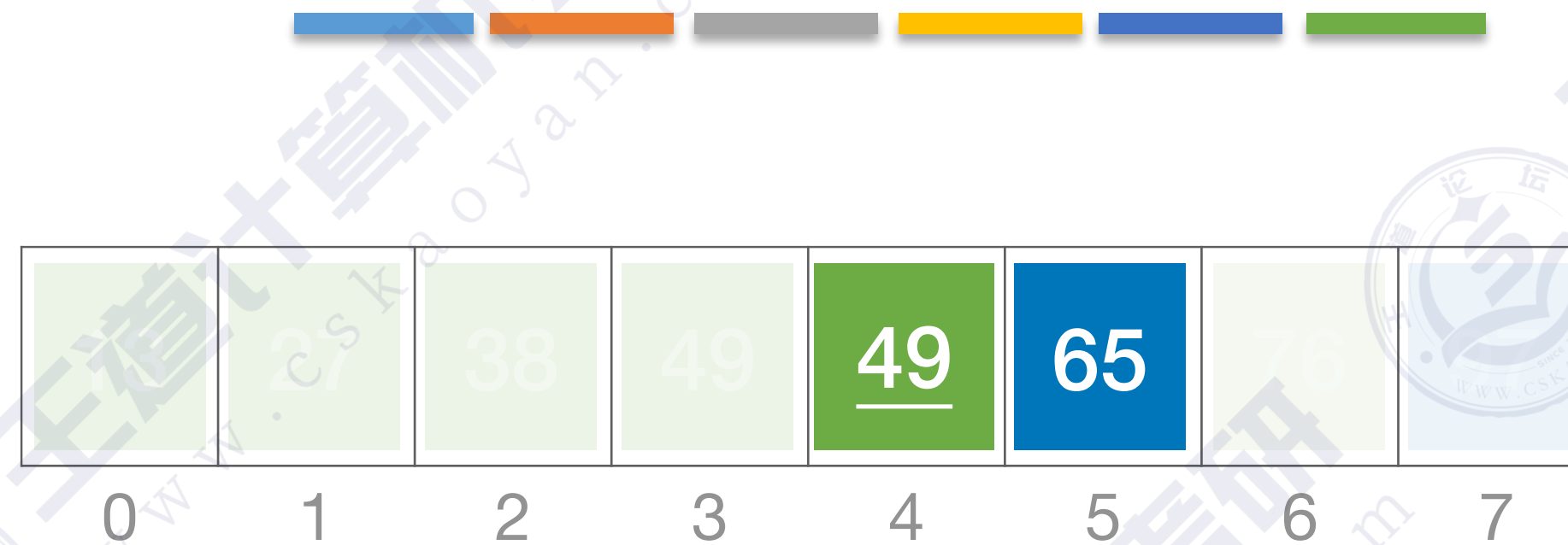
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第三层
QuickSort

#96, l=4, h=5 p=4
#97, l=4, h=7 p=6
#98, l=0, h=7 p=3

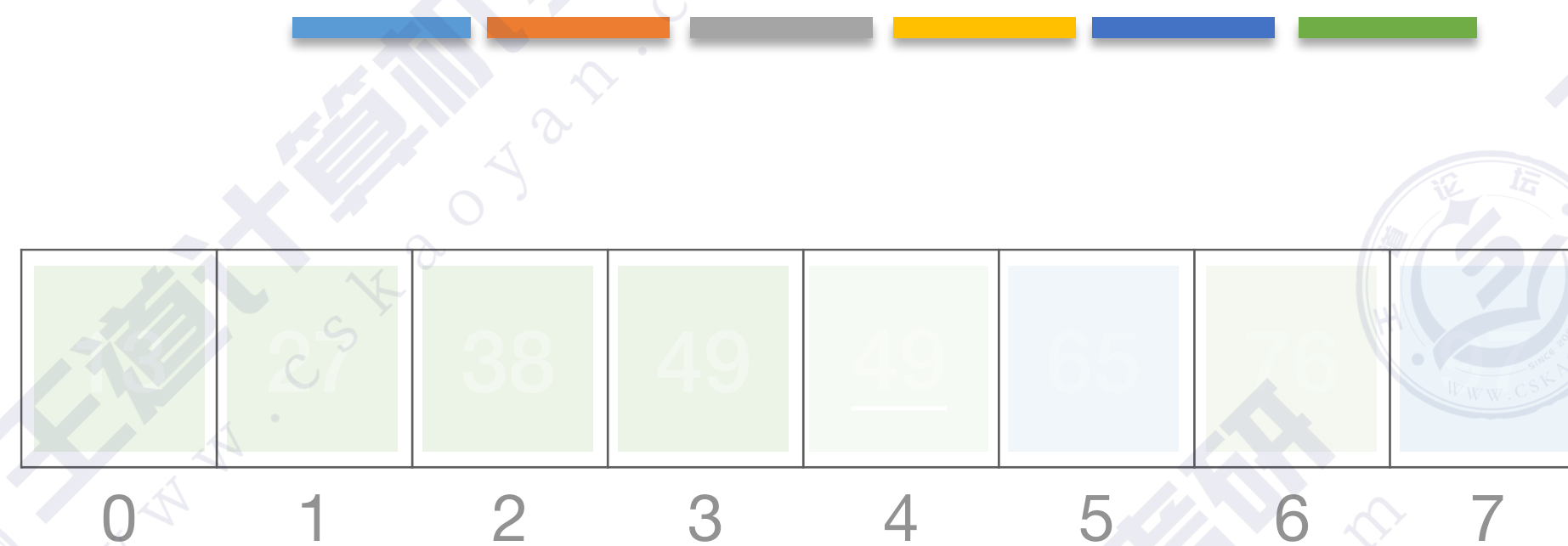
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```

快速排序



第四层
QuickSort

$l=4, h=3$
#97, $l=4, h=5$ $p=4$
#97, $l=4, h=7$ $p=6$
#98, $l=0, h=7$ $p=3$

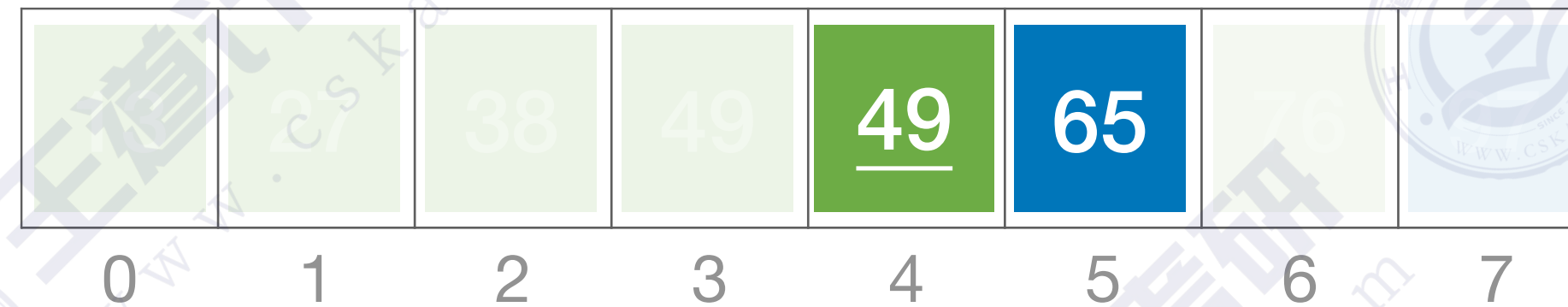
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){             //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];          //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];          //比枢轴大的元素移动到右端
    }
    A[low]=pivot;                //枢轴元素存放到最后位置
    return low;                  //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第三层
QuickSort

#97, l=4, h=5 p=4
#97, l=4, h=7 p=6
#98, l=0, h=7 p=3

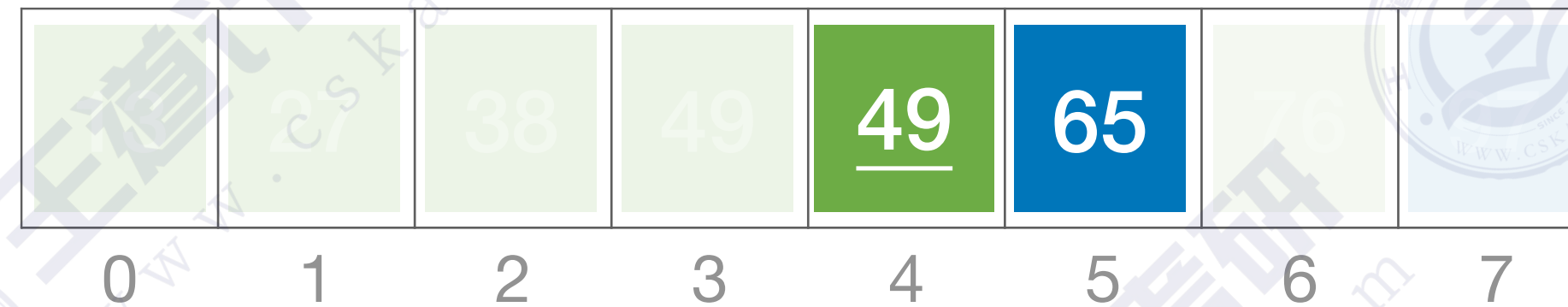
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];          //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];          //比枢轴大的元素移动到右端
    }
    A[low]=pivot;                //枢轴元素存放到最后位置
    return low;                  //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第三层
QuickSort

#97, l=4, h=5 p=4
#97, l=4, h=7 p=6
#98, l=0, h=7 p=3

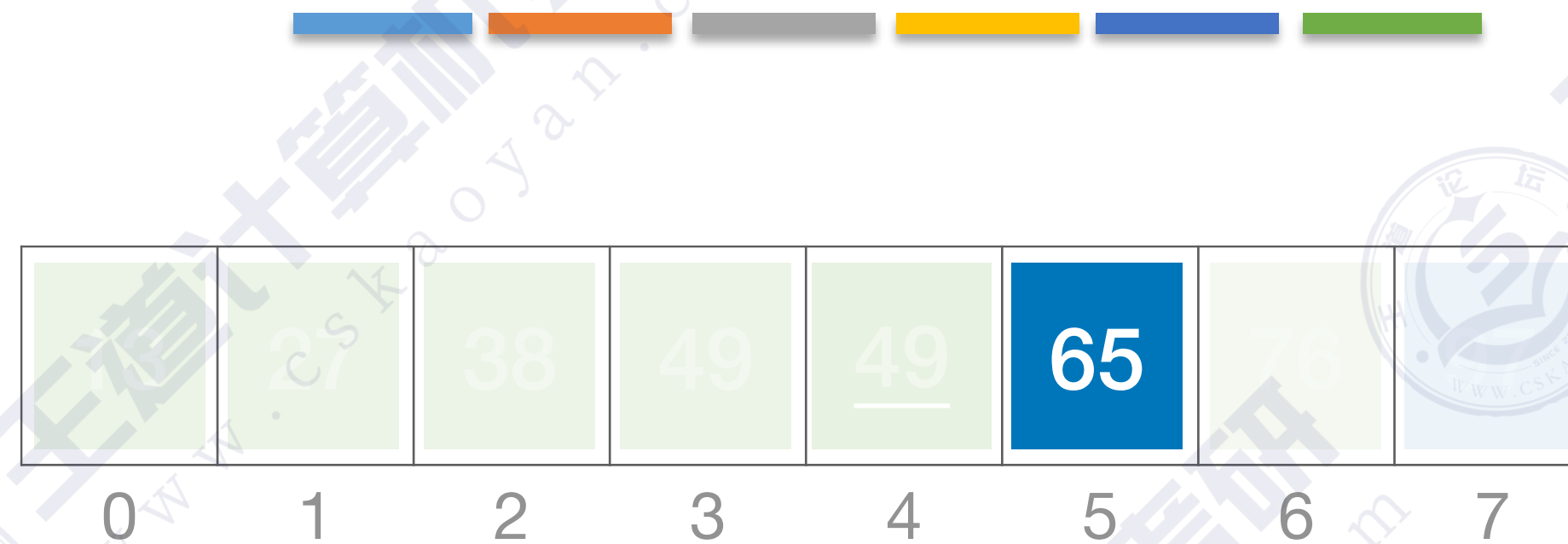
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```

快速排序



第四层
QuickSort

$l=5, h=5$
#98, $l=4, h=5$ $p=4$
#97, $l=4, h=7$ $p=6$
#98, $l=0, h=7$ $p=3$

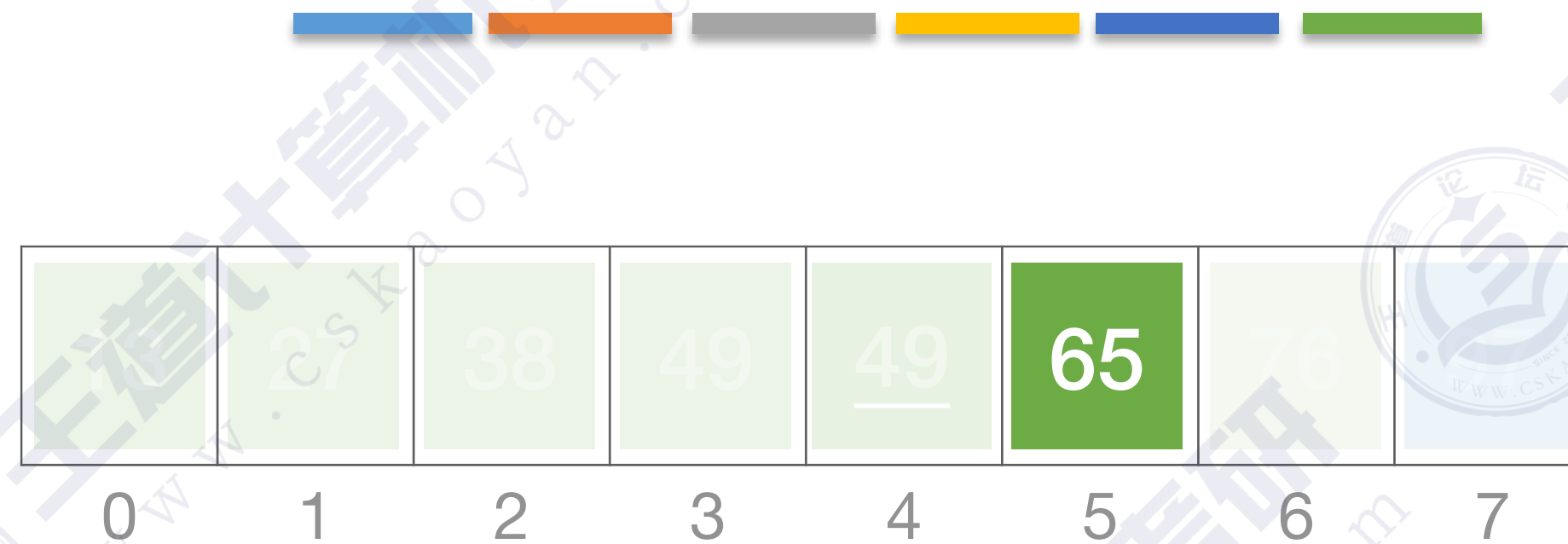
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){             //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];          //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];          //比枢轴大的元素移动到右端
    }
    A[low]=pivot;                //枢轴元素存放到最后位置
    return low;                  //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第四层
QuickSort

$l=5, h=5$
#98, $l=4, h=5$ $p=4$
#97, $l=4, h=7$ $p=6$
#98, $l=0, h=7$ $p=3$

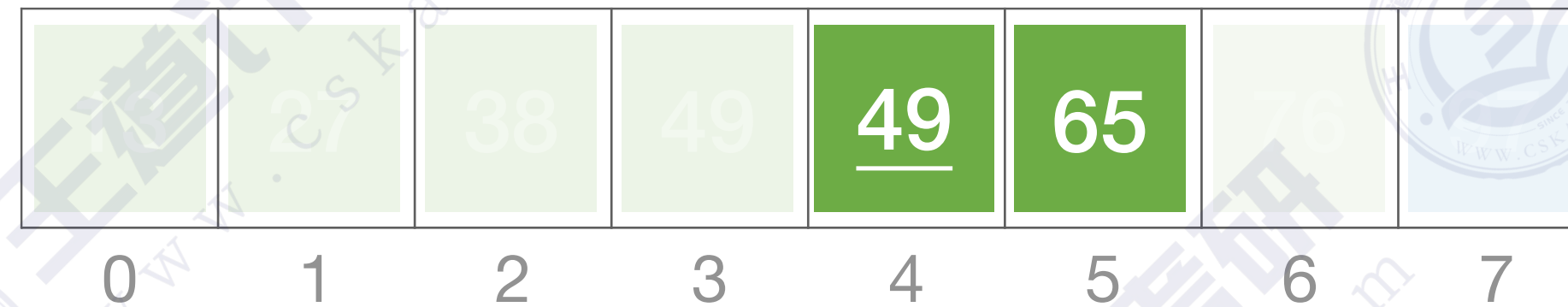
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第三层
QuickSort

#98, l=4, h=5 p=4
#97, l=4, h=7 p=6
#98, l=0, h=7 p=3

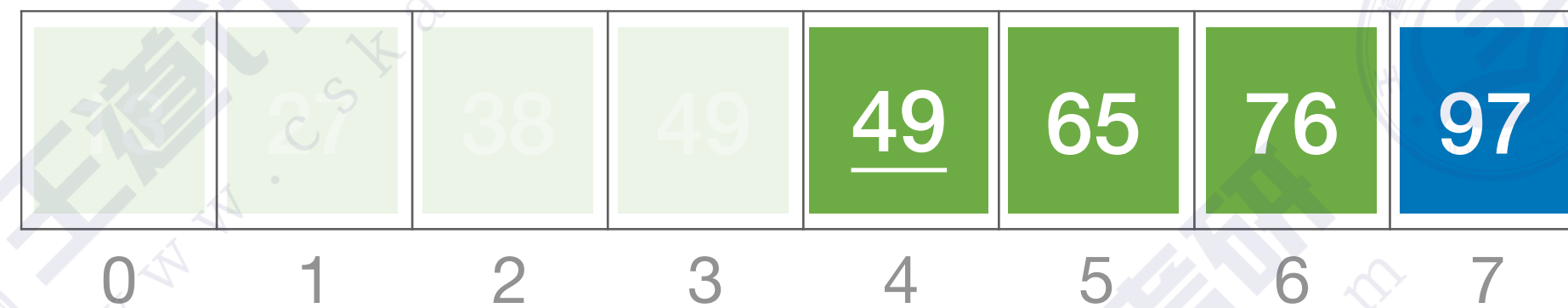
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){             //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];          //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];          //比枢轴大的元素移动到右端
    }
    A[low]=pivot;                //枢轴元素存放到最后位置
    return low;                  //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```

快速排序



第二层
QuickSort

#97, l=4, h=7 p=6
#98, l=0, h=7 p=3

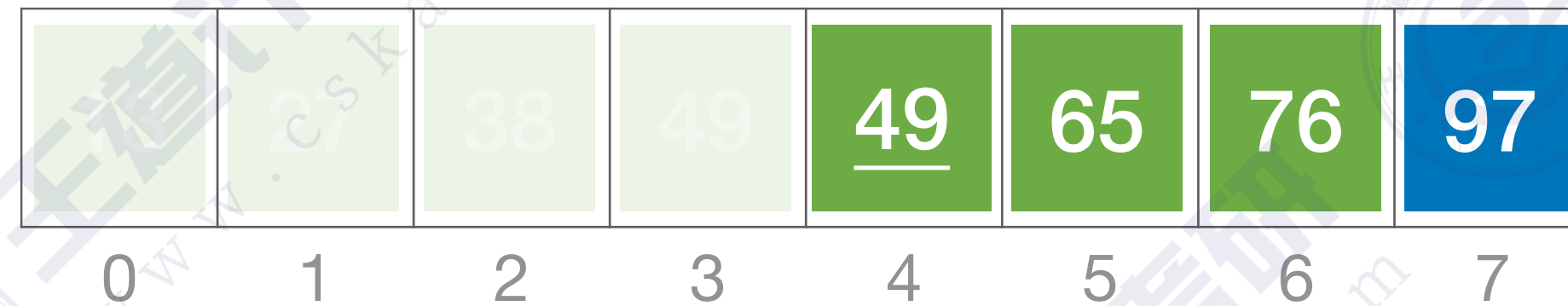
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];          //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];          //比枢轴大的元素移动到右端
    }
    A[low]=pivot;                //枢轴元素存放到最后位置
    return low;                  //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第二层
QuickSort

#97, l=4, h=7 p=6
#98, l=0, h=7 p=3

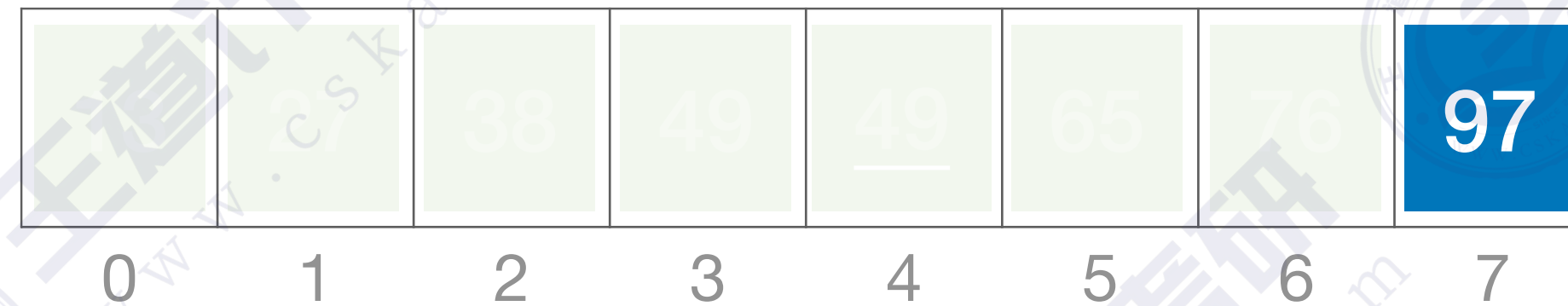
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



第三层
QuickSort

$l=7, h=7$
#98, $l=4, h=7$ $p=6$
#98, $l=0, h=7$ $p=3$

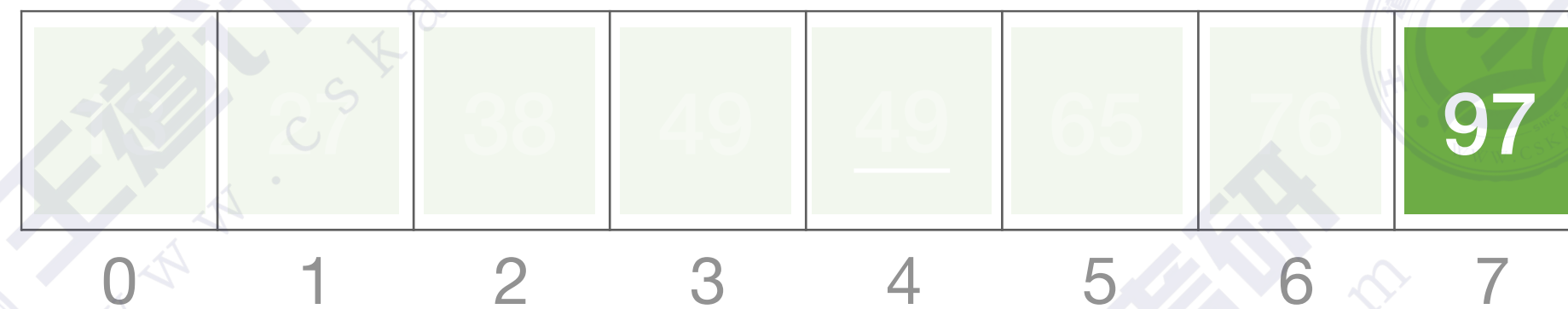
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){             //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];          //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];          //比枢轴大的元素移动到右端
    }
    A[low]=pivot;                //枢轴元素存放到最后位置
    return low;                  //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```

快速排序



第三层
QuickSort

$l=7, h=7$

#98, $l=4, h=7$
 $p=6$

#98, $l=0, h=7$
 $p=3$

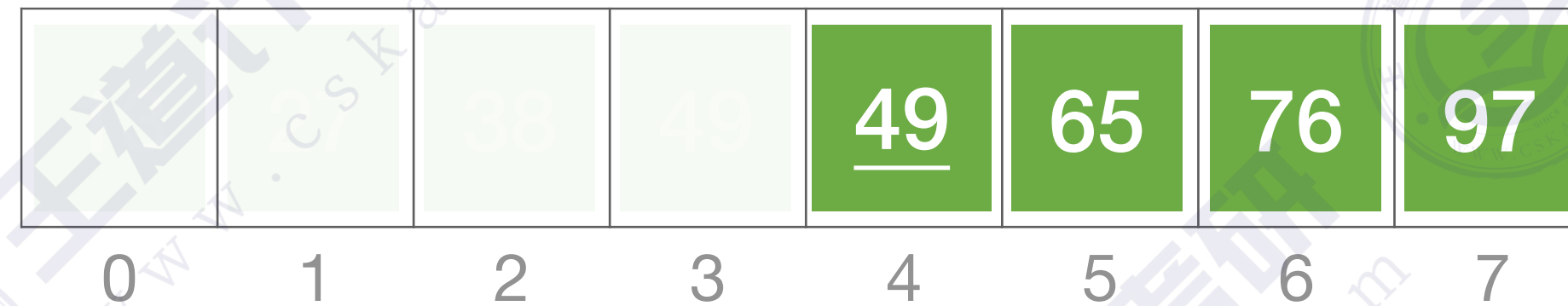
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



#98, l=4, h=7 p=6
#98, l=0, h=7 p=3

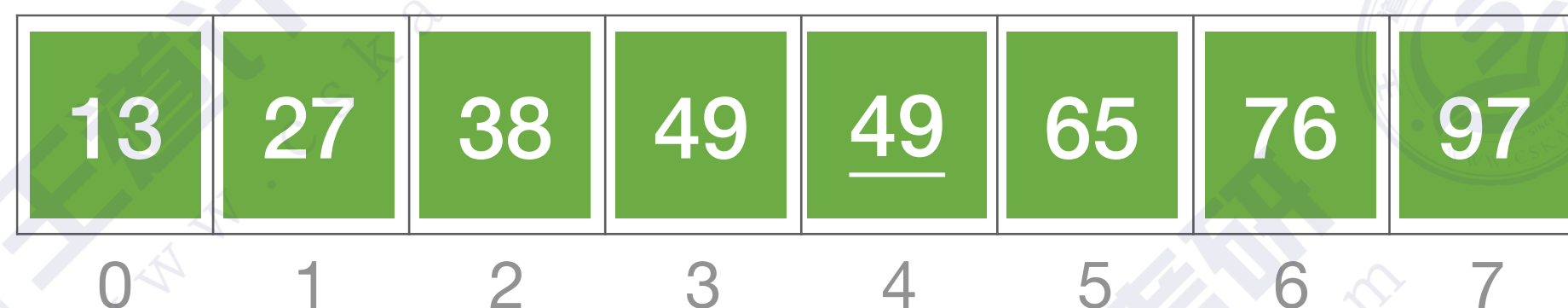
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];          //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];          //比枢轴大的元素移动到右端
    }
    A[low]=pivot;                //枢轴元素存放到最后位置
    return low;                  //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```


快速排序



#98, l=0, h=7
p=3

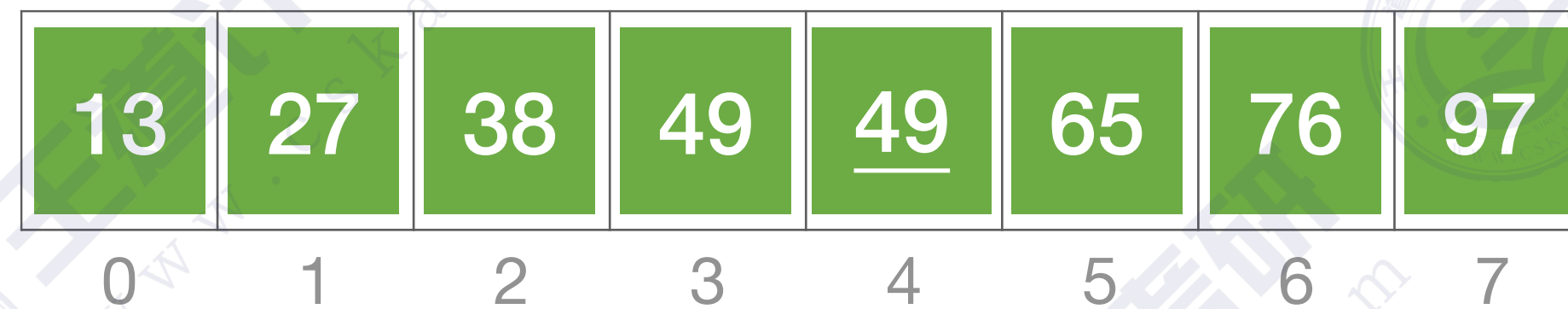
递归工作栈

//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```

快速排序



递归工作栈

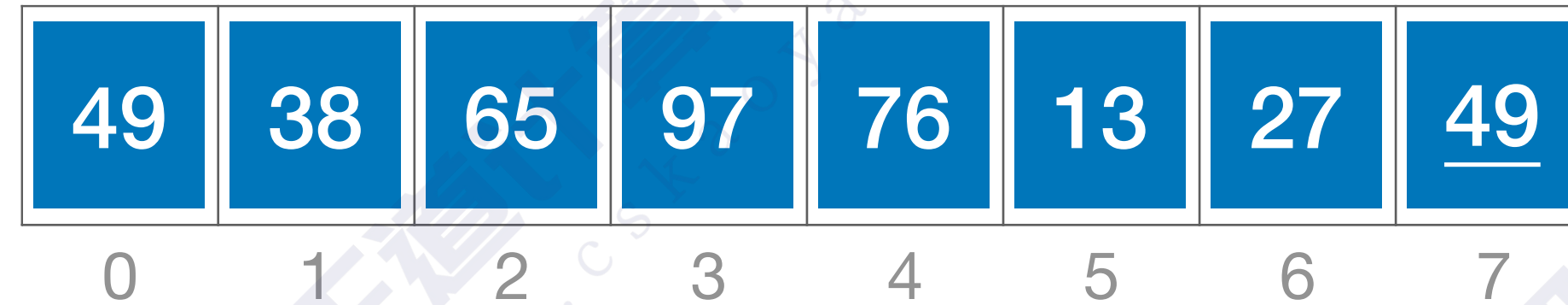
//用第一个元素将待排序序列划分成左右两个部分

```
int Partition(int A[],int low,int high){
    int pivot=A[low];           //第一个元素作为枢轴
    while(low<high){            //用low、high搜索枢轴的最终位置
        while(low<high&&A[high]>=pivot) --high;
        A[low]=A[high];         //比枢轴小的元素移动到左端
        while(low<high&&A[low]<=pivot) ++low;
        A[high]=A[low];         //比枢轴大的元素移动到右端
    }
    A[low]=pivot;               //枢轴元素存放到最后位置
    return low;                 //返回存放枢轴的最终位置
}
```

```
93 //快速排序
94 void QuickSort(int A[],int low,int high){
95     if(low<high){ //递归跳出的条件
96         int pivotpos=Partition(A,low,high); //划分
97         QuickSort(A,low,pivotpos-1); //划分左子表
98         QuickSort(A,pivotpos+1,high); //划分右子表
99     }
100 }
```

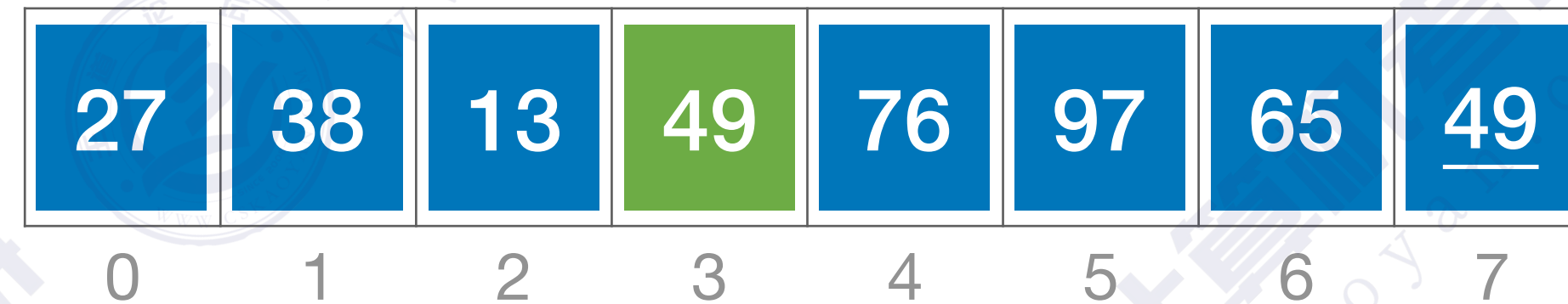

算法效率分析

初始序列



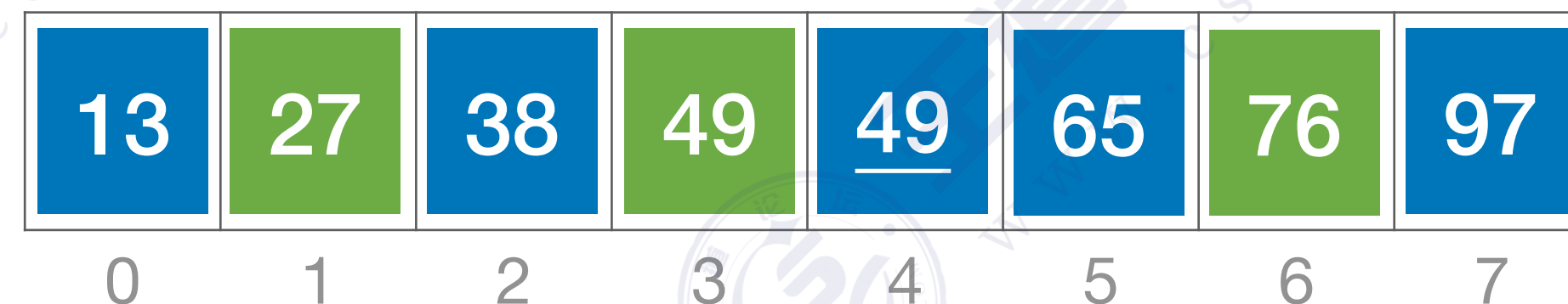
(待排序元素: 0~7)

第一层QuickSort处理后:



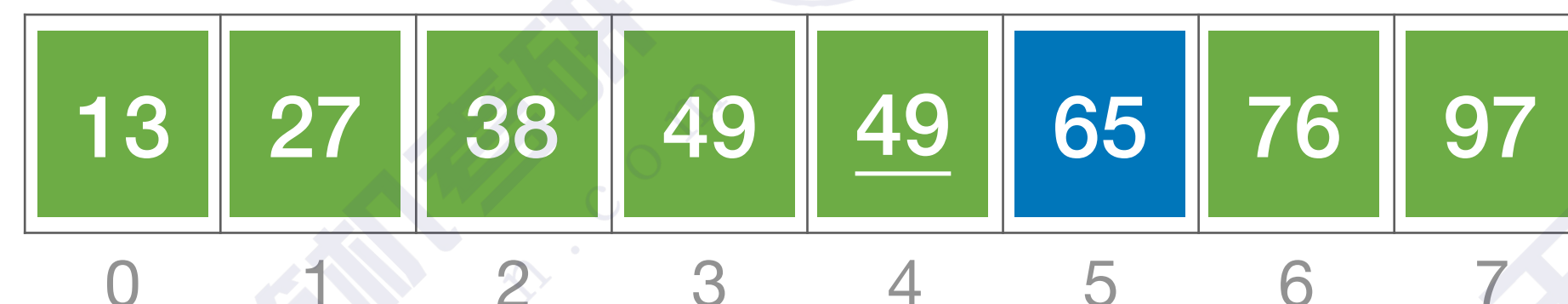
(待排序元素: 0~2, 4~7)

第二层QuickSort处理后:



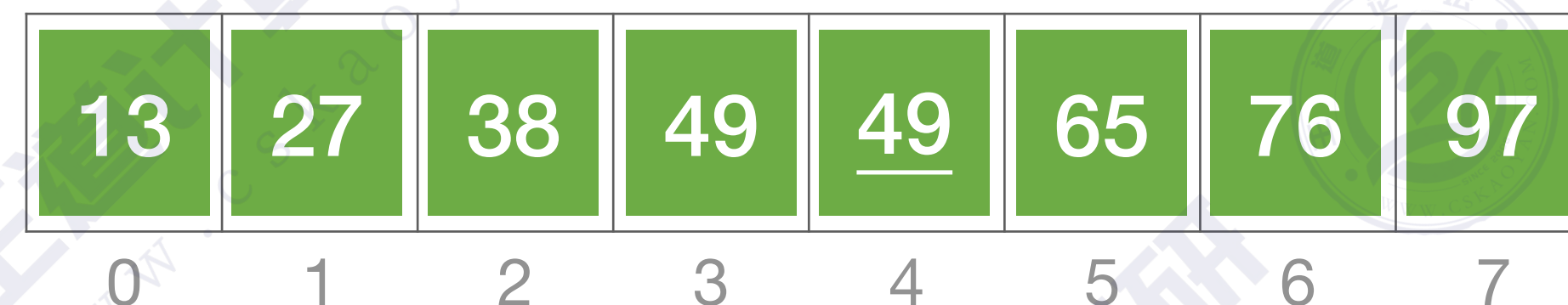
(待排序元素: 0~0, 2~2, 4~5, 7~7)

第三层QuickSort处理后:



(待排序元素: 5~5)

第四层QuickSort处理后:

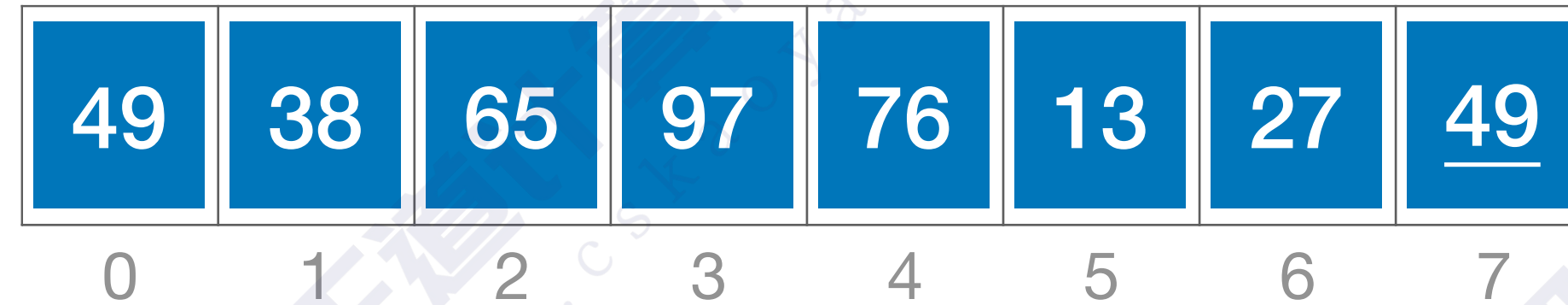


每一层的QuickSort 只需要处理剩余的待排序元素, 时间复杂度不超过 $O(n)$

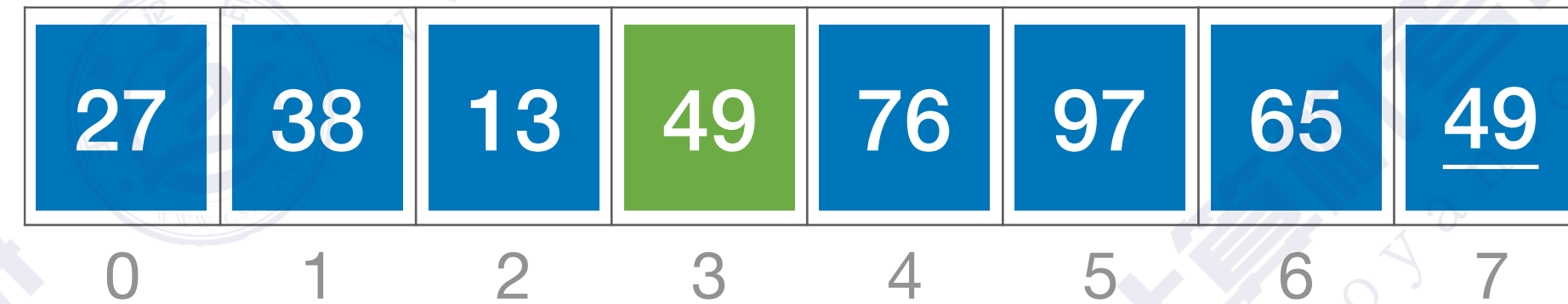
时间复杂度= $O(n \times \text{递归层数})$

算法效率分析

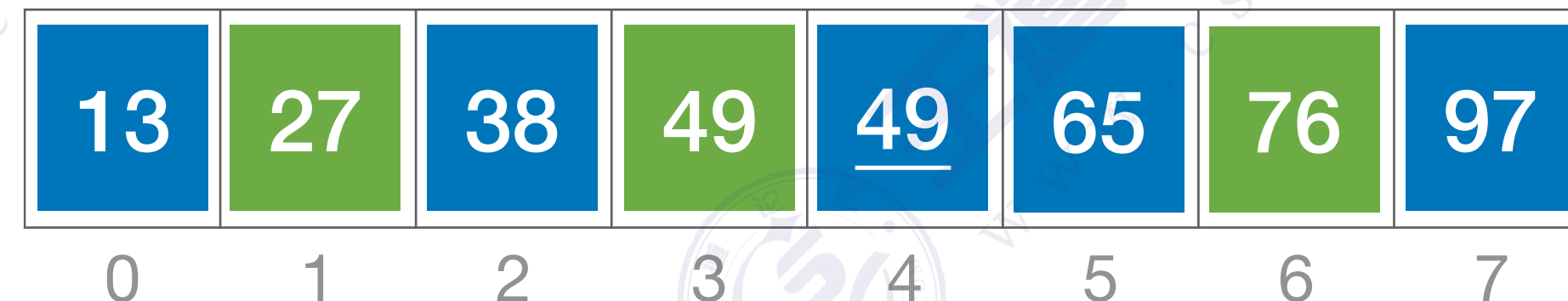
初始序列



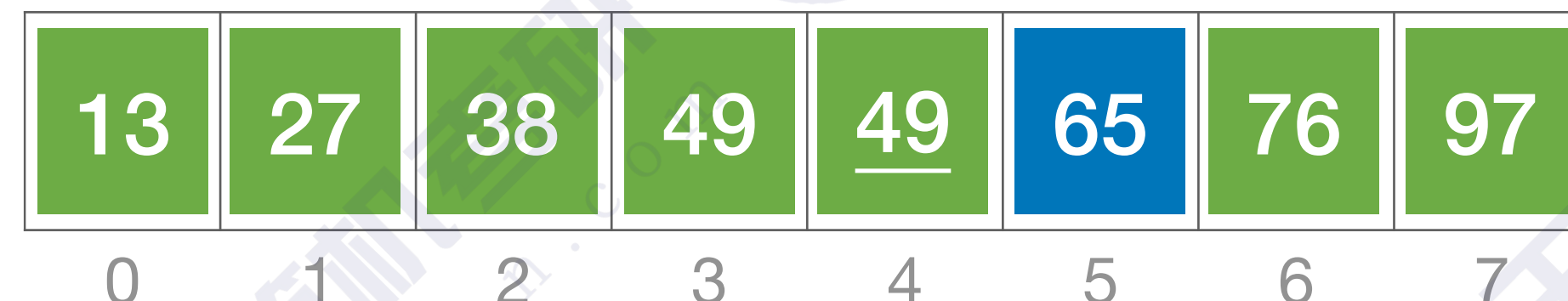
第一层QuickSort处理后:



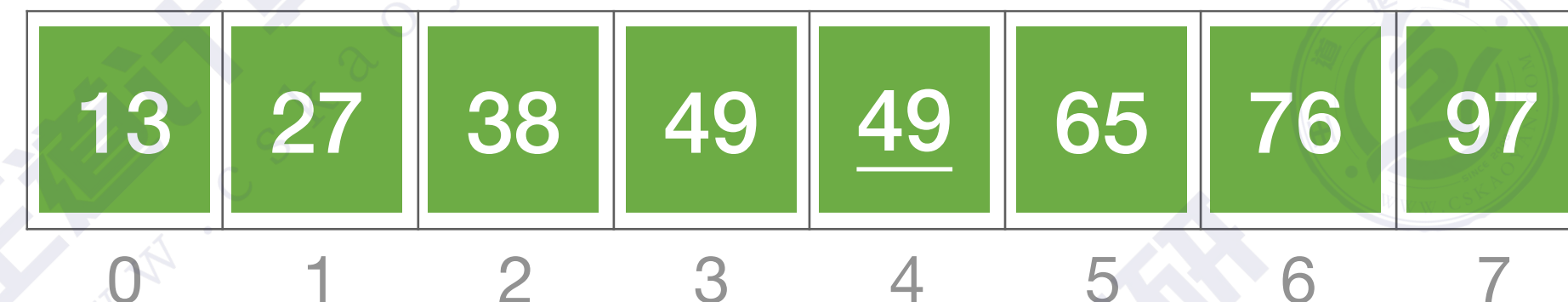
第二层QuickSort处理后:



第三层QuickSort处理后:



第四层QuickSort处理后:



空间复杂度=O(递归层数)

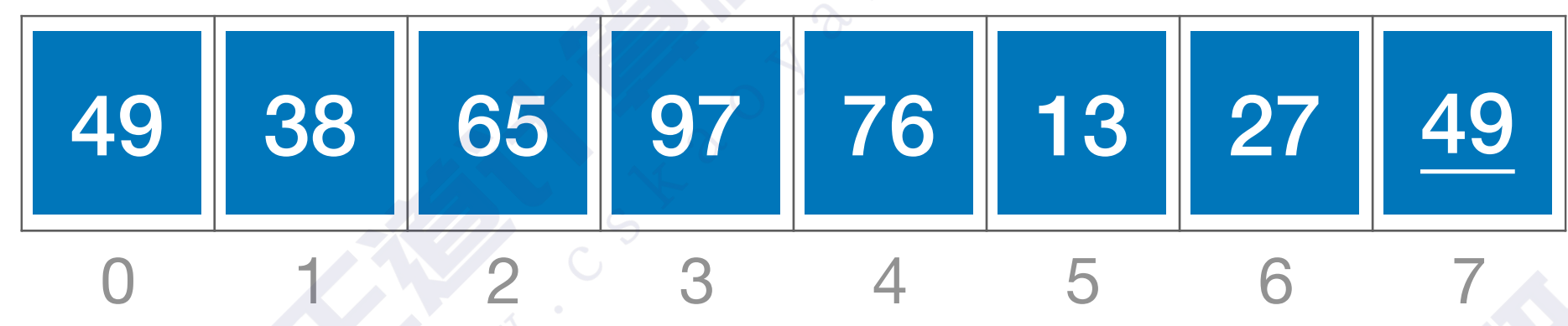
第四层
QuickSort

$l=5, h=5$
#98, $l=4, h=5$ $p=4$
#97, $l=4, h=7$ $p=6$
#98, $l=0, h=7$ $p=3$

递归工作栈

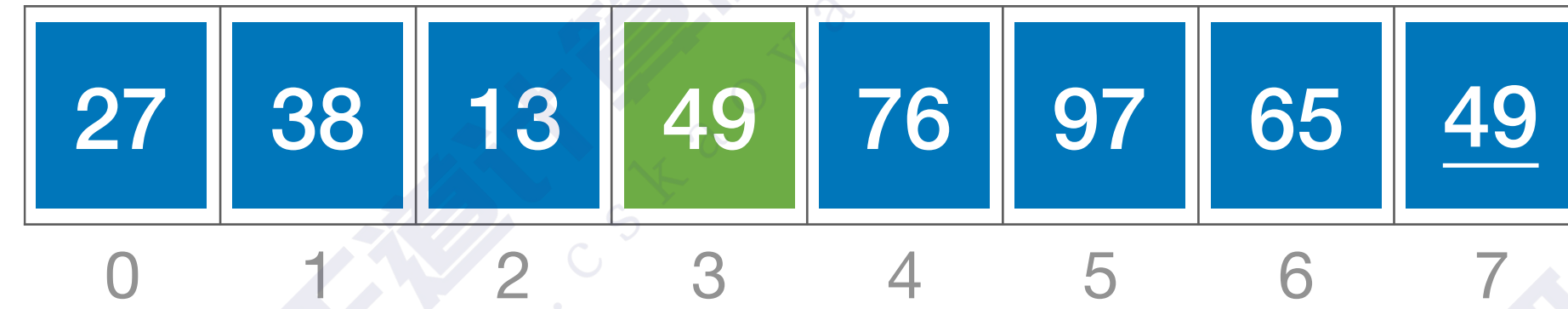
算法效率分析

初始序列



算法效率分析

第一层QuickSort处理后:

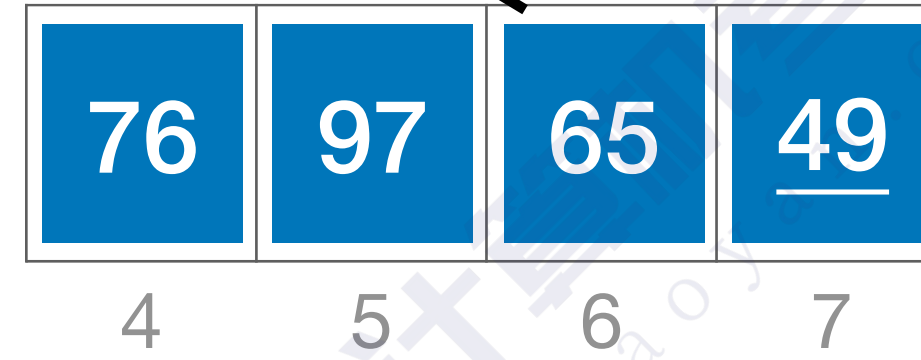
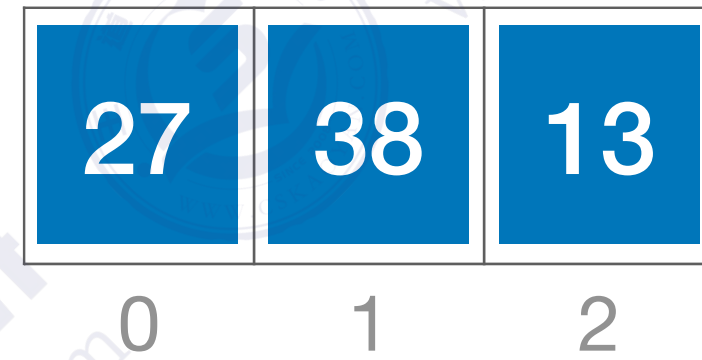


算法效率分析

第一层QuickSort处理后:

49

第二层QuickSort
要处理的部分:



算法效率分析

第一层QuickSort处理后:

49

第二层QuickSort处理后:

13	27	38
----	----	----

0 1 2

49	65	76	97
----	----	----	----

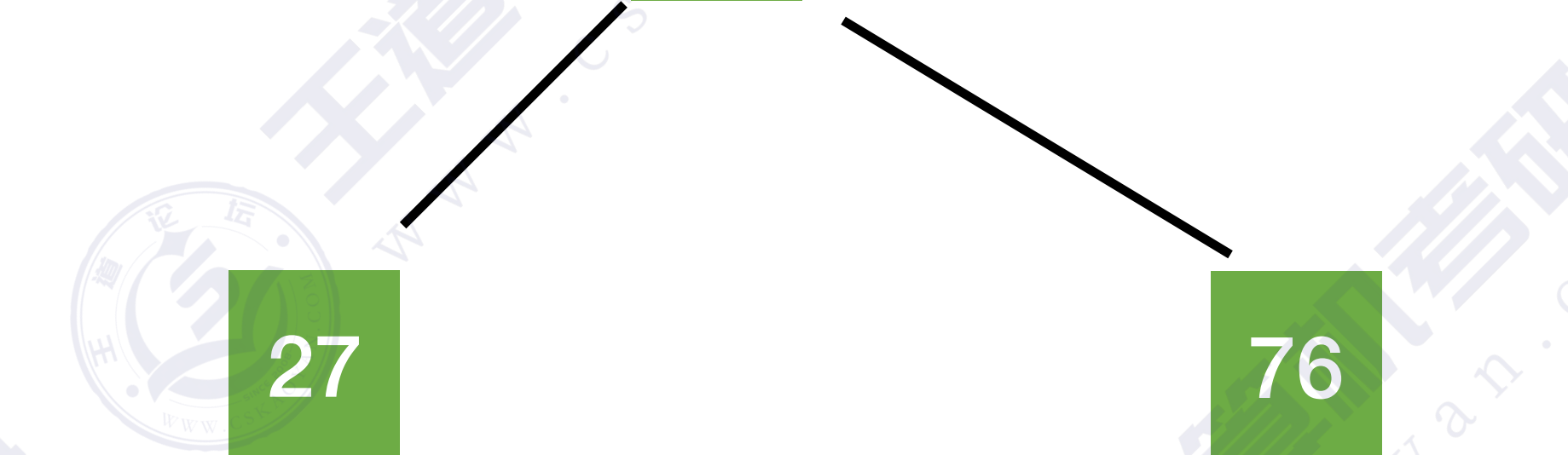
4 5 6 7

算法效率分析

第一层QuickSort处理后:



第二层QuickSort处理后:



第三层QuickSort
要处理的部分:



算法效率分析

第一层QuickSort处理后:

49

第二层QuickSort处理后:

27

76

第三层QuickSort处理后:

13

38

49

65

97

0

2

4

5

7

算法效率分析

第一层QuickSort处理后:

49

第二层QuickSort处理后:

27

76

第三层QuickSort处理后:

13

38

49

97

第四层QuickSort
要处理的部分:

65

5

算法效率分析

第一层QuickSort处理后:

49

第二层QuickSort处理后:

27

76

第三层QuickSort处理后:

13

38

49

97

第四层QuickSort处理后:

65

5

算法效率分析

第一层QuickSort处理后:

49

第二层QuickSort处理后:

27

76

第三层QuickSort处理后:

13

38

49

97

第四层QuickSort处理后:

65

把n个元素组织成二叉树，二叉树的层数就是递归调用的层数

n个结点的二叉树
最小高度 = $\lfloor \log_2 n \rfloor + 1$
最大高度 = n

算法效率分析

n个结点的二叉树

最小高度 = $\lfloor \log_2 n \rfloor + 1$

最大高度 = n

时间复杂度 = $O(n \times \text{递归层数})$

空间复杂度 = $O(\text{递归层数})$

最好时间复杂度 = $O(n \log_2 n)$

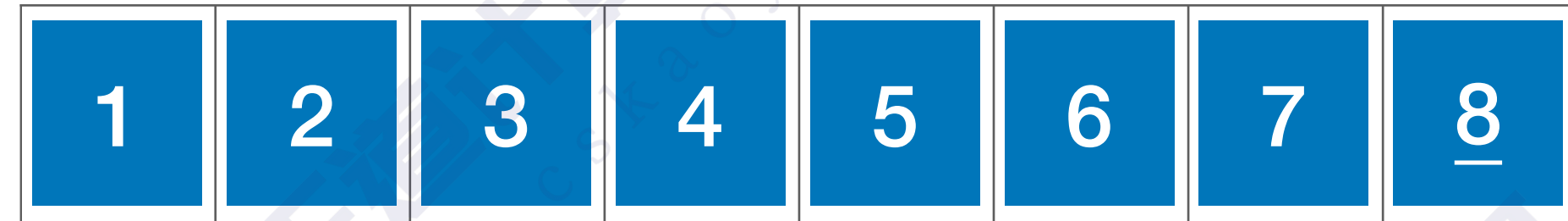
最坏时间复杂度 = $O(n^2)$

最好空间复杂度 = $O(\log_2 n)$

最坏空间复杂度 = $O(n)$

最坏的情况

初始序列



0

1

2

3

4

5

6

7



low



high

最坏的情况

第一层QuickSort处理后:



最坏的情况

第一层QuickSort处理后:



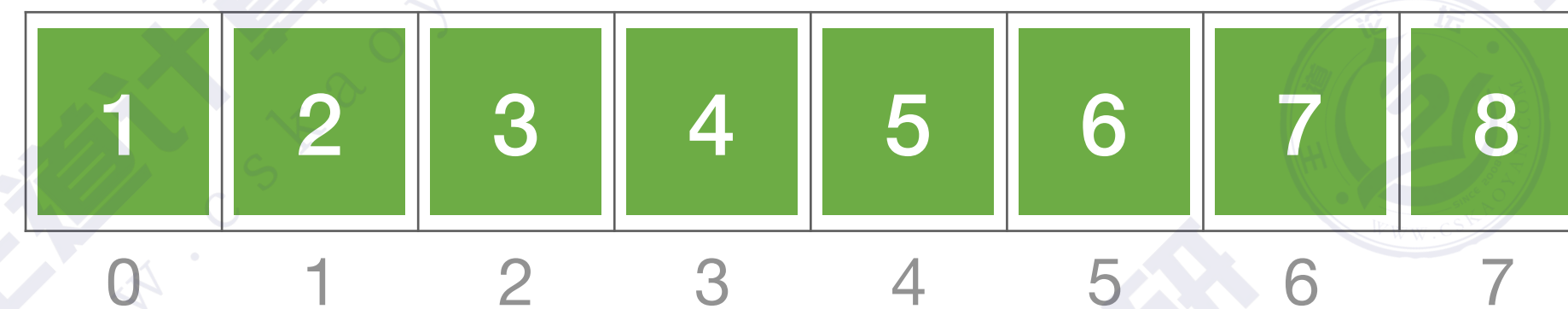
第二层QuickSort处理后:



第三层QuickSort处理后:



第8层QuickSort处理后:



若每一次选中的“**枢轴**”将待排序序列划分为**很不均匀**的两个部分，则会导致递归深度增加，算法**效率变低**

若初始序列**有序或逆序**，则快速排序的**性能最差**（因为每次选择的都是最靠边的元素）

比较好的情况

初始序列

49	38	65	97	76	13	27	<u>49</u>
0	1	2	3	4	5	6	7

第一层QuickSort处理后:

27	38	13	49	76	97	65	<u>49</u>
0	1	2	3	4	5	6	7

第二层QuickSort处理后:

13	27	38	49	<u>49</u>	65	76	97
0	1	2	3	4	5	6	7

第三层QuickSort处理后:

13	27	38	49	<u>49</u>	65	76	97
0	1	2	3	4	5	6	7

第四层QuickSort处理后:

13	27	38	49	<u>49</u>	65	76	97
0	1	2	3	4	5	6	7

若每一次选中的“**枢轴**”将待排序序列划分为**均匀**的两个部分，则递归深度最小，算法**效率最高**

快速排序算法优化思路：尽量选择可以把数据中分的枢轴元素。

eg：①选头、中、尾三个位置的元素，取中间值作为枢轴元素；②随机选一个元素作为枢轴元素

算法效率分析

时间复杂度= $O(n \times \text{递归层数})$

空间复杂度= $O(\text{递归层数})$

最好时间复杂度= $O(n \log_2 n)$

最坏时间复杂度= $O(n^2)$

每次选的枢轴元素都能将序列划分成均匀的两部分

最好空间复杂度= $O(\log_2 n)$

最坏空间复杂度= $O(n)$

若序列原本就有序或逆序，则时、空复杂度最高（可优化，尽量选择可以把数据中分的枢轴元素。）

厉害厉害



快速排序是所有内部排序算法中平均性能最优的排序算法

平均时间复杂度= $O(n \log_2 n)$

稳定性



2	2	1
---	---	---



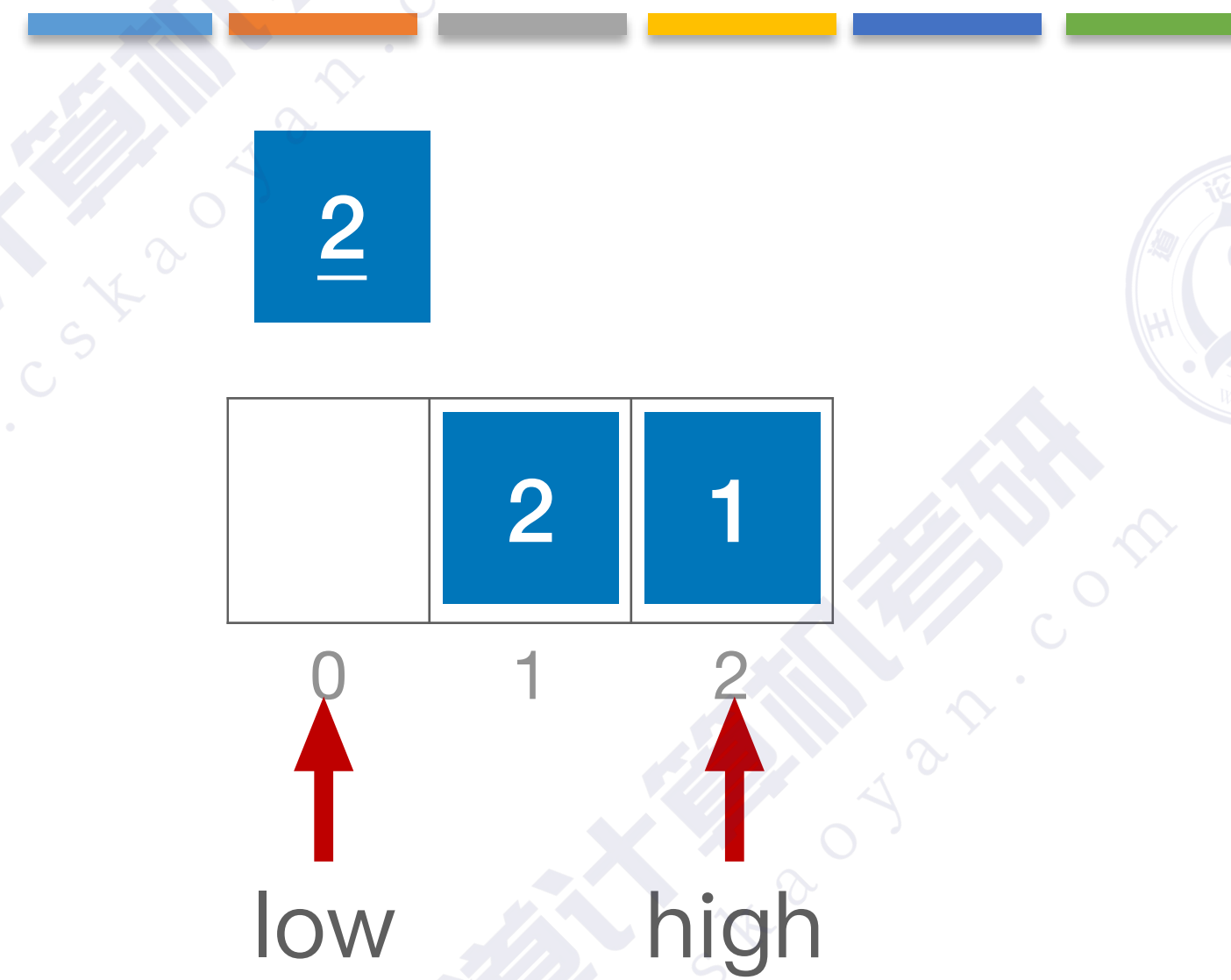
low

1



high

稳定性



稳定性



2

1	2	
---	---	--

0

low

1

2

high

稳定性



2

1	2	
---	---	--

0

1

2

low high

稳定性



2

1	2	
---	---	--

0

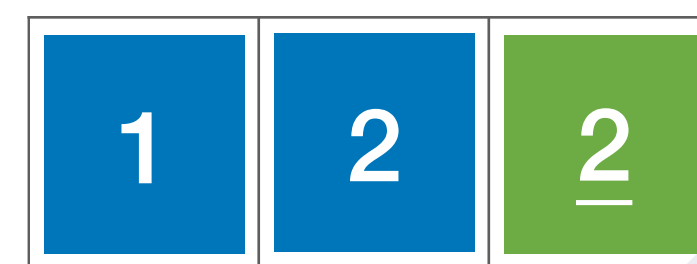
1

2



l
high

稳定性



0

1

2



l
bigh

稳定性



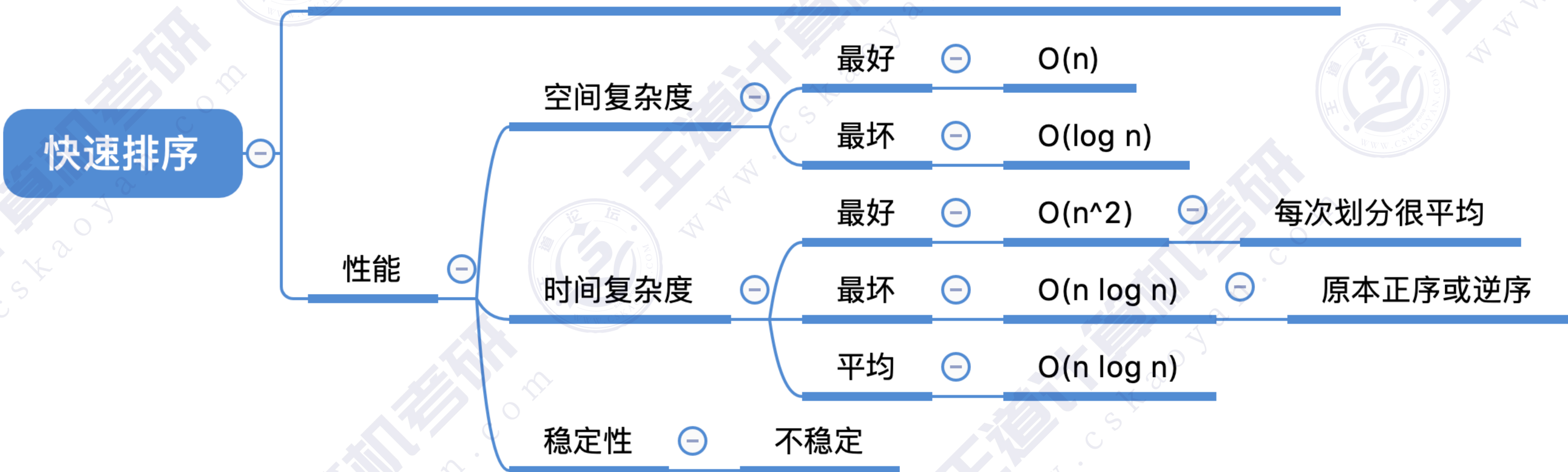
0 1 2

不稳定!

知识回顾与重要考点

算法思想：在待排序表 $L[1...n]$ 中任取一个元素 $pivot$ 作为枢轴（或基准，通常取首元素），通过一趟排序将待排序表划分为独立的两部分 $L[1...k-1]$ 和 $L[k+1...n]$ ，使得 $L[1...k-1]$ 中的所有元素小于 $pivot$ ， $L[k+1...n]$ 中的所有元素大于等于 $pivot$ ，则 $pivot$ 放在了其最终位置 $L(k)$ 上，这个过程称为一次“划分”。然后分别递归地对两个子表重复上述过程，直至每部分内只有一个元素或空为止，即所有元素放在了其最终位置上。

算法表现主要取决于递归深度，若每次“划分”越均匀，则递归深度越低。
“划分”越不均匀，递归深度越深



注：408原题中说，对所有尚未确定最终位置的所有元素进行一遍处理称为“一趟”排序，因此一次“划分” $\neq$ 一趟排序。
一次划分可以确定一个元素的最终位置，而一趟排序也许可以确定多个元素的最终位置。